

SENIOR INTERVIEW PREPARATION

Speech, Audio, and Agentic Voice AI System Design

Production-grade interview preparation with mock questions and real-world cases from OpenAI, Amazon, Google, ElevenLabs, Meta, Spotify, and Microsoft.

10 chapters • 80 mock interview Q&A • 20 system-design case studies • 3 runnable labs
• a runnable end-to-end voice-agent codebase



Crack the Speech & Audio AI Engineering Interview

Hear · Understand · Speak — A Production-Grade Interview Handbook for Speech, Audio & Agentic Voice AI Engineers

First Edition — 2026.

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, or other electronic or mechanical methods — without the prior written permission of the author, except in the case of brief quotations embodied in reviews and certain other non-commercial uses permitted by copyright law.

Model names, benchmark numbers, tool names, and framework APIs change frequently; verify current details before relying on them in an interview or in production. Company and product names are trademarks of their respective owners and are used here for identification and commentary only. This book is independent and is not affiliated with, authorized by, or endorsed by any of them. Code samples are provided “as is” without warranty of any kind.

Author: AI Engineering Insider

Web: aiengineeringinsider.com

Newsletter: aiengineeringinsider.substack.com/subscribe

How to Use This Handbook

Subscribe → aiengineeringinsider.substack.com/subscribe

This handbook is built for one outcome: walking into a Speech, Audio, or Agentic Voice AI interview and answering like a senior engineer who has actually shipped these systems. It assumes you already know Python and the basics of deep learning, and it focuses on the reasoning, trade-offs, and production realities that interviewers probe for.

Chapters 1–8 follow the same predictable structure so you can study efficiently (Chapter 9 instead collects 20 worked system-design case studies):

- **Overview** — a technically precise but beginner-friendly explanation of the topic, its core concepts, and how it fits the end-to-end voice stack.
- **Production Code** — realistic, commented Python that connects directly to the chapter and to the book’s running voice-agent project.
- **Comparison Tables** — the models, metrics, and design choices an interviewer expects you to recall by name.
- **10 Mock Interview Q&A** (Chapters 1–8, 80 in total) — senior, system-design, production, and debugging questions with detailed answers. Every question is listed in the Table of Contents so you can drill specific topics. Chapter 9 replaces the Q&A with 20 end-to-end case studies.

The three pillars — **Hear** (ASR), **Understand** (SLU/NLU), and **Speak** (TTS) — are never independent in production. A voice agent chains them under a latency budget: capture and clean audio, turn it into meaning, decide what to do, and speak back. Study each pillar deeply, but always reason about the end-to-end pipeline, because that is what system-design interviews test.

Pair every chapter with the companion `voice_agent/` Python project shipped with this book. Read the chapter, then open the module it explains (`features.py` , `asr.py` , `nlu.py` , `tts.py` , `pipeline.py` , and so on) and run `python -m voice_agent.pipeline --demo` . Seeing the code execute end-to-end is the fastest way to make the concepts stick.

Currency. Model versions, benchmark numbers, and prices reflect mid-2025 and move quickly. Treat any specific figure as a starting point and re-check the current model card or pricing before quoting it in an interview or committing it to a design. Time-sensitive claims are dated inline (for example, “as of mid-2025”), and the Chapter 9 estimates are

labeled illustrative.

Contents

Subscribe → aiengineeringinsider.substack.com/subscribe

How to Use This Handbook	iii
The Handbook at a Glance	x
1 Audio Foundations & Signal Processing	1
1.1 Overview	1
1.1.1 From Time Domain to Frequency Domain	2
1.1.2 Perceptual Features: Mel and MFCC	2
1.2 Architecture Diagram: The Audio Feature Pipeline	2
1.3 Production Code: Feature Extraction	3
1.4 Reference Tables	4
1.4.1 Window Functions	4
1.4.2 Which Feature for Which Model	4
1.5 Mock Interview Questions & Answers	5
Why is 16 kHz the standard sample rate for speech models when music uses 44.1 kHz?	5
Explain aliasing. How does it occur and how do you prevent it?	5
Walk me through the STFT. What trade-off does the window length control?	5
Why convert a spectrogram to the Mel scale and then take the log?	6
When would you use MFCCs over log-Mel spectrograms in 2025?	6
A teammate reports that fine-tuning destroyed accuracy on loud recordings but not quiet ones. What do you check?	6
How would you design an augmentation strategy for a 100-hour domain ASR dataset?	7
What is spectral leakage and why does the choice of window function matter?	7
Design the audio ingestion layer for a service that accepts user uploads in arbitrary formats. What are the failure modes?	7
What is the difference between magnitude, power, and log-power spectrograms, and where is phase?	7
References	8
2 ASR I — Acoustic Modeling	9
2.1 Overview	9
2.1.1 Self-Supervised Acoustic Encoders	9
2.1.2 The Conformer	10
2.2 Architecture Diagram: CTC vs RNN-Transducer	10
2.3 Production Code: Wav2Vec2 CTC Inference	10
2.4 Comparison Tables	11
2.4.1 ASR Paradigm Trade-offs	12
2.4.2 Self-Supervised Encoders	12
2.5 Mock Interview Questions & Answers	12
Explain the CTC loss and the role of the blank token.	12
Why does the industry use RNN-Transducer for on-device streaming ASR instead of CTC or attention models?	12
Walk me through how Wav2Vec2 achieves strong ASR with only ten minutes of labeled data.	13
What problem does the Conformer solve that a pure Transformer encoder does not?	13
Your CTC model outputs phonetically correct but misspelled words. How do you fix it without retraining?	13
Compare HuBERT and Wav2Vec2. When would you pick one over the other?	13
Design an ASR system for a call center handling 8 kHz telephone audio across ten languages. Walk me through the choices.	14
What is the alignment problem in ASR and how do CTC, attention, and RNN-T each address it?	14
How would you debug an ASR model whose training loss decreases but validation WER stays flat?	14
Why did end-to-end neural ASR replace the GMM-HMM pipeline, and what parts of the old pipeline survive?	15
References	15
3 ASR II — Whisper, Diarization & Evaluation	16
3.1 Overview	16
3.1.1 Diarization: Who Spoke When	16
3.1.2 Keyword Spotting and Wake Words	16

3.1.3	Evaluation	17
3.2	Architecture Diagram: A Transcription + Diarization Service	17
3.3	Production Code: Whisper Backend with Word Timestamps	17
3.4	Comparison Tables	18
3.4.1	Whisper Model Sizes	19
3.4.2	ASR & Diarization Metrics	19
3.5	Mock Interview Questions & Answers	19
	How is Whisper trained, and why is it so robust without fine-tuning?	19
	A user reports Whisper outputs “Thank you for watching” on a silent recording. Diagnose and fix.	20
	Design a meeting-transcription product. How do you get accurate speaker-labeled transcripts?	20
	Explain WER and its limitations. What does a WER of 15% actually mean?	20
	What are the hard constraints in wake-word detection, and how do you meet them?	20
	When would you fine-tune Whisper versus use it zero-shot, and how would you fine-tune efficiently?	21
	Your diarization system reports DER of 8% but users complain labels are wrong. Reconcile this.	21
	How do you build a streaming (live-caption) transcription system from models that are mostly offline?	21
	What is Real-Time Factor and why can a model with RTF below 1.0 still fail in production?	21
	Compare faster-whisper, WhisperX, and whisper.cpp. Which would you deploy and why?	22
	References	22
4	Spoken Language Understanding	23
4.1	Overview	23
4.1.1	The LLM Shift	23
4.2	Architecture Diagram: SLU in the Voice Agent	24
4.3	Production Code: Intent + Slot Extraction with an LLM	24
4.4	Comparison Tables	25
4.4.1	SLU Approaches	26
4.4.2	Auxiliary SLU Tasks	26
4.5	Mock Interview Questions & Answers	26
	Cascaded versus end-to-end SLU: how do you choose?	26
	How do you stop ASR errors from corrupting downstream understanding?	27
	When should you use an LLM for intent classification, and when is a small fine-tuned classifier better?	27
	Why run punctuation restoration and inverse text normalization, and where do they sit?	27
	Design dialogue state tracking for a multi-turn flight-booking agent.	27
	How would you add emotion recognition, and what are the pitfalls?	28
	An LLM-based NLU sometimes returns invalid JSON or hallucinated slots. How do you make it production-safe?	28
	What is the difference between intent classification and slot filling, and how are they trained jointly?	28
	Design the NLU layer for a voice agent that must respond within 300 ms of end-of-speech.	28
	What is an audio-LLM and how does it change SLU?	29
	References	29
5	Text-to-Speech & Voice Synthesis	30
5.1	Overview	30
5.1.1	Autoregressive vs Parallel Acoustic Models	30
5.1.2	Vocoders	30
5.1.3	The Frontier: Voice Cloning, Diffusion, and LM-based TTS	30
5.2	Architecture Diagram: Neural TTS	31
5.3	Production Code: Streaming TTS Interface	31
5.4	Comparison Tables	32
5.4.1	Acoustic Models	32
5.4.2	Vocoders	33
5.5	Mock Interview Questions & Answers	33
	Walk me through a modern neural TTS pipeline end to end.	33
	Why did the field move from autoregressive Tacotron 2 to parallel FastSpeech 2?	33
	Your TTS sounds robotic and metallic. How do you debug it?	34
	How does zero-shot voice cloning work, and what are the risks?	34
	How do you control prosody, making the agent emphasize a word or ask a question?	34
	Design a real-time TTS service for a voice agent with a strict latency budget.	34
	What is a neural audio codec and why is it central to modern TTS?	35
	How do you evaluate TTS quality, both subjectively and objectively?	35
	Compare diffusion-based and language-model-based TTS with the classic two-stage cascade.	35
	The text front-end mispronounces “Dr.”, “2024”, and a product name. How do you fix pronunciation robustly?	36
	References	36

6	Production Voice AI I — Codecs, VAD & Streaming	37
6.1	Overview	37
6.1.1	Voice Activity Detection and Endpointing	37
6.1.2	Neural Audio Codecs	37
6.1.3	Streaming Speech-to-Speech	37
6.2	Architecture Diagram: Real-Time Voice Loop	38
6.3	Production Code: Streaming VAD with Endpointing	38
6.4	Comparison Tables	39
6.4.1	Latency Budget (target ≈ 300 ms)	39
6.4.2	VAD & Codec Building Blocks	40
6.5	Mock Interview Questions & Answers	40
	What is the end-to-end latency budget for a voice agent, and how do you spend it?	40
	Explain endpointing. Why is a fixed silence timeout insufficient?	40
	How do you implement barge-in so a user can interrupt the agent?	41
	What is a neural audio codec and where does it fit in a production voice system?	41
	Contrast cascaded voice pipelines with end-to-end speech-to-speech models.	41
	Why must you run a VAD before Whisper, and what else does the VAD buy you?	41
	Design the audio transport for a cloud voice agent over an unreliable mobile network.	42
	A user complains the agent “talks over them.” Walk through your debugging.	42
	How do partial/streaming ASR results help latency, and what is the downside?	42
	What is acoustic echo cancellation and why is it essential for a hands-free voice agent?	43
	References	43
7	Production Voice AI II — Safety, Eval & Deployment	44
7.1	Overview	44
7.1.1	Evaluation Beyond a Single Number	44
7.1.2	On-Device Optimization	44
7.1.3	Observability	45
7.2	Architecture Diagram: Voice MLOps Loop	45
7.3	Production Code: Audio Watermark Verification + Quality Gate	45
7.4	Comparison Tables	46
7.4.1	Audio Quality & Verification Metrics	46
7.4.2	On-Device Optimization Techniques	47
7.5	Mock Interview Questions & Answers	47
	How would you defend a voice-authentication system against spoofing?	47
	What is audio watermarking and why does it matter for generative voice?	47
	Design the evaluation strategy for an ASR model before and after deployment.	48
	You need Whisper-quality ASR on a phone with no network. How do you get there?	48
	How do you detect and respond to data drift in a deployed voice system?	48
	What are the privacy and compliance considerations for storing voice data?	49
	Walk me through quantizing a speech model. What breaks and how do you catch it?	49
	What should you log and monitor in a production voice agent?	49
	Design a canary/A-B rollout for a new ASR model. What do you measure and when do you roll back?	49
	How do you measure speech-enhancement or denoising quality, and what is the catch?	50
	References	50
8	Agentic Voice AI & Specialized Topics	51
8.1	Overview	51
8.1.1	Tool Calling and Grounding	51
8.1.2	Specialized Topics	51
8.2	Architecture Diagram: The Full Voice Agent	52
8.3	Production Code: The Orchestrator (Capstone)	52
8.4	Comparison Tables	54
8.4.1	Speech Translation Approaches	54
8.4.2	Multilingual / Frontier Models	54
8.5	Mock Interview Questions & Answers	54
	Design a complete voice agent from microphone to speaker. What are the components and the hard parts?	54
	How does tool calling work in a voice agent, and how do you keep it safe and fast?	55
	How do you reduce hallucination when a voice agent answers factual questions?	55
	Cascaded versus direct speech-to-speech translation for a real-time interpreter. Which and why?	55
	How would you give a voice agent memory across a long conversation and across sessions?	56
	What changes when you move from a turn-based agent to a full-duplex one?	56
	Your voice agent works in English but fails on code-switching and accented speech. How do you fix it?	56

Design observability and evaluation for a deployed voice agent. What does success look like? . . .	57
Walk through what happens, stage by stage, in the 800 ms after a user says “cancel my 3pm meeting.”	57
What are the biggest open challenges in agentic voice AI today?	57
References	58
9 Real-World System Design Case Studies	59
9.1 The Staff-Level System-Design Framework	59
9.2 Case 1 — OpenAI Whisper: Robust Batch ASR at Scale	60
Brief: transcribe arbitrary audio in 99 languages, robust to noise, accent, and domain shift, with no per-customer tuning.	60
9.3 Case 2 — Amazon Alexa: Far-Field Voice Assistant	61
Brief: an always-on, far-field smart-speaker assistant that wakes on “Alexa,” understands a request, and acts, across millions of devices.	61
9.4 Case 3 — Apple Siri: Privacy-First On-Device Speech	62
Brief: a phone assistant and dictation engine that is fast, works offline, personalizes to the user, and keeps audio private.	62
9.5 Case 4 — Google Live Caption: Streaming On-Device RNN-T	63
Brief: caption any audio playing on a phone, videos, calls, podcasts, in real time, offline, with low battery cost.	63
9.6 Case 5 — Zoom / Otter.ai: Meeting Transcription & Diarization	64
Brief: live captions during a meeting plus an accurate speaker-attributed transcript and an LLM summary with action items afterward.	64
9.7 Case 6 — ElevenLabs: Zero-Shot Voice Cloning TTS	65
Brief: generate highly natural, emotional speech in any voice, including cloning a speaker from seconds of audio, for dubbing, audiobooks, and agents, via a low-latency API.	65
9.8 Case 7 — Amazon Polly / Google Cloud TTS: Cloud TTS at Scale	66
Brief: a multi-tenant cloud API turning text into natural speech in dozens of languages and voices, with SSML control, at massive throughput and predictable latency.	66
9.9 Case 8 — Azure Custom Neural Voice: Brand Voice Cloning	67
Brief: let an enterprise build a custom, on-brand synthetic voice from a consenting professional speaker, then synthesize unlimited content in that voice, responsibly.	67
9.10 Case 9 — Meta SeamlessM4T: Speech-to-Speech Translation	68
Brief: translate speech in one language into speech (or text) in another across many pairs, preserving expressiveness, with a near-real-time option.	68
9.11 Case 10 — Descript Overdub: Voice Editing for Creators	69
Brief: edit recorded speech by editing its transcript, fixing a flubbed word generates new audio in the creator’s cloned voice, seamlessly stitched into the original.	69
9.12 Case 11 — Shazam: Audio Fingerprinting at Planet Scale	70
Brief: identify a song from a few noisy seconds captured on a phone in a bar, matching against tens of millions of tracks in under a second.	70
9.13 Case 12 — Spotify: Music Recommendation & Audio Understanding	71
Brief: recommend the next track and build personalized playlists for hundreds of millions of users over tens of millions of tracks, including brand-new songs with no history.	71
9.14 Case 13 — YouTube Content ID: Copyright Matching at Scale	72
Brief: scan every upload against rights-holder reference audio/video and flag matches, even partial, pitched, or noisy reuse, at hundreds of hours uploaded per minute.	72
9.15 Case 14 — Suno: Text-to-Music Generation	73
Brief: generate a complete, radio-quality song, vocals, lyrics, instrumentation, from a short text prompt in about a minute.	73
9.16 Case 15 — Moises / Spleeter: Music Source Separation	73
Brief: split a finished song into stems, vocals, drums, bass, other — for remixing, transcription, or karaoke.	73
9.17 Case 16 — Krisp: Real-Time Noise Cancellation	74
Brief: remove background noise from outgoing mic and incoming speaker audio of any call, in real time, on the user’s device, without touching the conferencing app.	74
9.18 Case 17 — Zoom / Discord: The Real-Time Audio Transport	75
Brief: carry low-latency, intelligible voice between many participants over unreliable networks, with echo cancellation and graceful degradation under loss.	75
9.19 Case 18 — Pindrop: Voice Anti-Spoofing & Fraud Detection	76
Brief: in a bank call center, authenticate genuine callers and flag fraudsters, including deepfakes and replay, in real time, with low friction for real customers.	76
9.20 Case 19 — Meta AudioSeal: Watermarking & Deepfake Provenance	77
Brief: make AI-generated speech detectable as synthetic by embedding an inaudible, robust watermark at generation time, and detect it (even in an edited clip) at scale.	77
9.21 Case 20 — OpenAI Realtime API: Full-Duplex Voice Agents	78

Brief: power a production voice agent that listens and speaks over a live audio stream, calls tools, handles interruptions, and answers phone calls, with human-like latency.	78
References	79
10 Hands-On Labs	80
10.1 Before You Start	80
10.2 Lab 1 — Features, the Frequency Domain, and Why Phase Matters	80
10.3 Lab 2 — Measuring ASR Error: WER Is Not CER	82
10.4 Lab 3 — Endpointing Is Your Latency Budget	83
10.5 Where to Take These Next	84

The Handbook at a Glance

Subscribe → aiengineeringinsider.substack.com/subscribe

Ch.	Title	Main Focus
1	Audio Foundations & Signal Processing	Sampling, Nyquist, FFT/STFT, Mel/MFCC features, augmentation, and the audio deep-learning architecture families. The vocabulary every later chapter assumes.
2	ASR I: Acoustic Modeling	Classical GMM-HMM, CTC, RNN-Transducer, Wav2Vec2/HuBERT self-supervision, and the Conformer encoder that dominates production ASR.
3	ASR II: Whisper, Diarization & Evaluation	Whisper architecture and fine-tuning, speaker verification, “who spoke when” diarization, wake-word spotting, and WER/CER/RTF evaluation.
4	Spoken Language Understanding	Intent classification, entity extraction, end-to-end SLU, dialogue state tracking, emotion, ITN/punctuation, and LLM-based understanding.
5	Text-to-Speech & Voice Synthesis	TTS evolution, acoustic models (Tacotron2, FastSpeech2), neural vocoders, prosody control, voice cloning/conversion, and diffusion/LM-based TTS.
6	Production Voice AI I: Codecs, VAD & Streaming	Neural audio codecs, voice activity detection, turn-taking, streaming speech-to-speech, and the real-time latency budget.
7	Production Voice AI II: Safety, Eval & Deployment	Anti-spoofing, watermarking, subjective/objective audio evaluation, on-device optimization, and MLOps/observability for voice systems.
8	Agentic Voice AI & Specialized Topics	Full-duplex voice agents, tool calling, barge-in, speech translation, multilingual ASR, and the capstone end-to-end voice-agent build.
9	Real-World System Design Case Studies	Twenty production systems from top companies (Whisper, Alexa, Siri, ElevenLabs, Shazam, Spotify, YouTube Content ID, Krisp, Pindrop, OpenAI Real-time, and more), each with brief, architecture, diagram, and trade-off.
10	Hands-On Labs	Three CPU-only labs (features/DSP and why phase matters, WER vs CER, endpointing latency) with runnable code and the <i>real</i> measured output, reproducible without a GPU.

If you are interviewing in two weeks, read Chapters 1–3 first (they anchor 60% of technical screens), then skim 4–8 for the system-design round. If you have more time, work the running project chapter by chapter.

Audio Foundations & Signal Processing

Subscribe → aiengineeringinsider.substack.com/subscribe

What an interviewer is really testing: do you understand what a model actually sees when you hand it “audio”? Sampling, the frequency domain, and perceptual features are the vocabulary every later chapter assumes.

1.1 Overview

Every speech and audio model, no matter how modern, is ultimately a function of numbers that came from a microphone. Sound is a continuous pressure wave. To make it computable we **sample** it: measure its amplitude at a fixed rate and store each measurement at a fixed **bit depth**. The result is **PCM** (Pulse-Code Modulation), a one-dimensional array of integers that almost every audio library loads natively. Speech models standardize on 16,000 samples per second (16 kHz); music and general audio often use 44,100 Hz or 48,000 Hz.

The single most important theorem in the field is **Nyquist–Shannon**: to represent a frequency f without distortion, you must sample at a rate of at least $2f$. Human speech energy concentrates below 8 kHz, which is exactly why 16 kHz is the speech standard, it captures everything that matters with no waste. Sample above the Nyquist limit and high frequencies “fold back” as **aliasing**, corrupting the signal permanently; the fix is an anti-aliasing low-pass filter *before* sampling.

- **Sample rate**, samples per second (Hz). 16 kHz for speech, 48 kHz for high-fidelity audio. Resample, never assume.
- **Bit depth**, bits per sample (16-bit integer is standard). Sets the dynamic range (≈ 96 dB for 16-bit).
- **Nyquist–Shannon theorem**, faithful reconstruction requires $f_s \geq 2f_{\max}$.
- **Aliasing**, frequencies above $f_s/2$ alias to lower frequencies; prevented by low-pass filtering before sampling.
- **SNR (signal-to-noise ratio)**, signal power over noise power, in dB. ASR accuracy degrades sharply below ~ 10 dB SNR.

1.1.1 From Time Domain to Frequency Domain

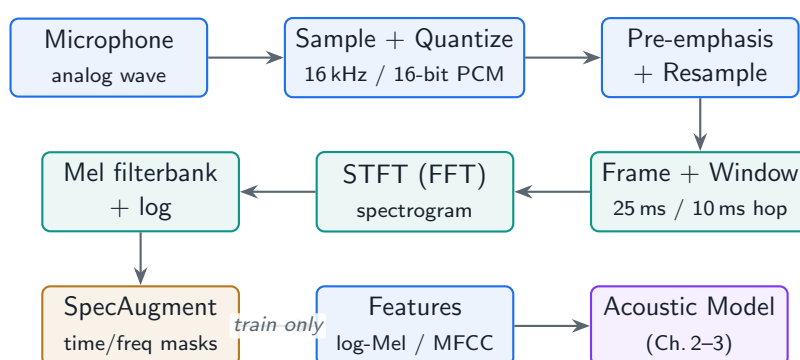
A raw waveform is hard for a model to interpret directly, because meaning in speech lives in *which frequencies are present and how they change over time*. The **Fourier Transform** decomposes a signal into its constituent frequencies. In practice we use the **FFT** (Fast Fourier Transform, $O(N \log N)$) and, for audio, the **STFT** (Short-Time Fourier Transform): slide a short analysis window (e.g. 25 ms) across the signal with a small hop (e.g. 10 ms) and run an FFT on each frame. The result is a two-dimensional time–frequency matrix, the **spectrogram**, which is the primary input format for most audio models.

1.1.2 Perceptual Features: Mel and MFCC

Human hearing is not linear in frequency: we resolve pitch finely at low frequencies and coarsely at high ones. The **Mel scale** encodes this. Passing a power spectrogram through a **Mel filterbank** and taking the logarithm produces the **log-Mel spectrogram**, the de facto standard input to Whisper, TTS acoustic models, and most classifiers. Apply a Discrete Cosine Transform to the log-Mel spectrum and you get **MFCCs**, the compact, decorrelated features that powered classical ASR and still appear in speaker-ID and low-resource pipelines.

1.2 Architecture Diagram: The Audio Feature Pipeline

The diagram below is the mental model to draw on a whiteboard when asked “what happens to audio before it reaches the model?” Everything from Chapter 2 onward plugs into the right edge of this pipeline.



Interviewers love the question “why log-Mel and not raw FFT?” Two reasons: the **Mel filterbank** compresses hundreds of FFT bins into 80 perceptually spaced channels (smaller, less redundant), and the **log** compresses the enormous dynamic range of audio so a network can learn from quiet and loud content alike. Wav2Vec2 and HuBERT are the exception, they learn features from the raw waveform.

1.3 Production Code: Feature Extraction

The function below is the `features.py` module of the book's voice-agent project. It computes 80-bin log-Mel features (the Whisper base/v2 layout; large-v3 uses 128 bins) and, optionally, MFCCs for a classical speaker-ID branch. Note the explicit resampling guard, a mismatched sample rate is the single most common silent bug in audio pipelines.

`voice_agent/features.py`

```
"""Audio feature extraction for the voice-agent pipeline."""
from __future__ import annotations
import numpy as np
import torch
import torchaudio
import torchaudio.transforms as T

TARGET_SR = 16_000  # speech standard; matches Whisper / Wav2Vec2

class FeatureExtractor:
    """Turn a raw waveform into model-ready features.

    Produces 80-bin log-Mel features (Whisper base/v2; v3 uses 128) and, on demand,
    MFCCs for the speaker-ID branch. All audio is force-resampled to 16 kHz
    so downstream models never receive an unexpected sample rate.
    """

    def __init__(self, n_mels: int = 80, n_fft: int = 400, hop: int = 160):
        # 400 samples = 25 ms window, 160 samples = 10 ms hop at 16 kHz.
        self.mel = T.MelSpectrogram(
            sample_rate=TARGET_SR, n_fft=n_fft, hop_length=hop,
            n_mels=n_mels, power=2.0, f_min=0.0, f_max=8000.0,
        )
        self.mfcc = T.MFCC(
            sample_rate=TARGET_SR, n_mfcc=13,
            melkwargs={"n_fft": n_fft, "hop_length": hop, "n_mels": n_mels},
        )

    def _ensure_16k_mono(self, wav: torch.Tensor, sr: int) -> torch.Tensor:
        if wav.dim() == 2:  # (channels, time) -> mono
            wav = wav.mean(dim=0, keepdim=True)
        if sr != TARGET_SR:  # resample guard
            wav = torchaudio.functional.resample(wav, sr, TARGET_SR)
        return wav

    def log_mel(self, wav: torch.Tensor, sr: int) -> torch.Tensor:
        wav = self._ensure_16k_mono(wav, sr)
        mel = self.mel(wav)  # (1, n_mels, T)
        log_mel = torch.log(mel + 1e-6)  # avoid log(0)
        # per-utterance normalization stabilizes training and inference
        return (log_mel - log_mel.mean()) / (log_mel.std() + 1e-6)

    def mfcc_features(self, wav: torch.Tensor, sr: int) -> torch.Tensor:
        wav = self._ensure_16k_mono(wav, sr)
```

```

return self.mfcc(wav)                                # (1, 13, T)

@staticmethod
def snr_db(signal: np.ndarray, noise: np.ndarray) -> float:
    """Estimate SNR in dB - useful for quality gating before ASR."""
    p_sig = np.mean(signal ** 2) + 1e-12
    p_noise = np.mean(noise ** 2) + 1e-12
    return float(10.0 * np.log10(p_sig / p_noise))

```

In production this extractor sits behind the audio I/O layer (Chapter 6) and feeds the ASR encoder (Chapters 2–3). The per-utterance normalization line matters more than it looks: skip it and a fine-tuned model trained on normalized features will quietly lose several points of accuracy on louder or quieter test audio.

The most common production incident in audio ML is a **silent sample-rate mismatch**: a model trained at 16 kHz is fed 8 kHz telephone audio (or 44.1 kHz without resampling). There is no crash, just a large, mysterious accuracy drop. Always assert the sample rate at the boundary of your system.

1.4 Reference Tables

1.4.1 Window Functions

Window	Side-lobe attenuation	Typical use
Hann	−31 dB	General-purpose speech/music (default)
Hamming	−43 dB	Classical ASR feature extraction
Blackman	−58 dB	High-fidelity spectral analysis
Rectangular	−13 dB	Maximum time resolution (rarely used)

1.4.2 Which Feature for Which Model

Model	Input representation
Whisper (base/v2)	80-bin log-Mel, 16 kHz (large-v3 uses 128-bin)
Wav2Vec2 / HuBERT	Raw waveform (model learns its own features)
Classical ASR (Kaldi)	39-dim MFCC + delta + delta-delta
Speaker ID (ECAPA-TDNN)	80-bin log-Mel filterbank
Music tagging	Chroma + spectral contrast + MFCC

Default to **80-bin log-Mel at 16 kHz** for most speech tasks (Whisper large-v3 expects 128 bins), it matches the pretrained checkpoints you are most likely to fine-tune. Only

reach for raw-waveform models (Wav2Vec2) when you have a self-supervised pretraining budget or need the last point of accuracy on a niche domain.

Early on I lost two days to a “broken” model that turned out to be fed 8 kHz telephone audio at a 16 kHz input. No crash, no error, just garbage transcripts and a sinking feeling. Now the first line of every audio service I write asserts the sample rate at the boundary. A mild heresy too: stop grinding MFCC cepstral math for interviews. Outside Kaldi recipes and microcontrollers I have not hand-rolled MFCCs in years, log-Mel and raw waveform won.

1.5 Mock Interview Questions & Answers

Q1.1

Why is 16 kHz the standard sample rate for speech models when music uses 44.1 kHz? By the Nyquist–Shannon theorem, a 16 kHz sample rate faithfully represents frequencies up to 8 kHz. The acoustically important energy in human speech, formants, fricatives, and most consonant cues, lives below 8 kHz, so 16 kHz captures essentially all of it. Sampling higher (e.g. 44.1 kHz) wastes compute and memory: you would nearly triple the sequence length the model must process for no accuracy gain on speech. Music uses 44.1/48 kHz because instruments and cymbals carry meaningful energy up to 20 kHz, the edge of human hearing. The practical takeaway interviewers want: match your sample rate to the bandwidth of the signal, and always resample telephone (8 kHz) or studio (48 kHz) inputs to the rate your model was trained on.

Q1.2

Explain aliasing. How does it occur and how do you prevent it? Aliasing happens when a signal contains frequencies above the Nyquist limit ($f_s/2$) at the moment of sampling. Those high frequencies cannot be represented, so they “fold back” and masquerade as lower frequencies, permanently corrupting the data, no amount of later processing can recover the original. The classic visual analogy is a car wheel appearing to spin backwards on film. Prevention is an anti-aliasing low-pass filter applied in the analog or pre-decimation stage, before or during sampling, that removes energy above $f_s/2$. This is also why naive downsampling by simply dropping samples is wrong: you must low-pass filter first. Libraries like `torchaudio.functional.resample` and `librosa.resample` do this correctly; rolling your own decimation is a common production bug.

Q1.3

Walk me through the STFT. What trade-off does the window length control? The STFT slices the signal into short overlapping frames, applies a window function to each, and runs an FFT per frame, producing a time–frequency matrix. The window length sets the fundamental time–frequency resolution trade-off, a consequence of the uncertainty principle. A long window (say 50 ms) gives fine frequency resolution, you can separate closely spaced

harmonics, but poor time resolution, so fast transients smear. A short window (say 10 ms) localizes events precisely in time but blurs frequency. For speech we typically use a 25 ms window with a 10 ms hop: long enough to resolve formants, short enough to track phoneme transitions. The hop length (overlap) separately controls how many frames per second the downstream model sees, which affects sequence length and latency.

Q1.4

Why convert a spectrogram to the Mel scale and then take the log? Two independent perceptual motivations. The Mel scale addresses *frequency* perception: humans discriminate pitch finely at low frequencies and coarsely at high ones, so a Mel filterbank packs many narrow filters down low and fewer wide filters up high. This both matches perception and compresses, say, 201 FFT bins into 80 Mel channels, reducing redundancy and parameter count. The log addresses *amplitude* perception: loudness perception is roughly logarithmic, and audio has a huge dynamic range. Taking the log compresses that range so a network can learn from both quiet and loud regions without the loud parts dominating the gradient. Together they turn raw spectra into the compact, well-conditioned log-Mel features that Whisper and most TTS acoustic models consume.

Q1.5

When would you use MFCCs over log-Mel spectrograms in 2025? Rarely for deep models, often for classical or resource-constrained ones. MFCCs add a DCT step on top of log-Mel, which decorrelates the channels and compacts energy into the first 13 coefficients. Neural networks do not need that decorrelation, they learn their own transformations, and the DCT throws away locality that CNNs exploit, so modern deep ASR uses log-Mel or raw waveform. MFCCs remain useful in three cases: classical GMM-HMM systems and Kaldi recipes that assume them; very low-compute settings (keyword spotting on microcontrollers) where 13 coefficients beat 80 channels; and as cheap, compact features for clustering or speaker-ID initialization (HuBERT's first iteration clusters MFCCs). The senior-level point: the DCT is a hand-designed decorrelator that deep learning made largely obsolete.

Q1.6

A teammate reports that fine-tuning destroyed accuracy on loud recordings but not quiet ones. What do you check? This pattern points at a normalization or clipping mismatch, not the model architecture. First, check for **clipping**: loud recordings may have hit the 0 dBFS ceiling and been hard-limited at capture, which no model can recover, inspect the waveform for flat-topped peaks. Second, check feature normalization: if training used per-utterance mean/variance normalization but inference does not (or uses a fixed global scale), louder utterances will land outside the distribution the model saw. Third, check the input gain / automatic gain control in the capture path. Fourth, verify resampling did not introduce intermodulation on hot signals. The fix is usually to apply the exact same normalization at inference as in training and to add a loudness-normalization (e.g. to -23 LUFS) or peak-limiting stage before feature extraction.

Q1.7

How would you design an augmentation strategy for a 100-hour domain ASR dataset? The goal is to simulate the variability of deployment without collecting more data. I would layer four families. (1) **SpecAugment**, time and frequency masking directly on the log-Mel features; it is cheap, applied on-the-fly, and the single highest-ROI ASR augmentation. (2) **Speed/tempo perturbation** at 0.9x and 1.1x, which triples effective data and adds speaker-rate diversity. (3) **Noise and room simulation**, convolve with measured room impulse responses and add MUSAN or domain noise at sampled SNRs (5–20 dB) to match the target acoustic environment. (4) **Gain perturbation** of a few dB. I would keep a clean validation set, apply augmentation only at training time, and tune SpecAugment mask sizes first since they give the largest gain. In practice, augmentations should reflect real deployment conditions, adding reverb is pointless if the product is always close-talking headset audio.

Q1.8

What is spectral leakage and why does the choice of window function matter? Spectral leakage is energy from one true frequency spreading into neighboring FFT bins. It arises because the FFT assumes the windowed frame repeats periodically; if the signal does not complete an integer number of cycles within the frame, the discontinuity at the frame edges injects spurious broadband energy. A window function tapers the frame to near-zero at its edges, smoothing that discontinuity and concentrating energy in the correct bin. The trade-off is the main-lobe width versus side-lobe height: a rectangular window has the narrowest main lobe (best frequency resolution) but terrible -13 dB side lobes (worst leakage), while Blackman has very low side lobes but a wide main lobe. Hann is the pragmatic default for speech; Hamming is common in classical ASR feature pipelines.

Q1.9

Design the audio ingestion layer for a service that accepts user uploads in arbitrary formats. What are the failure modes? The ingestion layer must normalize a chaotic input space into one canonical representation: 16 kHz, mono, float32 PCM, loudness-normalized. I would decode with a robust backend (FFmpeg via `torchaudio` or `soundfile`), then enforce a pipeline: detect and downmix multi-channel to mono, resample to 16 kHz with proper anti-aliasing, trim or flag silence, and run a loudness/clip check. The failure modes to defend against: corrupt or truncated files (wrap decoding in error handling and reject gracefully); unexpected sample rates (never trust the header blindly, resample explicitly); variable bit depth and float-vs-int scaling (normalize to $[-1, 1]$ float); extremely long files (chunk or reject to protect memory); silent or near-silent uploads (gate with an energy/VAD check); and codec artifacts from heavily compressed sources. Each rejected file should return a clear, actionable error rather than silently producing garbage transcripts.

Q1.10

What is the difference between magnitude, power, and log-power spectrograms, and where is phase? The STFT produces complex values; their magnitude is $|STFT(t, f)|$, the **magnitude spectrogram**. Squaring gives the **power spectrogram** $|STFT(t, f)|^2$, which is what the Mel filterbank integrates over. Taking $10 \log_{10}$ of power yields the **log-**

power (dB) spectrogram, compressing dynamic range for visualization and modeling. All three discard **phase**, the angle $\angle STFT(t, f)$. Phase is perceptually subtle but essential for reconstruction: to invert a spectrogram back to a waveform you need phase, which is why vocoders and source-separation models either predict phase, use Griffin–Lim to estimate it iteratively, or operate on complex spectrograms directly. The interview-level insight: classification and ASR can throw phase away because the information they need is in the magnitude, but anything that *generates* audio must account for phase.

References

- Davis, S. & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), 357–366.
- Oppenheim, A. V. & Schafer, R. W. (2009). *Discrete-time signal processing* (3rd ed.). Pearson.
- Park, D. S. Chan, W. Zhang, Y. Chiu, C.-C. Zoph, B. Cubuk, E. D. & Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. *Proceedings of Interspeech 2019*, 2613–2617.
- Shannon, C. E. (1949). Communication in the presence of noise. *Proceedings of the IRE*, 37(1), 10–21.

ASR I — Acoustic Modeling

Subscribe → aiengineeringinsider.substack.com/subscribe

*The first pillar, **Hear**. This chapter is the engine room of speech recognition: how models learn to map audio frames to symbols without hand-aligned data. Expect CTC, RNN-Transducer, self-supervision, and the Conformer to dominate any ASR-focused screen.*

2.1 Overview

Automatic Speech Recognition (ASR) maps a variable-length audio signal to a variable-length text sequence. The central difficulty is **alignment**: we have, say, 1,000 audio frames and 12 output characters, and no labeled correspondence between them. The history of ASR is largely the history of solving alignment.

The classical solution was the **GMM-HMM** pipeline: a Gaussian Mixture Model estimated $P(\text{features} | \text{phoneme})$, a Hidden Markov Model handled the temporal dynamics, a pronunciation lexicon mapped words to phonemes, and an n-gram language model scored word sequences. A Viterbi/beam-search decoder fused these scores. It worked, but every stage was trained separately and required expert tuning.

Modern ASR collapses this into end-to-end neural models, and three paradigms matter for interviews:

- **CTC (Connectionist Temporal Classification)**, predicts a label (or a special *blank*) per frame and marginalizes over all alignments. Conditional independence between outputs; fast, non-autoregressive.
- **Attention encoder-decoder (AED / LAS)**, an autoregressive decoder attends over the encoded audio. Strong accuracy, but offline by nature and prone to over/under-generation on long audio. Whisper is an AED.
- **RNN-Transducer (RNN-T)**, combines a frame-synchronous encoder with an autoregressive prediction network through a joint network. The dominant choice for *streaming* on-device ASR (Google, Apple, Amazon).

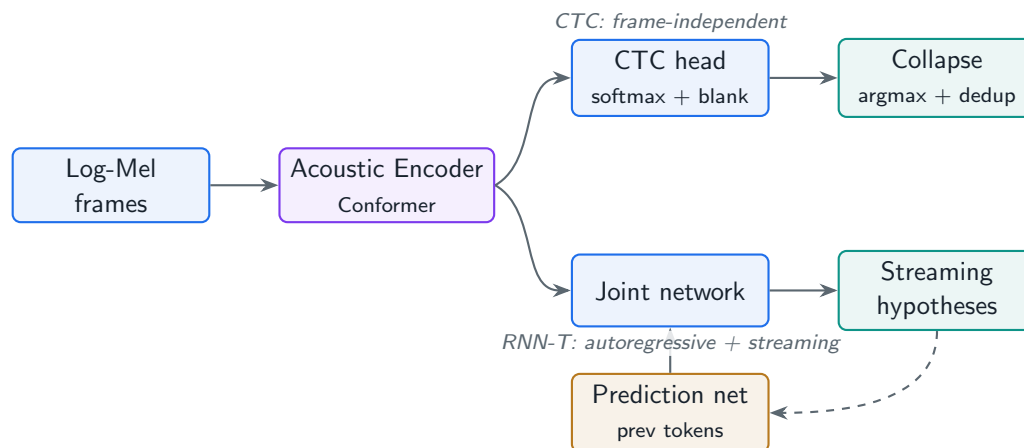
2.1.1 Self-Supervised Acoustic Encoders

Labeled speech is expensive; unlabeled audio is nearly free. **Wav2Vec 2.0** (Baevski et al., 2020) pretrains a CNN feature encoder plus a Transformer context network on raw waveform using a contrastive task over quantized latent units, then fine-tunes with a small CTC head, reaching competitive accuracy with as little as ten minutes of labeled data. **HuBERT** (Hsu et al., 2021) swaps the contrastive objective for masked prediction of offline k-means cluster IDs, which is simpler to tune and iteratively refined. These encoders are the backbone of low-resource ASR.

2.1.2 The Conformer

The **Conformer** (Gulati et al., 2020) is, as of mid-2025, the production-default encoder: each block interleaves a convolution module (local phoneme patterns) with multi-head self-attention (long-range context), wrapped in Macaron-style feed-forward layers. It powers NeMo, SeamlessM4T, and Google’s USM, and it is the answer to “what encoder would you reach for today?”

2.2 Architecture Diagram: CTC vs RNN-Transducer



The defining question “CTC or RNN-T?” reduces to one trade-off. CTC assumes output tokens are conditionally independent given the audio, so it cannot model strong language structure on its own (it leans on an external LM). RNN-T adds a prediction network that conditions each output on previous outputs, giving an implicit language model *and* frame-synchronous streaming, at higher training cost and memory. For streaming products, RNN-T wins; for simple, fast batch transcription, CTC is hard to beat.

2.3 Production Code: Wav2Vec2 CTC Inference

This is the `asr.py` CTC backend of the voice-agent project. It loads a fine-tuned Wav2Vec2 model and decodes greedily, with the collapse rule implemented explicitly so the mechanics are visible (production would use a beam-search decoder with an n-gram LM via `pyctcdecode`).

`voice_agent/asr_ctc.py`

```

"""CTC-based ASR backend using a fine-tuned Wav2Vec2 encoder."""
from __future__ import annotations
import torch
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

class CTCRecognizer:
    """Greedy CTC decoding with explicit blank-collapse.

    The collapse rule is the heart of CTC: take the argmax label at every
  
```

```

frame, remove consecutive duplicates, then drop the blank token. This
turns a frame-level label lattice into clean text.
"""

def __init__(self, model_id: str = "facebook/wav2vec2-base-960h",
              device: str = "cpu"):
    self.device = device
    self.processor = Wav2Vec2Processor.from_pretrained(model_id)
    self.model = Wav2Vec2ForCTC.from_pretrained(model_id).to(device).eval()
    self.blank_id = self.model.config.pad_token_id

@torch.inference_mode()
def transcribe(self, waveform: torch.Tensor, sr: int = 16_000) -> str:
    # processor handles normalization + tensor packing
    inputs = self.processor(
        waveform.squeeze().cpu().numpy(),
        sampling_rate=sr, return_tensors="pt",
    ).input_values.to(self.device)

    logits = self.model(inputs).logits          # (1, T, vocab)
    pred_ids = logits.argmax(dim=-1)[0]         # (T,) greedy per frame
    return self._collapse(pred_ids)

def _collapse(self, ids: torch.Tensor) -> str:
    out, prev = [], None
    for tok in ids.tolist():
        if tok != prev and tok != self.blank_id: # CTC collapse rule
            out.append(tok)
        prev = tok
    return self.processor.decode(out).strip().lower()

```

In the voice agent this CTC backend is the low-latency fallback; the Whisper backend (Chapter 3) is the accuracy default. Exposing both behind one `transcribe` interface lets the pipeline trade latency for accuracy at runtime.

Greedy CTC decoding ignores the language model entirely, so it produces phonetically plausible but lexically wrong output (“recognize speech” vs “wreck a nice beach”). For any real product, add beam search with a KenLM n-gram via `pyctcdecode`; the WER improvement is typically 10–20% relative for a few milliseconds of extra latency.

2.4 Comparison Tables

2.4.1 ASR Paradigm Trade-offs

Paradigm	Streaming	Implicit LM	Best for
CTC	Yes (causal)	No	Fast batch ASR; SSL fine-tuning
AED (Whisper)	No	Yes	Highest offline accuracy; multitask
RNN-T	Yes	Yes	On-device, low-latency streaming
GMM-HMM	Yes	External	Legacy; teaching the fundamentals

2.4.2 Self-Supervised Encoders

Model	Objective	Note
Wav2Vec2	Contrastive over quantized units	10 min labeled → usable ASR
HuBERT	Masked prediction of k-means IDs	Simpler to tune; iterative refinement
WavLM	HuBERT + denoising + speaker mix	Strong on overlapping speech

For a new language or domain with little labeled data, start from a multilingual self-supervised checkpoint (XLS-R or a HuBERT variant) and fine-tune a CTC head. You will reach a usable WER with one to ten hours of transcribed audio, far less than training from scratch.

If I could delete one misconception it is that “end-to-end” means “no alignment.” It does not. CTC and RNN-T marginalize over alignments instead of labeling them. The candidates who can explain the blank token’s two jobs in one breath consistently outperform those reciting five architectures. And against a lot of blog hype: attention encoder-decoders are not the answer to everything. I would not put Whisper anywhere near a streaming product.

2.5 Mock Interview Questions & Answers

Q2.1

Explain the CTC loss and the role of the blank token. CTC solves alignment by introducing a special *blank* symbol and defining the probability of a target transcript as the sum over all frame-level label sequences (“alignments”) that collapse to it. The collapse rule removes consecutive duplicate labels and then deletes blanks, so “h-h-ε-i” and “ε-h-i-i” both yield “hi”. The blank serves two purposes: it lets the model emit nothing on a frame (most frames), and it acts as a separator so genuine repeated characters (the two l’s in “hello”) are not collapsed into one, “l-ε-l” survives. The loss is computed efficiently with the forward–backward (dynamic programming) algorithm, summing probabilities of all valid alignments. The key limitation interviewers probe: CTC assumes outputs are conditionally independent given the input, so it has no built-in language model.

Q2.2

Why does the industry use RNN-Transducer for on-device streaming ASR instead of CTC or attention models? RNN-T is frame-synchronous and autoregressive at once, which is exactly what streaming needs. Like CTC, it processes audio frame by frame, so it can emit tokens as audio arrives with bounded latency, attention encoder-decoders like Whisper cannot, because

the decoder attends over the whole utterance and is designed for fixed chunks. Unlike CTC, RNN-T's prediction network conditions each output on previously emitted tokens, giving it an internal language model so it produces fluent text without a separate LM. The trade-off is cost: the RNN-T loss is computed over a 3-D lattice (time x label x vocab), which is memory-hungry to train, and the joint network adds compute. But for a phone assistant that must show words as you speak, RNN-T's combination of streaming and implicit LM is why Google, Apple, and Amazon standardized on it.

Q2.3

Walk me through how Wav2Vec2 achieves strong ASR with only ten minutes of labeled data. The trick is decoupling representation learning from the labeled task. In pretraining, a 1-D CNN feature encoder converts raw waveform to latent vectors at 20 ms resolution. Some latents are masked, and a Transformer context network must identify the correct quantized latent for each masked position among distractors, a contrastive task that requires no transcripts and can run on thousands of hours of unlabeled audio. This forces the network to learn phonetically meaningful representations. Fine-tuning then adds a small linear CTC head on top and trains on the limited labeled data; because the encoder already “understands” speech structure, only the mapping from representations to characters must be learned, which needs very little supervision. The insight: self-supervision moves the expensive learning to free unlabeled data, leaving a tiny supervised problem.

Q2.4

What problem does the Conformer solve that a pure Transformer encoder does not? Speech has structure at two scales: local patterns (formant transitions, phoneme shapes) that span tens of milliseconds, and long-range dependencies (prosody, coarticulation, context) that span seconds. A pure Transformer captures global context well via self-attention but has no inductive bias for local patterns, so it needs more data and depth to learn them. A pure CNN captures local patterns efficiently but struggles with long-range context. The Conformer interleaves both within each block: a depthwise-separable convolution module models local structure while multi-head self-attention models global context, sandwiched between two half-step feed-forward layers (the Macaron design). This two-path design is why the Conformer consistently beats same-size Transformers on ASR benchmarks and became the production-default encoder.

Q2.5

Your CTC model outputs phonetically correct but misspelled words. How do you fix it without retraining? This is the signature symptom of greedy CTC decoding with no language model: the acoustic model is right about the sounds but has no lexical knowledge. The fix is decoding-time, not training-time. Replace greedy argmax with beam search over the CTC posteriors and fuse an external n-gram language model (shallow fusion) using a library like `pyctcdecode` with a KenLM model trained on in-domain text. The beam search now balances acoustic probability against word-level plausibility, correcting “wreck a nice beach” to “recognize speech”. You can also add a lexicon constraint so only valid words are emitted. This typically buys 10–20% relative WER improvement for a few milliseconds of latency and zero model retraining. If misspellings persist on domain terms, add those terms to the LM and lexicon.

Q2.6

Compare HuBERT and Wav2Vec2. When would you pick one over the other? Both are self-supervised raw-waveform encoders, but they differ in the pretraining target. Wav2Vec2 uses a contrastive objective: identify the true quantized latent among distractors. HuBERT uses masked prediction of *offline* targets, cluster IDs from k-means run on MFCCs (first iteration) or on prior HuBERT features (later iterations), making it a BERT-style classification task. HuBERT's offline targets make training more stable and avoid the sensitivity of contrastive learning to negative sampling

and temperature, so it is often easier to reproduce and tends to edge out Wav2Vec2 on low-resource ASR. I would pick HuBERT (or WavLM, which extends it with denoising and speaker augmentation) when stability and overlapping-speech robustness matter, and Wav2Vec2/XLS-R when I want the mature multilingual checkpoints. In practice the choice is often dictated by which pretrained checkpoint covers my languages.

Q2.7

Design an ASR system for a call center handling 8 kHz telephone audio across ten languages. Walk me through the choices. First, the data reality: 8 kHz narrowband, noisy, code-switching, domain jargon. I would not upsample-and-pretend; I would either fine-tune a model on 8 kHz features or train at 16 kHz and resample consistently, validated on real telephone audio. For the encoder, a multilingual self-supervised backbone (XLS-R) fine-tuned per language or with a shared head plus language ID. For decoding, RNN-T if agents need live captions (streaming), otherwise a Whisper-class AED for best offline accuracy on recorded calls. I would add a per-language n-gram LM with call-center vocabulary (product names, account terms) for shallow fusion, a domain lexicon, and inverse text normalization for numbers and dates. Operationally: a VAD front-end to drop hold music and silence, diarization to separate agent and caller, and a confidence-gated human-review queue for low-confidence segments. I would track WER per language and per acoustic condition, because the aggregate number hides the worst-performing slice.

Q2.8

What is the alignment problem in ASR and how do CTC, attention, and RNN-T each address it? The alignment problem is that input frames and output tokens have different, unknown-correspondence lengths, and we lack frame-level labels. CTC handles it implicitly by summing over all monotonic alignments via the blank token and dynamic programming, alignment is marginalized out, never explicitly computed. Attention encoder-decoders handle it with a soft, learned attention distribution: at each decoding step the decoder computes attention weights over all encoder frames, which is a flexible (non-monotonic) alignment, powerful but capable of pathological skips or repeats on long audio. RNN-T handles it with an explicit lattice over time and label axes where each path is a monotonic alignment, combining encoder and prediction-network outputs in the joint network and marginalizing with a forward-backward algorithm like CTC, but with autoregressive label dependence. The spectrum runs from strict monotonic (CTC, RNN-T) to free (attention).

Q2.9

How would you debug an ASR model whose training loss decreases but validation WER stays flat? Decreasing loss with flat WER means the model is optimizing the objective but not the metric you care about, so I would separate the layers. First confirm the evaluation itself: is text normalization consistent between hypothesis and reference (case, punctuation, numbers)? A normalization mismatch can mask real gains entirely. Second, inspect actual transcripts, not just the number, maybe WER is dominated by a few catastrophic long-utterance failures (common with attention models) while most outputs improve. Third, check for a train/val distribution gap: augmentation might help training-like audio while validation is a different acoustic condition. Fourth, look at the decoding setup, a too-narrow beam or a misweighted LM can cap WER regardless of acoustic improvements. Fifth, verify the loss is the right proxy (CTC loss can keep dropping by sharpening blanks without improving content). I would also plot WER versus utterance length and SNR to localize the failure.

Q2.10

Why did end-to-end neural ASR replace the GMM-HMM pipeline, and what parts of the old pipeline survive? GMM-HMM trained acoustic model, lexicon, and language model separately, each with its own objective and expert tuning, so errors compounded and the system could not jointly optimize for transcription accuracy. End-to-end models learn the audio-to-text mapping with a single objective, exploit large data and GPUs, and remove the need for hand-built pronunciation dictionaries and forced alignments, yielding large WER reductions and far simpler pipelines. But several ideas survive. The n-gram or neural language model lives on as a shallow-fusion rescorer for CTC and RNN-T decoders. Pronunciation lexicons still help for rare proper nouns and constrained-vocabulary tasks. The HMM's monotonic-alignment assumption reappears in CTC and RNN-T. And WFST decoders from the Kaldi era still power some production systems. Knowing the classical pipeline is what lets you reason about what an LM rescorer or lexicon is actually doing at inference time.

References

- Baevski, A. Zhou, Y. Mohamed, A. & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33, 12449–12460.
- Graves, A. Fernández, S. Gomez, F. & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of the 23rd International Conference on Machine Learning*, 369–376.
- Graves, A. (2012). Sequence transduction with recurrent neural networks. *arXiv preprint arXiv:1211.3711*.
- Gulati, A. Qin, J. Chiu, C.-C. Parmar, N. Zhang, Y. Yu, J. . . . & Pang, R. (2020). Conformer: Convolution-augmented Transformer for speech recognition. *Proceedings of Interspeech 2020*, 5036–5040.
- Hsu, W.-N. Bolte, B. Tsai, Y.-H. H. Lakhota, K. Salakhutdinov, R. & Mohamed, A. (2021). HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29, 3451–3460.

ASR II — Whisper, Diarization & Evaluation

Subscribe → aiengineeringinsider.substack.com/subscribe

*The applied half of **Hear**: the model you will actually deploy (Whisper), the “who spoke when” problem (diarization), always-on wake words, and the metrics that decide whether your system ships. Interviewers test whether you can operate ASR, not just describe it.*

3.1 Overview

If Chapter 2 was the theory of acoustic modeling, this chapter is the practice. **Whisper** (Radford et al., 2022) is the current default for offline ASR: an attention encoder-decoder trained on 680,000 hours of weakly supervised, multilingual web audio. That scale makes it remarkably robust to accents, noise, and domain shift with zero fine-tuning, and it is multitask, transcription, translation to English, language identification, and timestamp prediction, all selected by special tokens fed to the decoder.

- **Input**, 80-bin log-Mel (128 for large-v3) over a fixed 30-second window at 16 kHz; shorter clips are zero-padded.
- **Encoder**, standard Transformer with sinusoidal positional encoding.
- **Decoder**, autoregressive Transformer with cross-attention to the encoder, conditioned on a task prompt of special tokens.
- **Special tokens**, `<|transcribe|>`, `<|translate|>`, `<|en|>`, `<|nospeech|>`, and timestamp tokens.

3.1.1 Diarization: Who Spoke When

Speaker diarization partitions audio by speaker identity without knowing the speakers in advance. The standard pipeline runs VAD to find speech, segments it at likely speaker-change points, extracts a fixed-length **speaker embedding** (x-vector or ECAPA-TDNN) per segment, clusters the embeddings, and re-segments to refine boundaries. Combined with ASR (as in WhisperX), it yields speaker-attributed transcripts, “Agent: ... / Caller: ...”.

3.1.2 Keyword Spotting and Wake Words

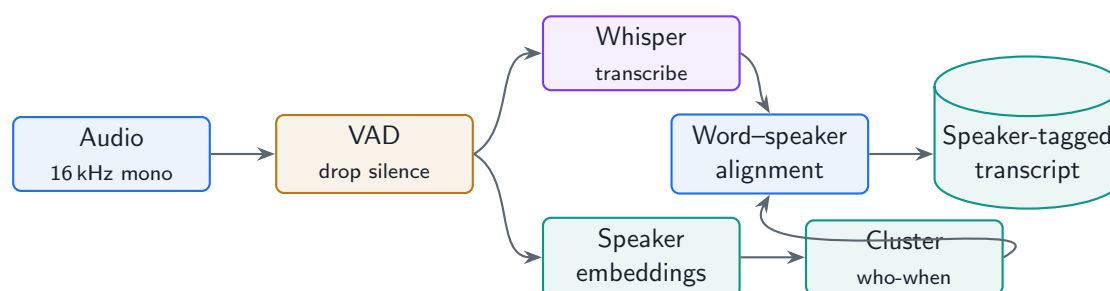
Keyword spotting (KWS) is the always-on, ultra-low-power detector that wakes a device on “Hey Siri” or “Alexa”. The engineering constraint is brutal: run continuously on a microcontroller at milliwatts, with

a very low false-accept rate (no accidental wakes) *and* a very low false-reject rate (never miss the user). Tiny depthwise-separable CNNs under 100K parameters dominate.

3.1.3 Evaluation

You cannot manage what you do not measure. **WER** (Word Error Rate) is the primary ASR metric; **CER** (Character Error Rate) suits CJK languages; **DER** (Diarization Error Rate) scores diarization; and **RTF** (Real-Time Factor) governs whether a system is fast enough to deploy.

3.2 Architecture Diagram: A Transcription + Diarization Service



Whisper does not diarize. A very common interview gotcha is assuming a single model gives you speaker labels, it does not. You run ASR and a separate embedding+clustering pipeline, then align words to speakers by timestamp. WhisperX and NeMo do exactly this orchestration; knowing they are two systems stitched together shows real experience.

3.3 Production Code: Whisper Backend with Word Timestamps

The `asr.py` default backend uses `faster-whisper` (a CTranslate2 re-implementation, 4x faster than the reference) and returns word-level timestamps so the pipeline can align speakers and drive barge-in (Chapter 8).

`voice_agent/asr.py`

```

"""Whisper ASR backend (faster-whisper) with word-level timestamps."""
from __future__ import annotations
from dataclasses import dataclass

@dataclass
class Word:
    text: str
    start: float
    end: float
    prob: float

class WhisperRecognizer:
    """Accuracy-default ASR backend for the voice agent.
```

```

Wraps faster-whisper. Returns both the full transcript and timed words,
which downstream stages use for diarization alignment and barge-in.
"""

def __init__(self, model_size: str = "base", device: str = "cpu",
              compute_type: str = "int8"):
    from faster_whisper import WhisperModel
    self.model = WhisperModel(model_size, device=device,
                              compute_type=compute_type)

def transcribe(self, audio_path: str, language: str | None = None):
    segments, info = self.model.transcribe(
        audio_path,
        language=language,          # None => auto language ID
        vad_filter=True,           # skip non-speech regions
        word_timestamps=True,      # needed for alignment/barge-in
        beam_size=5,
    )
    words, text_parts = [], []
    for seg in segments:
        text_parts.append(seg.text)
        for w in (seg.words or []):
            words.append(Word(w.word.strip(), w.start, w.end,
                              w.probability))

    return {
        "text": "".join(text_parts).strip(),
        "language": info.language,
        "words": words,
    }

```

The `vad_filter` and `word_timestamps` flags are the production-relevant choices: VAD filtering removes the silence-hallucination failure mode, and word timestamps are what let the orchestrator (Chapter 8) interrupt the agent’s own speech the instant the user starts talking.

Whisper **hallucinates** on silence and non-speech audio, it can emit “Thank you for watching” or repeated phrases from its training distribution when fed music or quiet. Always gate Whisper with a VAD and treat low- `avg_logprob` or high `no_speech_prob` segments as suspect. This is a frequent production-debugging interview question.

3.4 Comparison Tables

3.4.1 Whisper Model Sizes

Model	Params	Approx. RTF (CPU)	When to use
tiny	39 M	<0.05	Edge / fastest, lowest accuracy
base	74 M	<0.10	Balanced default for demos
small	244 M	~0.15	Good accuracy/speed trade-off
medium	769 M	~0.35	High accuracy, server-side
large-v3	1.5 B	~0.80	Best accuracy, GPU recommended
distil-large-v3	756 M	~0.25	Near-large accuracy, much faster

3.4.2 ASR & Diarization Metrics

Metric	Definition
WER	$(S + D + I)/N$ over words: substitutions, deletions, insertions, references
CER	Same formula at character level; better for CJK and agglutinative languages
RTF	Inference time / audio duration; <1 for real-time, <0.1 for production
DER	(missed + false alarm + speaker confusion) / total speech time

Fine-tune **small** or **medium** on 10 hours of in-domain audio before reaching for large-v3 out of the box. A fine-tuned medium usually beats a zero-shot large-v3 on your domain at one-third the inference cost, a strong, cost-aware answer in system-design rounds.

The weirdest bug I have shipped: a transcription service that politely typed “Thank you for watching!” over thirty seconds of silence. Whisper is a language model wearing an ASR costume, and with no audio to anchor it, it falls back to YouTube. A VAD gate fixed it in an afternoon. An unpopular opinion that has saved real money, too: large-v3 is rarely worth it. distil/turbo at a fraction of the cost is the right default, and a fine-tuned medium beats zero-shot large on your own domain.

3.5 Mock Interview Questions & Answers

Q3.1

How is Whisper trained, and why is it so robust without fine-tuning? Whisper is trained on 680,000 hours of audio paired with transcripts scraped from the web, weak supervision rather than carefully curated data. Two design choices create its robustness. First, scale and diversity: the data spans accents, recording conditions, languages, and domains far beyond any academic corpus, so the model sees the long tail at training time instead of failing on it at test time. Second, multitask training with special tokens: the same model learns transcription, X-to-English translation, language ID, and voice-activity detection, which forces shared representations that generalize. The trade-off is that weak supervision injects label noise, so Whisper is excellent at robust general transcription but can be beaten by a smaller model fine-tuned on a specific narrow domain, and it inherits quirks like hallucination on non-speech.

Q3.2

A user reports Whisper outputs “Thank you for watching” on a silent recording. Diagnose and fix. This is Whisper hallucination: the autoregressive decoder, conditioned on near-empty acoustic evidence, falls back to high-probability phrases from its training distribution (YouTube sign-offs, subtitles). The decoder is a language model, so with no acoustic signal to anchor it, it generates fluent nonsense. Fixes, in order: gate the input with a VAD so silent or non-speech segments never reach the model; enable `vad_filter` in faster-whisper; raise the `no_speech_threshold` and filter segments with high `no_speech_prob` or very low `avg_logprob`; disable or limit `condition_on_previous_text`, which can cause repetition loops; and for known-clean pipelines, set `compression_ratio` thresholds to catch degenerate repeats. The root-cause framing, decoder behaving as an LM under weak evidence, is what a senior answer includes.

Q3.3

Design a meeting-transcription product. How do you get accurate speaker-labeled transcripts? This is ASR plus diarization plus alignment, and they are separate systems. I would run a VAD to isolate speech, transcribe with Whisper (or a streaming model for live captions) capturing word-level timestamps, and in parallel extract ECAPA-TDNN speaker embeddings over short windows and cluster them (agglomerative or spectral, or pyannote’s pipeline) to answer “who spoke when”. Then align: assign each word to the speaker whose segment overlaps its timestamp. Challenges to call out: overlapping speech (use a diarization model that handles overlap, or separation); unknown speaker count (let clustering estimate it, or use online clustering for live); far-field and varying mic distance (the hardest acoustic case); and speaker re-identification across meetings if you want persistent labels. I would measure WER and DER separately and surface low-confidence segments for human correction. Latency target sets whether this is batch (best accuracy) or streaming (harder).

Q3.4

Explain WER and its limitations. What does a WER of 15% actually mean? WER is the edit distance between hypothesis and reference at the word level, normalized by reference length: $(S + D + I)/N$ where S , D , I are substitutions, deletions, insertions and N is the number of reference words. A WER of 15% means roughly one error per seven words, but that single number hides everything that matters. It weights all words equally, so misrecognizing a critical entity (a drug name, an account number) counts the same as a dropped “the”, even though one is catastrophic and the other harmless. It is sensitive to text normalization: case, punctuation, and number formatting differences inflate WER unless both sides are normalized identically. It can exceed 100% (many insertions). And aggregate WER hides per-speaker, per-accent, and per-SNR disparities. Senior practice is to report WER alongside entity error rate, normalized consistently, and sliced by the conditions you care about.

Q3.5

What are the hard constraints in wake-word detection, and how do you meet them? A wake-word detector runs always-on, so it must fit a milliwatt power budget and a sub-100K-parameter footprint on a microcontroller, while simultaneously achieving a very low false-accept rate (no embarrassing random wakes, often specified as a handful per day) and a very low false-reject rate (never ignore the user). These goals trade off directly along the detection threshold. To meet them: use a tiny depthwise-separable CNN or a small GRU over log-Mel features; train with heavy hard negatives (phonetically similar phrases, TV/background speech) to push down false accepts; use an FRR-at-fixed-FAR operating point chosen from a DET curve rather than raw accuracy; add a small streaming buffer and a two-stage design (cheap always-on detector triggers a larger verifier) to balance power and precision; and quantize to int8. Personalization and on-device adaptation further

cut false rejects for a specific user's voice.

Q3.6

When would you fine-tune Whisper versus use it zero-shot, and how would you fine-tune efficiently? Use Whisper zero-shot when the domain is broad and conversational, it is already excellent there and fine-tuning risks catastrophic forgetting. Fine-tune when you have a consistent domain shift: heavy jargon (medical, legal), a specific accent or low-resource language, a fixed vocabulary, or a target like better timestamps. Efficient fine-tuning: prepare a `datasets.Dataset` with audio resampled to 16 kHz and normalized transcripts; use the Whisper feature extractor and tokenizer; apply LoRA adapters to the decoder attention projections (rank 16–32) so you train under 1% of parameters and avoid forgetting; train with the seq2seq trainer, evaluating WER on a held-out in-domain set every few hundred steps; then merge adapters and export to CTranslate2 for faster-whisper deployment. Start from small or medium, not large, for cost. Always keep a general-domain eval set to catch regression on out-of-domain audio.

Q3.7

Your diarization system reports DER of 8% but users complain labels are wrong. Reconcile this. DER and perceived quality measure different things. DER is time-weighted: it sums missed speech, false alarm, and speaker confusion over total speech duration. A system can score a respectable 8% DER while making the errors users actually notice. For example, brief but frequent speaker-confusion at turn boundaries contributes little to time-weighted DER but is exactly where a reader's eye lands, so the transcript “feels” wrong. Overlapping speech is often excluded from the standard DER collar, so overlap errors users hear are not penalized. And consistent label *permutation* (Speaker 1 vs Speaker 2 swapped) can be low-DER yet maddening if labels must map to real names. The fix is to measure what users feel: report word-level diarization error (speaker-attributed WER, or cpWER), evaluate without a forgiving collar, include overlap, and check boundary precision. The lesson: optimize the metric that matches the user experience.

Q3.8

How do you build a streaming (live-caption) transcription system from models that are mostly offline? Offline models like Whisper assume a full 30-second window, so streaming requires an orchestration layer. The standard approach is chunked streaming with context: buffer audio into overlapping windows (e.g. emit every 1 second with a few seconds of left-context), run the model on each, and stitch results, using the overlap to stabilize boundary words. To reduce flicker, mark recent words as “tentative” and only commit them once they stop changing across chunks (local agreement / LocalAgree policy). For true low latency, prefer a natively streaming architecture (RNN-T) or a streaming-tuned Whisper variant rather than fighting an offline model. Add a VAD to chunk on natural pauses, which both cuts latency and avoids cutting words. Key trade-off to articulate: latency versus accuracy and stability, smaller chunks mean faster captions but more revisions and lower accuracy from reduced context.

Q3.9

What is Real-Time Factor and why can a model with RTF below 1.0 still fail in production? RTF is processing time divided by audio duration; $RTF < 1.0$ means the model transcribes faster than real time on average. But “on average” is the trap. Production cares about tail latency and concurrency, not mean throughput. A model with RTF 0.3 can still blow a latency SLA if requests queue under load, if a single long utterance occupies a worker for seconds (head-of-line blocking), or if cold starts and model loading add seconds to the first request. Streaming systems also care about *first-token latency* and *finalization latency*, which RTF does not capture at all. And RTF

measured on a fast dev GPU misleads if production runs quantized on CPU. Senior practice: budget end-to-end p95/p99 latency including queueing and network, size worker pools for concurrency, chunk long audio to avoid blocking, and treat RTF as a necessary but insufficient capacity-planning input.

Q3.10

Compare faster-whisper, WhisperX, and whisper.cpp. Which would you deploy and why? All three are deployment-oriented re-implementations of Whisper with different priorities. **faster-whisper** uses the CTranslate2 inference engine with int8/fp16 quantization, giving roughly 4x speedup and lower memory than the reference PyTorch model, my default for server-side GPU or CPU transcription. **WhisperX** builds on faster-whisper and adds forced-alignment for accurate word timestamps plus pyannote diarization, so it is the choice when I need speaker-attributed, precisely timed transcripts (meetings, subtitles). **whisper.cpp** is a C/C++ port with GGML quantization optimized for on-device and edge (Apple Silicon, mobile, no Python runtime), so it wins for offline desktop or embedded apps where dependencies and footprint matter. Decision rule: server with diarization needs to WhisperX; pure fast server transcription to faster-whisper; edge/offline to whisper.cpp. The unifying point is they share weights but optimize different deployment constraints.

References

- Bredin, H. (2023). pyannote.audio 2.1 speaker diarization pipeline: Principle, benchmark, and recipe. *Proceedings of Interspeech 2023*, 1983–1987.
- Desplanques, B. Thienpondt, J. & Demuynck, K. (2020). ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. *Proceedings of Interspeech 2020*, 3830–3834.
- Radford, A. Kim, J. W. Xu, T. Brockman, G. McLeavey, C. & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*.

Spoken Language Understanding

Subscribe → aiengineeringinsider.substack.com/subscribe

*The second pillar, **Understand**. Turning a transcript, or raw audio, into structured meaning: intents, entities, dialogue state, and emotion. This is where the speech stack meets NLP, and where LLMs have reshaped the playbook.*

4.1 Overview

Spoken Language Understanding (SLU) converts speech into a machine-actionable representation. The classical decomposition has three jobs: **intent classification** (what does the user want? “book_flight”), **slot filling / entity extraction** (the parameters, destination=Tokyo, date=Friday), and **dialogue state tracking** (accumulating those across turns). On top sit auxiliary tasks: **emotion / sentiment recognition**, **punctuation restoration**, and **inverse text normalization (ITN)** (turning “twenty twenty five” into “2025”).

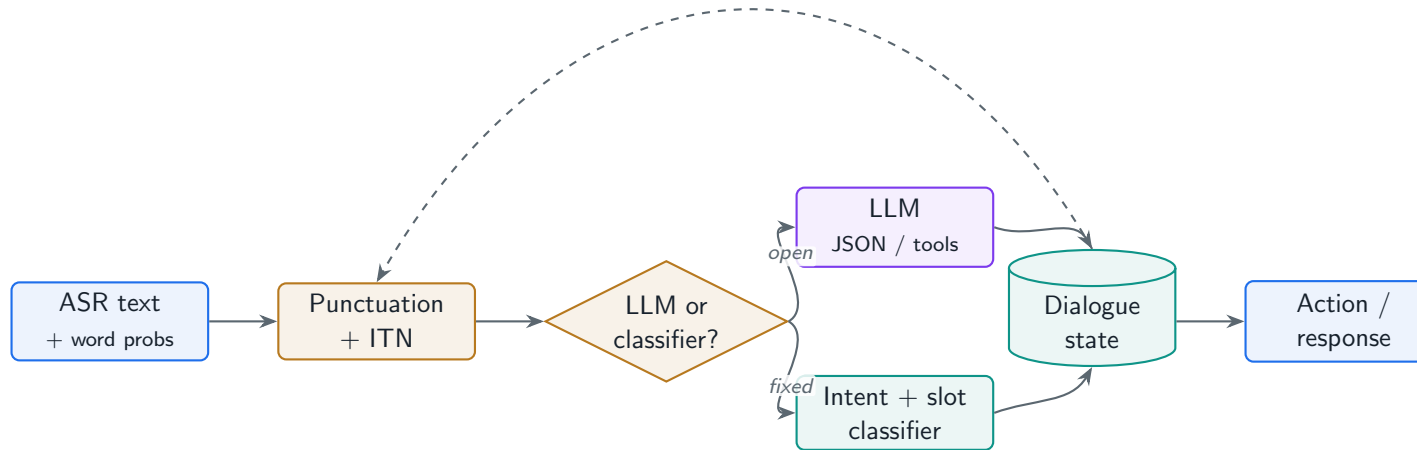
There are two architectural philosophies:

- **Cascaded (ASR → NLU)**, transcribe with ASR, then run a text NLU model. Modular, debuggable, reuses text models; but ASR errors propagate and prosody (the *way* something is said) is discarded.
- **End-to-End SLU**, predict intent/slots directly from audio. Preserves prosody and avoids cascaded error compounding; but needs audio-labeled intent data and is harder to debug.

4.1.1 The LLM Shift

The 2023+ playbook increasingly replaces brittle intent/slot classifiers with an LLM that consumes the transcript (and conversation history) and emits structured output via function calling or constrained JSON. This collapses intent and slot filling into one flexible step, handles open-domain requests, and tracks state in context, at the cost of latency, cost, and the need for guardrails. Knowing when a small fine-tuned classifier still beats an LLM (tight latency, fixed intents, high volume) is a strong signal.

4.2 Architecture Diagram: SLU in the Voice Agent



The biggest accuracy lever in cascaded SLU is not the NLU model, it is feeding it ASR *confidence*. A transcript with low-confidence words on the critical slot (the destination city) should trigger a clarifying question, not a confident wrong booking. Passing word probabilities downstream, instead of throwing them away after ASR, is what separates a robust agent from a demo.

4.3 Production Code: Intent + Slot Extraction with an LLM

The `nlu.py` module routes between a fast rule/classifier path and an LLM path. Below is the LLM path with constrained JSON output and a Pydantic schema, the pattern most production voice agents now use.

`voice_agent/nlu.py`

```

"""Spoken-language understanding: intent + slots as structured output."""
from __future__ import annotations
import json
from dataclasses import dataclass, field

@dataclass
class Interpretation:
    intent: str
    slots: dict = field(default_factory=dict)
    confidence: float = 0.0
    needs_clarification: bool = False

INTENTS = ["book_flight", "check_status", "cancel", "smalltalk", "unknown"]

SYSTEM_PROMPT = (
    "You convert a user's spoken request into JSON. "
    "Return EXACTLY: {\"intent\": <one of " + str(INTENTS) + ">, "

```

```

    "\"slots\": {<key>: <value>}, \"confidence\": <0..1>}. \"
    \"Use only evidence present in the text. Do not invent slot values.\"
)

class LLMUnderstanding:
    \"\"\"LLM-backed SLU with a strict schema and a confidence gate.\"\"\"

    def __init__(self, llm_client, min_conf: float = 0.55):
        self.llm = llm_client          # any chat client with .complete()
        self.min_conf = min_conf

    def interpret(self, transcript: str, asr_min_word_prob: float = 1.0
                  ) -> Interpretation:
        raw = self.llm.complete(system=SYSTEM_PROMPT, user=transcript)
        try:
            data = json.loads(raw)
        except json.JSONDecodeError:
            return Interpretation(intent="unknown", needs_clarification=True)

        conf = float(data.get("confidence", 0.0))
        # Combine model confidence with upstream ASR reliability: a shaky
        # transcript should never produce a confident action.
        effective = min(conf, asr_min_word_prob)
        return Interpretation(
            intent=data.get("intent", "unknown"),
            slots=data.get("slots", {}),
            confidence=effective,
            needs_clarification=effective < self.min_conf,
        )

```

The `effective = min(conf, asr_min_word_prob)` line is the production heart of this module: it propagates ASR uncertainty into the decision so the agent asks “Did you say Tokyo or Kyoto?” instead of silently booking the wrong city.

Never let an LLM invent slot values. Without a strict “use only evidence in the text” instruction and schema validation, an LLM will helpfully hallucinate a date or confirmation number. Validate every slot against the transcript (or a database) before taking an irreversible action, and require explicit confirmation for high-stakes intents like `cancel` or `payment`.

4.4 Comparison Tables

4.4.1 SLU Approaches

Approach	Uses prosody	Latency	Best for
Cascaded classifier	No	Low	Fixed intents, high volume, tight SLA
End-to-End SLU	Yes	Low	Prosody-sensitive tasks; closed domain
LLM (cascaded)	No	Higher	Open domain, complex multi-slot requests
Audio-LLM	Yes	Higher	Frontier; emotion + content jointly

4.4.2 Auxiliary SLU Tasks

Task	Why it matters in a voice agent
Punctuation restoration	Raw ASR has none; needed for readable text and LLM input
Inverse text norm. (ITN)	“twenty twenty five” → “2025”; dates, money, units
Emotion / sentiment	Route angry callers; adapt TTS tone; safety
Dialogue state tracking	Carry slots across turns; resolve references

Run **punctuation restoration** and **ITN** *before* handing a transcript to an LLM or classifier. Un-punctuated, unnormalized ASR output (“i need three tickets for the twelfth”) measurably degrades downstream understanding; restoring structure first is cheap and consistently improves intent and slot accuracy.

There is a reflex right now to put an LLM behind every intent. I once watched a team replace a 40ms on-device classifier handling five intents with a 700ms LLM call, then spend a whole sprint adding guardrails against the hallucinated sixth intent the classifier could never have produced. Use the LLM where the intent space is genuinely open. For a fixed menu, a small classifier is faster, cheaper, and physically cannot invent an option.

4.5 Mock Interview Questions & Answers

Q4.1

Cascaded versus end-to-end SLU: how do you choose? The decision hinges on three factors: data, prosody, and debuggability. Cascaded (ASR then a text NLU model) is the default because it reuses mature text models, is easy to inspect (you can read the transcript and see where it broke), and lets each stage be improved independently. Its weaknesses are that ASR errors propagate into NLU and that prosody, sarcasm, emphasis, question intonation, is lost the moment audio becomes text. End-to-end SLU predicts intent and slots directly from audio, so it preserves prosody and avoids compounding ASR errors, which helps for tasks where *how* something is said carries meaning, or in very noisy conditions where the transcript is unreliable. The cost is that it needs audio labeled with semantic targets (scarce and expensive) and is a black box to debug. In practice I default to cascaded and only go end-to-end when prosody is task-critical or I have the labeled audio to justify it.

Q4.2

How do you stop ASR errors from corrupting downstream understanding? Treat ASR output as uncertain evidence, not ground truth. The core technique is to propagate confidence: keep word-level probabilities (or n-best hypotheses / a lattice) from ASR and feed them to NLU rather than collapsing to a single string. A low-confidence word on a critical slot should lower the action's effective confidence and trigger a clarifying question instead of a guess. Other defenses: pass n-best hypotheses to the LLM so it can pick the most coherent reading; add a domain language model and lexicon so ASR is less likely to mangle in-domain terms in the first place; use phonetic or fuzzy matching when resolving slot values against a known list (so "Tokyo" still matches if ASR wrote "Toyko"); and require explicit confirmation for irreversible actions. The mindset is end-to-end uncertainty management, not pretending the transcript is perfect.

Q4.3

When should you use an LLM for intent classification, and when is a small fine-tuned classifier better? Use an LLM when the intent space is open or large, requests are compositional (multiple intents and slots in one utterance), the domain changes often, or you want zero/few-shot coverage without collecting training data. The LLM also collapses intent and slot filling into one step and handles conversational context naturally. Prefer a small fine-tuned classifier (DistilBERT-class) when you have a fixed, well-defined set of intents, strict latency and cost budgets, very high request volume, or need deterministic, auditable behavior. A 50-millisecond on-device classifier serving millions of requests will beat an LLM on cost and tail latency, and it cannot hallucinate an intent outside its label set. Many mature systems use a hybrid: a fast classifier handles the head of common intents, and anything it rejects or scores low falls through to an LLM. The interview point is matching the tool to the latency/flexibility/cost frontier, not defaulting to "LLM for everything".

Q4.4

Why run punctuation restoration and inverse text normalization, and where do they sit? Raw ASR output is typically lowercase, unpunctuated, and spoken-form ("call me at four oh seven"). That hurts both humans reading it and models consuming it. Punctuation restoration adds sentence boundaries and commas, which improves readability and gives downstream LLMs/classifiers the syntactic structure they were trained on. ITN converts spoken numbers, dates, currencies, and units into written form ("four oh seven" to a phone number, "twenty twenty five" to "2025"), which is essential for slot extraction and for displaying results. Both sit between ASR and NLU in a cascaded pipeline. They can be separate models (a punctuation transformer, an FST-based or neural ITN) or folded into the ASR/LLM step. The practical lesson: normalizing text structure before understanding is a cheap, reliable accuracy gain that beginners often skip.

Q4.5

Design dialogue state tracking for a multi-turn flight-booking agent. Dialogue state is the accumulated, structured belief about what the user wants across turns: a slot frame (origin, destination, date, passengers) plus discourse context (what was just asked, pending confirmations). I would represent state explicitly as a typed structure, update it each turn by merging the new interpretation with the prior state, and handle overrides ("actually make it Friday") and references ("the earlier one"). With an LLM, I keep the running state in the prompt and ask the model to return the *updated* full state, then validate it against a schema and a database (valid airport codes, available dates) before acting. Critical design points: confirm before irreversible actions; track which slots are still missing to drive the next question; support correction and cancellation at any turn; and persist state so a dropped call can resume. Failure modes to guard: state drift (the LLM silently changing a confirmed slot), and over-asking (re-requesting a slot already filled). I would log every state transition for debugging and evaluation.

Q4.6

How would you add emotion recognition, and what are the pitfalls? Emotion recognition (SER) classifies affect, anger, frustration, neutral, happy, from speech. I would extract it from audio, not text, because prosody (pitch, energy, speaking rate) carries most of the signal; a fine-tuned Wav2Vec2/HuBERT or an audio-LLM works well. In a voice agent, emotion routes angry callers to a human, adapts TTS tone, and flags safety situations. Pitfalls are significant. Emotion labels are subjective and culturally variable, so datasets are noisy and models transfer poorly across populations, acted-emotion corpora (like IEMOCAP) do not match real-world spontaneous affect. There is serious bias risk: SER can systematically misread accents, genders, or neurodivergent speech, leading to unfair routing. It is also easily confounded (excited vs angry have similar arousal). So I would treat SER output as a soft signal with calibrated thresholds, validate across demographic slices for fairness, avoid high-stakes automated decisions based on emotion alone, and be transparent about its use given the privacy sensitivity of inferring emotional state.

Q4.7

An LLM-based NLU sometimes returns invalid JSON or hallucinated slots. How do you make it production-safe? Two failure classes: format violations and content violations. For format, do not rely on the model being well-behaved, use constrained decoding or function-calling / structured-output modes that guarantee schema-valid JSON, and on top of that validate with a strict parser (Pydantic), retrying or falling back to a clarification on failure. For content (hallucinated slots), instruct the model to use only evidence present in the input and to leave unknown slots empty, then validate every slot against the source: check that a returned city actually appears in (or fuzzy-matches) the transcript, and that values exist in the backing database before acting. Add a confidence field and gate actions on it, combined with upstream ASR confidence. Require explicit user confirmation for irreversible or high-stakes intents. Log mismatches to build evals. The principle: never let model output directly trigger an action, it passes through schema validation, grounding checks, and a confirmation policy first.

Q4.8

What is the difference between intent classification and slot filling, and how are they trained jointly? Intent classification is sentence-level: assign the utterance to one of K intents (“book_flight”). Slot filling is token-level sequence labeling: tag each word with a slot role using a BIO scheme (B-dest, I-dest, O), extracting the parameters. They are complementary, intent says *what*, slots say *with which arguments*. Classically they were separate models, but joint models (e.g. a shared encoder with a sentence-classification head for intent and a token-classification head for slots, as in JointBERT) train them together because they are correlated: knowing the intent constrains which slots are plausible, and vice versa. Joint training shares representations and improves both, especially on small data. With LLMs, the distinction blurs, a single structured-output call returns intent and slots together, but understanding the BIO sequence-labeling formulation still matters for fine-tuned, low-latency deployments and for designing the output schema.

Q4.9

Design the NLU layer for a voice agent that must respond within 300 ms of end-of-speech. A 300 ms budget after end-of-speech is tight, so I would architect for latency first. Use endpointing (VAD-based) to detect end-of-speech early and start NLU on partial transcripts where possible. Route by complexity: a fast on-device intent classifier (tens of milliseconds) handles the common, well-defined intents and returns immediately; only ambiguous or open-domain utterances fall through to an LLM, and even then I would stream the LLM and begin TTS on the first sentence. Pre-warm models to avoid cold starts, cache frequent interpretations, and run punctuation/ITN as a small fast model or fold it in. If an LLM is unavoidable on the hot path, use a small, quantized,

locally hosted model and speculative or constrained decoding to cut tokens. In practice, overlap stages: ASR finalization, NLU, and the start of response generation should pipeline, not run strictly sequentially. I would measure p95 end-of-speech to first-audio latency, not the mean, because the tail is what users feel.

Q4.10

What is an audio-LLM and how does it change SLU? An audio-LLM is a model that ingests audio directly, usually by encoding speech into embeddings or discrete tokens and feeding them into a language model alongside (or instead of) text, so it can reason over content *and* acoustics in one model (examples in spirit: Qwen-Audio, SALMONN, GPT-4o-style speech input). For SLU this is significant because it removes the cascaded boundary: the model can use prosody, tone, hesitation, and background cues that a transcript discards, doing intent, emotion, and even speaker context jointly. It also enables open-ended spoken instructions and audio question answering. The trade-offs are real: higher latency and cost, harder to debug (no intermediate transcript to inspect), more compute, and emerging but immature tooling and guardrails. I see audio-LLMs as the trajectory for rich, prosody-aware understanding, while cascaded ASR-then-LLM remains the pragmatic, debuggable default for most production agents today. Knowing both, and when each pays off, is the senior position.

References

- Chen, Q. Zhuo, Z. & Wang, W. (2019). BERT for joint intent classification and slot filling. *arXiv preprint arXiv:1902.10909*.
- Lugosch, L. Ravanelli, M. Ignoto, P. Tomar, V. S. & Bengio, Y. (2019). Speech model pre-training for end-to-end spoken language understanding. *Proceedings of Interspeech 2019*, 814–818.
- Tur, G. & De Mori, R. (Eds.). (2011). *Spoken language understanding: Systems for extracting semantic information from speech*. John Wiley & Sons.

Text-to-Speech & Voice Synthesis

Subscribe → aiengineeringinsider.substack.com/subscribe

*The third pillar, **Speak**. Generating natural, controllable speech from text: acoustic models, neural vocoders, prosody, voice cloning, and the modern diffusion and language-model TTS that now define the frontier.*

5.1 Overview

Text-to-Speech (TTS) synthesizes a speech waveform from text. Modern neural TTS is almost always a two-stage cascade plus a normalization front-end:

1. **Text front-end**, normalization (“\$5” → “five dollars”), optional grapheme-to-phoneme (G2P) conversion, and prosody/break prediction.
2. **Acoustic model**, text/phonemes → a mel-spectrogram. Autoregressive (**Tacotron 2**) or parallel/non-autoregressive (**FastSpeech 2**).
3. **Vocoder**, mel-spectrogram → waveform. The component that determines audio fidelity (**HiFi-GAN**, WaveNet, BigVGAN).

5.1.1 Autoregressive vs Parallel Acoustic Models

Tacotron 2 generates the mel frame-by-frame with attention over the text. It sounds natural but is slow and can fail catastrophically, attention collapse causes repeated or skipped words. **FastSpeech 2** predicts all frames in parallel using an explicit **duration predictor** (how long each phoneme lasts), plus pitch and energy predictors. It is far faster, more robust, and controllable, at a slight naturalness cost that modern variants have largely closed.

5.1.2 Vocoders

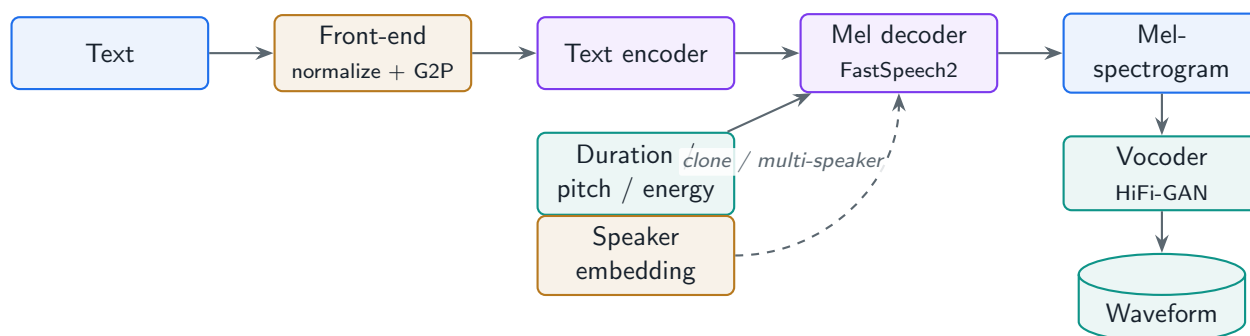
The vocoder reconstructs the waveform (including phase) from the mel-spectrogram. **WaveNet** was the breakthrough but autoregressive and slow; **HiFi-GAN** is the production default, a GAN vocoder that is fast, high-fidelity, and lightweight.

5.1.3 The Frontier: Voice Cloning, Diffusion, and LM-based TTS

Recent systems treat TTS as a generation problem over neural audio codec tokens. **VALL-E** casts TTS as language modeling over EnCodec tokens and clones a voice from a 3-second prompt. **Diffusion TTS** and flow-matching models (NaturalSpeech, Parler-TTS, F5-TTS) deliver state-of-the-art naturalness and

zero-shot cloning. **Voice conversion** changes speaker identity while keeping content. These also raise the misuse/safety issues covered in Chapter 7.

5.2 Architecture Diagram: Neural TTS



The vocoder, not the acoustic model, sets perceived audio quality. Teams often spend weeks tuning the text-to-mel model and ship with a mediocre vocoder, then wonder why output sounds robotic or buzzy. If your TTS sounds metallic, suspect the vocoder (or a mel-spec format mismatch between the two stages) before the acoustic model.

5.3 Production Code: Streaming TTS Interface

The `tts.py` module wraps a synthesizer behind a streaming-friendly interface so the voice agent can begin playback on the first audio chunk instead of waiting for the full utterance, essential for low perceived latency (Chapter 6).

`voice_agent/tts.py`

```

"""Text-to-speech with sentence-level streaming for low latency."""
from __future__ import annotations
import re
from typing import Iterator
import numpy as np

class StreamingTTS:
    """Synthesize and yield audio sentence-by-sentence.

    Splitting on sentence boundaries lets the agent start speaking the first
    sentence while later ones are still being generated, cutting time-to-first-
    audio - the latency users actually perceive in a conversation.
    """

    def __init__(self, engine, sample_rate: int = 22_050):
        self.engine = engine  # any model with .synthesize(text)->np
        self.sample_rate = sample_rate

```

```

@staticmethod
def _split_sentences(text: str) -> list[str]:
    # Keep the terminal punctuation; split on sentence enders.
    parts = re.split(r"(?<=[.!?])\s+", text.strip())
    return [p for p in parts if p]

def stream(self, text: str) -> Iterator[np.ndarray]:
    for sentence in self._split_sentences(text):
        audio = self.engine.synthesize(sentence) # (n_samples,) float32
        # Light de-click: short fade-in/out avoids boundary pops when
        # concatenating independently-synthesized sentences.
        audio = self._fade(audio, ms=8)
        yield audio

def synthesize_all(self, text: str) -> np.ndarray:
    return np.concatenate(list(self.stream(text)))

def _fade(self, audio: np.ndarray, ms: int) -> np.ndarray:
    n = int(self.sample_rate * ms / 1000)
    if len(audio) < 2 * n:
        return audio
    ramp = np.linspace(0.0, 1.0, n, dtype=audio.dtype)
    audio[:n] *= ramp
    audio[-n:] *= ramp[::-1]
    return audio

```

Sentence-level streaming is the single most impactful TTS latency optimization in a conversational agent: time-to-first-audio drops from “length of the whole reply” to “length of the first sentence”, which is what makes an agent feel responsive.

Concatenating independently synthesized sentences causes audible clicks and prosodic discontinuities at the joins. Apply short crossfades (as above) and, for quality-critical applications, synthesize with sentence context or use a model that supports streaming natively, rather than naive chunking.

5.4 Comparison Tables

5.4.1 Acoustic Models

Model	Type	Speed	Note
Tacotron 2	Autoregressive	Slow	Natural; attention can skip/repeat
FastSpeech 2	Parallel	Fast	Duration/pitch/energy control; robust
VITS	End-to-end (VAE+flow+GAN)	Fast	Text-to-wave in one model
VALL-E	LM over codec tokens	Med	Zero-shot cloning from 3 s prompt

5.4.2 Vocoders

Vocoder	Type	Note
WaveNet	Autoregressive	Breakthrough quality; too slow for real-time
HiFi-GAN	GAN	Production default: fast, high fidelity, light
BigVGAN	GAN (large)	Universal vocoder; robust across speakers
WaveGlow	Flow	Parallel; heavier than HiFi-GAN

For a conversational agent, default to **FastSpeech 2 (or VITS) + HiFi-GAN**: parallel, controllable, real-time, and robust against the word-skipping failures of autoregressive attention models. Reserve diffusion/LM-based TTS for offline, quality-first use cases (audiobooks, dubbing) where latency is not on the hot path.

When someone tells me their TTS “sounds robotic,” I bet on a mel-spectrogram parameter mismatch between the acoustic model and the vocoder before I even listen, and I win that bet more often than not. The vocoder is the most under-respected component in the stack: teams pour weeks into the text-to-mel model and bolt on whatever vocoder checkpoint was lying around, then blame the wrong half when it buzzes.

5.5 Mock Interview Questions & Answers

Q5.1

Walk me through a modern neural TTS pipeline end to end. Three stages plus a front-end. The text front-end normalizes the input, expanding numbers, dates, abbreviations, and currency to spoken form, and often converts graphemes to phonemes (G2P) and predicts prosodic breaks, because pronunciation and phrasing are ambiguous from raw text. The acoustic model maps text or phonemes to a mel-spectrogram; modern systems use a non-autoregressive model like FastSpeech 2 that predicts per-phoneme duration, pitch, and energy and generates all mel frames in parallel, or an end-to-end model like VITS. The vocoder converts the mel-spectrogram to a waveform, reconstructing phase; HiFi-GAN is the production default for fast, high-fidelity output. Speaker identity is injected via a speaker embedding for multi-speaker or cloning. The two halves must agree on mel-spectrogram parameters (n mels, hop, fmin/fmax) or you get artifacts. End to end: text to normalized phonemes to mel to waveform, with prosody and speaker as conditioning.

Q5.2

Why did the field move from autoregressive Tacotron 2 to parallel FastSpeech 2? Tacotron 2 generates mel frames one at a time with attention over the text, which has two problems at production scale: it is slow (latency grows with output length), and its soft attention can fail catastrophically, repeating or skipping words when attention collapses, especially on long or unusual text. FastSpeech 2 replaces soft attention with an explicit duration predictor that says how many mel frames each phoneme occupies, then expands and decodes all frames in parallel. This makes inference fast and length-stable (no skips/repeats), and the explicit pitch and energy predictors give direct, interpretable control over prosody. The historical cost was a small naturalness gap and the need for alignment supervision to train the duration predictor, but modern variants closed the quality gap. So the move was about robustness, speed, and controllability, exactly what a real-time product

needs.

Q5.3

Your TTS sounds robotic and metallic. How do you debug it? Metallic or buzzy output almost always points downstream at the vocoder or at a mel-spectrogram mismatch, not the text-to-mel model. First, check that the acoustic model and vocoder were trained with identical mel parameters, sample rate, n_fft, hop length, number of mel bins, fmin/fmax, and log scaling. A mismatch feeds the vocoder out-of-distribution mels and produces exactly that metallic artifact. Second, isolate the stages: vocode a ground-truth mel-spectrogram; if that sounds good, the acoustic model is producing bad mels (over-smoothed or wrong range); if it still sounds metallic, the vocoder is the culprit (under-trained, wrong checkpoint, or too small). Third, check the acoustic model for over-smoothing (an L1/L2 mel loss without adversarial or prosody terms produces flat, buzzy speech). Fourth, verify audio isn't clipping or being resampled incorrectly on output. The structured approach, swap in ground-truth mels to localize the fault, is what interviewers want to hear.

Q5.4

How does zero-shot voice cloning work, and what are the risks? Modern zero-shot cloning conditions synthesis on a short reference sample (often 3 seconds) without any per-speaker fine-tuning. Two common mechanisms: a speaker encoder produces a fixed embedding from the reference that conditions the acoustic model (the embedding-conditioning approach), or, as in VALL-E, TTS is cast as language modeling over neural codec tokens where the reference audio's codec tokens serve as a prompt, so the model continues in the same voice. Either way the model has learned, from many speakers, to disentangle content from timbre and to apply a new timbre on the fly. The risks are serious and a common interview follow-up: impersonation and fraud (voice phishing, defeating voice authentication), non-consensual cloning, and disinformation. Responsible deployment requires consent verification, audio watermarking and provenance (Chapter 7), spoofing countermeasures, and usage limits, the capability and its safeguards must ship together.

Q5.5

How do you control prosody, making the agent emphasize a word or ask a question? Prosody is pitch (F0), duration, energy, and pauses. In a FastSpeech 2-style model these are explicit predicted features, so you can intervene directly: scale the duration of a phoneme to lengthen a word, raise the pitch contour at the end for a question, or boost energy on the emphasized token. SSML (Speech Synthesis Markup Language) exposes this at the API level with tags for emphasis, breaks, pitch, and rate. For richer control, reference-based "style transfer" conditions synthesis on a reference clip with the desired delivery, or a style/global token (GST) learns a latent prosody space you can sample. LM-based and diffusion TTS can take natural-language style prompts ("say it cheerfully"). The trade-off: explicit control (FastSpeech2/SSML) is predictable and debuggable but coarse; learned style control is expressive but harder to steer precisely. For a question intonation specifically, the front-end should also detect the "?" and the model should be trained on questions so the rising terminal contour is natural.

Q5.6

Design a real-time TTS service for a voice agent with a strict latency budget. The metric that matters is time-to-first-audio, not total synthesis time. I would architect around streaming. Use a non-autoregressive acoustic model (FastSpeech2/VITS) plus HiFi-GAN so synthesis is fast and length-stable. Stream at the sentence level: split the LLM's reply on sentence boundaries and synthesize the first sentence immediately while later ones generate, and crossfade the joins. Better still, begin TTS on the LLM's streaming output, as soon as the first sentence is complete, start speaking. Pre-warm models to avoid cold starts, run on GPU or a well-quantized CPU path, and

cache common phrases (greetings, confirmations) as pre-rendered audio. Size the buffer so playback never underruns: generate slightly ahead of the playhead. Expose barge-in (Chapter 8) so the user can interrupt, which means TTS must be cancelable mid-stream. I would measure p95 time-to-first-audio and audio underrun rate, and keep the vocoder small enough to hit the budget on the target hardware.

Q5.7

What is a neural audio codec and why is it central to modern TTS? A neural audio codec (EnCodec, SoundStream, DAC) is an autoencoder that compresses a waveform into a sequence of discrete tokens via residual vector quantization and reconstructs it with high fidelity at very low bitrate. It matters for TTS because it turns continuous audio into a discrete token sequence, which lets you model speech with the same machinery as text language models. VALL-E and similar systems generate codec tokens autoregressively from text (and a voice prompt), then the codec decoder turns tokens back into a waveform, so TTS becomes “language modeling over audio tokens”. This unification brings LLM techniques (in-context learning, scaling, prompting) to speech generation, enabling zero-shot cloning from a short prompt and expressive, prosody-rich output. The codec also underpins streaming speech-to-speech systems (Chapter 6). The cost is that codec tokenization can lose fine detail and the multi-codebook structure complicates generation, but conceptually the codec is the bridge that made LM-style TTS possible.

Q5.8

How do you evaluate TTS quality, both subjectively and objectively? The gold standard is subjective: **MOS** (Mean Opinion Score), where human listeners rate naturalness on a 1–5 scale, and **CMOS** for comparing two systems side by side. MOS is what teams report, but it is expensive, slow, and sensitive to listener pool and instructions, so you cannot run it every commit. Objective proxies fill the gap: predicted-MOS models like UTMOS/NISQA estimate MOS automatically; mel-cepstral distortion (MCD) measures spectral distance to reference audio (needs parallel data); word/character error rate from running ASR on the synthesized speech checks intelligibility; and for cloning, speaker-similarity via a verification model’s cosine score. I would also test prosody appropriateness and edge cases (numbers, acronyms, foreign words, long sentences) since average MOS hides catastrophic mispronunciations. Practical workflow: gate releases on automatic metrics (predicted MOS, ASR-WER, speaker similarity) in CI, and run periodic human MOS/CMOS for calibration and final sign-off.

Q5.9

Compare diffusion-based and language-model-based TTS with the classic two-stage cascade. The classic cascade (FastSpeech2 + HiFi-GAN) is fast, controllable, robust, and cheap, which is why it dominates real-time products; its ceiling on naturalness and zero-shot cloning is lower. LM-based TTS (VALL-E family) models discrete codec tokens autoregressively, inheriting in-context learning so it clones voices from a short prompt and produces very natural, expressive speech, but it is slower (autoregressive over many tokens), can be unstable (repetition, hallucination), and is harder to control precisely. Diffusion/flow-matching TTS (NaturalSpeech 2/3, Voicebox, F5-TTS) iteratively denoises to a mel or latent, achieving state-of-the-art naturalness and strong zero-shot cloning with non-autoregressive parallelism, but each sample needs multiple denoising steps (latency) and the models are large. Decision: cascade for low-latency conversational agents; diffusion or LM TTS for offline, quality-first generation (audiobooks, dubbing, cloning) where you can spend compute. The frontier is shrinking diffusion steps and distilling LM-TTS to make these viable in real time.

Q5.10

The text front-end mispronounces “Dr.”, “2024”, and a product name. How do you fix pronunciation robustly? These are text-normalization and G2P failures, and the fix is layered, not a single model retrain. First, strengthen normalization: “Dr.” is context-dependent (Doctor vs Drive), so use a normalizer that disambiguates from context rather than a naive lookup, and handle numbers/dates/units with a robust ITN/TN module (“2024” as a year vs a quantity). Second, for out-of-vocabulary words and product/brand names, maintain a custom pronunciation lexicon that overrides the G2P model with explicit phonemes, and expose SSML `<phoneme>` tags so content authors can specify pronunciation. Third, for systematic G2P errors, fine-tune or correct the G2P model on domain terms. Fourth, add a regression test set of known-hard items (abbreviations, homographs, brand names, foreign words) and run it in CI so fixes do not regress. Put plainly: pronunciation correctness is mostly a front-end (TN/G2P/lexicon) engineering problem, and a curated lexicon plus SSML overrides is the reliable, controllable lever, far more dependable than hoping the model guesses right.

References

- Kong, J. Kim, J. & Bae, J. (2020). HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. *Advances in Neural Information Processing Systems*, 33, 17022–17033.
- Ren, Y. Hu, C. Tan, X. Qin, T. Zhao, S. Zhao, Z. & Liu, T.-Y. (2021). FastSpeech 2: Fast and high-quality end-to-end text to speech. *International Conference on Learning Representations*.
- Shen, J. Pang, R. Weiss, R. J. Schuster, M. Jaitly, N. Yang, Z. ... & Wu, Y. (2018). Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. *IEEE ICASSP 2018*, 4779–4783.
- Wang, C. Chen, S. Wu, Y. Zhang, Z. Zhou, L. Liu, S. ... & Wei, F. (2023). Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*.

Production Voice AI I — Codecs, VAD & Streaming

Subscribe → aiengineeringinsider.substack.com/subscribe

Where the three pillars converge under a real-time clock. This chapter is about the infrastructure that makes a voice agent feel instant: voice activity detection, turn-taking, neural codecs, streaming speech-to-speech, and the latency budget that governs every design decision.

6.1 Overview

A demo transcribes a file; a product holds a conversation. The gap between them is **real-time engineering**. The user’s perception of quality is dominated by one number: the **end-to-end latency** from when they stop speaking to when the agent starts speaking. Humans expect a conversational gap of roughly 200–300 ms; beyond 800 ms the interaction feels broken.

End-to-end response latency is the sum of: endpointing delay (deciding the user stopped), ASR finalization, NLU/LLM thinking, TTS time-to-first-audio, and network/ buffering. Every stage competes for the same 300 ms budget, so the system is designed around *overlap* (pipelining) and *early starts*, not sequential execution.

6.1.1 Voice Activity Detection and Endpointing

VAD classifies each short frame as speech or non-speech. It serves three roles: gating ASR (don’t transcribe silence, avoiding Whisper hallucination), saving compute, and, most subtly, **endpointing**: deciding when the user has *finished* a turn so the agent can respond. Endpointing is a latency/accuracy trade-off: wait too long and the agent feels sluggish; cut too early and you interrupt the user mid-thought. Silero VAD and WebRTC VAD are the common building blocks.

6.1.2 Neural Audio Codecs

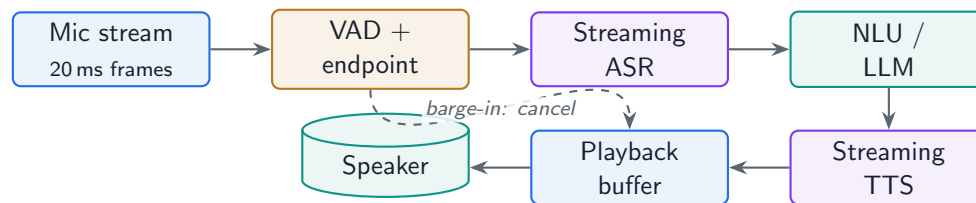
Neural codecs (EnCodec, SoundStream, DAC) compress audio to discrete tokens at very low bitrate with high reconstruction quality. They cut bandwidth for streaming, and, importantly, provide the discrete token vocabulary that powers LM-based TTS (Chapter 5) and end-to-end speech-to-speech models.

6.1.3 Streaming Speech-to-Speech

The frontier (Moshi, Hibiki, GPT-4o voice) is **full-duplex** speech-to-speech: models that listen and speak simultaneously over a shared stream, enabling natural overlap, backchannels (“mm-hmm”), and instant

interruption, without the rigid turn-taking of a cascaded pipeline.

6.2 Architecture Diagram: Real-Time Voice Loop



The dashed barge-in path is what separates a toy from a product. When the VAD detects the user speaking while the agent is talking, it must immediately cancel TTS playback and flush the pipeline. Without it, the user has to wait for the agent to finish a wrong answer, the most common complaint about early voice assistants.

6.3 Production Code: Streaming VAD with Endpointing

The `audio_io.py` module turns a raw frame stream into utterance boundaries with hangover-based endpointing, the logic that decides “the user has stopped talking”.

`voice_agent/audio_io.py`

```

"""Streaming VAD + endpointing for turn detection."""
from __future__ import annotations
from dataclasses import dataclass
import numpy as np

@dataclass
class VADConfig:
    frame_ms: int = 20
    speech_threshold: float = 0.5  # prob above which a frame is speech
    start_frames: int = 3          # consecutive speech frames to start
    hangover_ms: int = 600        # trailing silence => end of turn

class Endpointer:
    """Convert per-frame speech probabilities into utterance boundaries.

    Uses a start trigger (debounce against blips) and a hangover timer
    (trailing silence) to detect end-of-turn. The hangover length is the
    core latency/interruption trade-off: shorter feels snappier but risks
    cutting the user off mid-sentence.
    """

    def __init__(self, cfg: VADConfig = VADConfig()):
        self.cfg = cfg
        self.hangover_frames = cfg.hangover_ms // cfg.frame_ms
        self.reset()
  
```

```

def reset(self) -> None:
    self.in_speech = False
    self._speech_run = 0
    self._silence_run = 0
    self.buffer: list[np.ndarray] = []

def push(self, frame: np.ndarray, speech_prob: float):
    """Feed one frame. Returns the full utterance when a turn ends."""
    is_speech = speech_prob >= self.cfg.speech_threshold

    if not self.in_speech:
        self._speech_run = self._speech_run + 1 if is_speech else 0
        if self._speech_run >= self.cfg.start_frames:
            self.in_speech = True          # turn started
            self._silence_run = 0
            self.buffer.append(frame)
        return None

    # currently inside a turn
    self.buffer.append(frame)
    self._silence_run = 0 if is_speech else self._silence_run + 1
    if self._silence_run >= self.hangover_frames:
        utterance = np.concatenate(self.buffer) # end of turn
        self.reset()
        return utterance
    return None

```

This endpointer is the clock of the whole agent: it fires the event that triggers ASR finalization and the agent’s response. Tuning `hangover_ms` is the most user-visible knob in the system, it directly sets how patient versus how snappy the agent feels.

A fixed hangover timer cannot tell a thinking pause from end-of-turn, so it either interrupts slow speakers or feels laggy for fast ones. Production systems use **semantic endpointing**: combine the VAD silence signal with a small model that judges whether the transcript so far is a *complete* thought, so “I want to fly to...” (incomplete) waits longer than “I want to fly to Tokyo” (complete).

6.4 Comparison Tables

6.4.1 Latency Budget (target ≈ 300 ms)

Stage	Typical	Lever
Endpointing	200–600 ms	Hangover length; semantic endpointing
ASR finalization	50–200 ms	Streaming model; partial results
NLU / LLM	100–500 ms	Small model; streaming; routing
TTS first audio	50–200 ms	Sentence streaming; cached phrases
Network/buffer	20–100 ms	Codec; jitter buffer sizing

6.4.2 VAD & Codec Building Blocks

Component	Note
WebRTC VAD	Classic, ultra-light, energy/GMM based; fast but noise-sensitive
Silero VAD	Small neural VAD; robust, widely used in pipelines
EnCodec	Neural codec; 1.5–24 kbps; tokens for LM-TTS / S2S
Opus	Classic codec; the WebRTC real-time transport default

Stream everything and start early. The fastest perceived agent is not the one with the fastest models, it is the one that overlaps stages: begin ASR on partial audio, start the LLM on the finalized transcript, and start TTS on the LLM's first sentence. Optimize **time-to-first-audio**, not total processing time.

Two hard-won lessons. First, when an agent feels slow, look at the endpointer before the GPU bill; the latency is almost always hiding in a silence timeout, not the model. Second, the day we shipped barge-in without proper echo cancellation, the agent cheerfully interrupted itself because it heard its own voice through the speaker. AEC stops being optional the moment you keep the mic open during playback.

6.5 Mock Interview Questions & Answers

Q6.1

What is the end-to-end latency budget for a voice agent, and how do you spend it?

Conversational turn-taking in humans has gaps of roughly 200–300 ms, so that is the target from end-of-user-speech to start-of-agent-speech; past 800 ms it feels broken. The budget is consumed by five stages: endpointing (deciding the user stopped, often the largest single chunk at 200–600 ms), ASR finalization, NLU/LLM reasoning, TTS time-to-first-audio, and network/buffering. Because the sum easily exceeds 300 ms if run sequentially, the design trick is overlap: run ASR incrementally on partial audio so finalization is near-instant, start the LLM the moment the transcript finalizes and stream its tokens, and start TTS on the first complete sentence while the LLM is still generating. You also attack the biggest term, endpointing, with semantic endpointing rather than a long fixed timer. I would budget explicitly per stage, measure p95 not mean, and treat time-to-first-audio as the headline metric.

Q6.2

Explain endpointing. Why is a fixed silence timeout insufficient? Endpointing decides when the user has finished their turn so the agent can respond. The simplest method is a silence timeout: after N milliseconds of VAD-detected silence (the “hangover”), declare end-of-turn. The problem is that silence is ambiguous, people pause mid-thought to think, breathe, or search for a word. A short timeout interrupts them (“I’d like to book a flight to...” [agent jumps in]); a long timeout makes the agent feel sluggish on completed turns. A single fixed value cannot satisfy both. Production systems use semantic or acoustic-prosodic endpointing: combine the silence signal with cues like falling intonation and, critically, a lightweight model that judges whether the recognized text so far is a syntactically/semantically complete utterance, so incomplete thoughts get a longer grace period and complete ones respond fast. Some systems also predict turn-end from the audio

directly. The goal is adaptive patience rather than a one-size timer.

Q6.3

How do you implement barge-in so a user can interrupt the agent? Barge-in means the user can start speaking while the agent is talking and the agent stops immediately and listens. Implementation: keep the microphone open and the VAD running *during* TTS playback (full-duplex audio), and when the VAD detects genuine user speech, fire an interrupt that cancels TTS synthesis and flushes the playback buffer, then routes the new audio into ASR. Two hard problems: first, **acoustic echo cancellation**, the mic will pick up the agent's own speaker output, which would falsely trigger barge-in, so you need AEC (or headset/echo reference) to subtract the known playback signal. Second, distinguishing real interruptions from backchannels ("uh-huh", "yeah") that should *not* stop the agent, which needs a quick classifier or a short confirmation window. The pipeline must also be cancelable at every stage (TTS streaming, LLM generation) so an interrupt actually frees resources. Done right, barge-in is the single biggest contributor to an agent feeling natural.

Q6.4

What is a neural audio codec and where does it fit in a production voice system? A neural audio codec is a learned autoencoder (EnCodec, SoundStream, DAC) that encodes a waveform into discrete tokens via residual vector quantization and decodes them back with high fidelity at very low bitrate (often a few kbps vs tens for classical codecs at comparable quality). In production it plays two roles. As a transport codec, it cuts bandwidth for streaming audio to/from the cloud, important at scale and on poor networks. More importantly, it provides a discrete token representation of audio that unifies speech with language modeling: LM-based TTS (VALL-E) generates codec tokens, and full-duplex speech-to-speech models (Moshi) operate over codec streams. So the codec is both a bandwidth optimization and the representational substrate for modern generative speech. The trade-offs: neural codecs add encode/decode compute and latency versus classical codecs like Opus, and tokenization can lose subtle detail, so for pure real-time transport Opus is still common while neural codecs shine where you also need tokens for generation.

Q6.5

Contrast cascaded voice pipelines with end-to-end speech-to-speech models. A cascaded pipeline chains ASR, NLU/LLM, and TTS as discrete stages. Its strengths are modularity (swap any component), debuggability (inspect the transcript and the LLM output), reuse of mature text models, and controllability. Its weaknesses: latency accumulates across stages, prosody and emotion are lost at the ASR boundary, and rigid turn-taking makes overlap and backchannels awkward. End-to-end speech-to-speech models (Moshi, Hibiki, GPT-4o-style) process and generate audio directly, often full-duplex, so they preserve prosody, achieve very low latency, and support natural overlap and instant interruption. Their costs: they are black boxes (no transcript to audit or log), harder to constrain and add tools/guardrails to, expensive to train, and immature operationally. The pragmatic 2025 position is that cascaded pipelines dominate production because of controllability and tooling, while end-to-end S2S is the clear trajectory for naturalness and latency; many systems will be hybrids that keep a text trace for logging and tools while moving the conversational core toward S2S.

Q6.6

Why must you run a VAD before Whisper, and what else does the VAD buy you? Whisper is an attention decoder that behaves like a language model, so on silence or non-speech it hallucinates training-distribution phrases. A VAD gates the input so only speech reaches the model, eliminating that failure mode. Beyond hallucination prevention, the VAD saves compute (you do not transcribe silence, which on long calls is most of the audio), enables endpointing for turn-taking,

and lets you chunk audio at natural pauses so words are not split across windows in streaming. It also supports barge-in by detecting user speech during playback. The trade-off is that an overly aggressive VAD clips quiet speech onsets (dropping the first phoneme), so you keep a small pre-roll buffer before the detected speech start. So the VAD is not just an on/off switch; it is the timing and gating backbone of the real-time loop.

Q6.7

Design the audio transport for a cloud voice agent over an unreliable mobile network.

The enemies are latency, jitter, and packet loss. I would stream audio in small frames (20 ms) over a low-latency transport, WebRTC with Opus is the proven default, rather than uploading whole files, so processing overlaps capture. A jitter buffer on the receive side absorbs variable arrival times, sized as a trade-off between latency and smoothness (adaptive jitter buffering tunes this live). For packet loss, rely on the codec's packet-loss concealment and forward error correction (Opus has both), and design ASR to tolerate gaps. I would do VAD and possibly endpointing on the client to cut uplink bandwidth (send only speech) and to start the response clock early. Use a neural codec only if I need its tokens or its extreme bitrate savings on very constrained links, accepting the extra compute. On the downlink, stream TTS audio in chunks with its own jitter buffer and support barge-in cancellation. Throughout, I would degrade gracefully: lower bitrate, simpler model, or text fallback when the network is bad, and monitor mouth-to-ear latency and loss as first-class metrics.

Q6.8

A user complains the agent “talks over them.” Walk through your debugging. Talking over the user is a barge-in or endpointing failure, so I would localize which. First, is barge-in implemented and working? Check whether the mic and VAD stay active during TTS playback; if they are muted while the agent speaks (a common shortcut to avoid echo), the agent literally cannot hear the interruption, the fix is full-duplex audio with acoustic echo cancellation so the agent's own output does not self-trigger but the user's voice does. Second, if barge-in exists, is AEC failing so the agent hears itself and ignores the user, or is the interrupt firing but TTS not actually canceling (a non-cancelable synthesis/playback path)? Third, if the complaint is the agent responding before the user finishes, that is endpointing cutting off too early, the hangover is too short or semantic endpointing is missing, so it treats a thinking pause as end-of-turn. I would reproduce with logged audio and pipeline timestamps, check echo-reference alignment for AEC, verify the cancellation path end to end, and tune endpointing with semantic completion. The structured split, can't hear vs hears but doesn't stop vs stops too early, is the senior approach.

Q6.9

How do partial/streaming ASR results help latency, and what is the downside? Streaming ASR emits hypotheses as audio arrives instead of waiting for end-of-turn. This helps latency in two ways: the transcript is essentially ready the instant the user stops (finalization is near-zero because most of it was already computed), and you can begin downstream work, intent detection, even speculative LLM prompting, on partial text before the turn ends. It also enables live captions and lets endpointing use the evolving transcript for semantic completion. The downside is instability: partial results are revised as more context arrives, so words flicker and change, and any downstream action taken on a partial that later changes was wasted or wrong. So you distinguish tentative from committed text (only act on stable, finalized words), use a local-agreement policy to decide when a word is safe to commit, and avoid irreversible actions on partials. The trade-off is latency versus stability, and the engineering is about committing at the right moment.

Q6.10

What is acoustic echo cancellation and why is it essential for a hands-free voice agent? On a hands-free device the microphone picks up the agent’s own speaker output, so without intervention the system hears itself, which breaks ASR (it transcribes the agent), poisons barge-in (the agent’s voice falsely triggers “user is speaking”), and can cause feedback loops. Acoustic echo cancellation (AEC) removes this by using the known playback signal as a reference: an adaptive filter models the echo path (speaker to room to mic) and subtracts the predicted echo from the mic signal, leaving the user’s voice. It must adapt continuously because the echo path changes (the user moves, volume changes) and handle nonlinearities and double-talk (both speaking at once) gracefully, often paired with residual echo suppression and noise reduction. AEC is what makes full-duplex barge-in possible without a headset: the agent can keep its mic open during playback and still distinguish the user from itself. It is a classic DSP problem (with neural variants now) and a frequent gap in naive implementations that simply mute the mic during TTS, which then makes interruption impossible.

References

- Défossez, A. Copet, J. Synnaeve, G. & Adi, Y. (2023). High fidelity neural audio compression. *Transactions on Machine Learning Research*.
- Défossez, A. Mazaré, L. Orsini, M. Rérat, A. Kharitonov, E. Zeghidour, N. & others. (2024). Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*.
- Zeghidour, N. Luebs, A. Omran, A. Skoglund, J. & Tagliasacchi, M. (2021). SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30, 495–507.

Production Voice AI II — Safety, Eval & Deployment

Subscribe → aiengineeringinsider.substack.com/subscribe

The unglamorous engineering that decides whether a voice system survives contact with real users: anti-spoofing and watermarking, rigorous evaluation, on-device optimization, and the MLOps and observability that keep it healthy in production.

7.1 Overview

A model that scores well offline can still fail in production through latency, drift, abuse, or silent quality regressions. This chapter covers the four operational concerns that senior interviews probe: **security/safety**, **evaluation**, **deployment/optimization**, and **observability**.

- **Spoofing / presentation attacks**, replayed, synthesized (TTS), or voice-converted audio used to fool speaker verification or impersonate someone. Defended by **anti-spoofing / liveness** models (ASVspoof).
- **Deepfake provenance**, proving whether audio was AI-generated, via **watermarking** (e.g. AudioSeal) and detection classifiers.
- **Privacy**, voice is biometric PII; recordings, embeddings, and emotion inferences require consent, retention limits, and access control.

7.1.1 Evaluation Beyond a Single Number

Audio evaluation splits into **objective** metrics (WER, DER, MCD, PESQ/STOI for quality, EER for verification) and **subjective** ones (MOS/CMOS by human raters, now approximated by predicted-MOS models like UTMOS/NISQA). Mature teams gate releases on automatic metrics in CI and run periodic human evaluation for calibration.

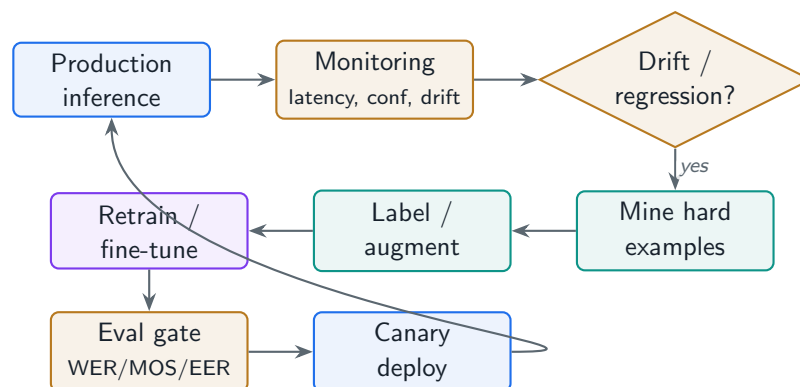
7.1.2 On-Device Optimization

Pushing models to the edge (privacy, latency, offline) requires **quantization** (int8/int4), **pruning**, **knowledge distillation**, and formats like ONNX, CoreML, TFLite, or GGML. The trade-off is always accuracy versus size/speed/power.

7.1.3 Observability

You cannot improve what you cannot see. Production voice systems log per-stage latency, ASR confidence, VAD/endpoint events, error rates, and audio quality indicators, and watch for **data drift** (the input distribution changing) that silently erodes accuracy.

7.2 Architecture Diagram: Voice MLOps Loop



The most valuable asset in a deployed voice system is the stream of *real production audio with low model confidence*. Mining these hard examples, labeling them, and folding them back into training is the flywheel that drives accuracy up over time, far more than chasing the latest architecture. Interviewers want to hear about this data loop, not just model choice.

7.3 Production Code: Audio Watermark Verification + Quality Gate

The `evaluate.py` module provides a release gate and a provenance check. Below it computes WER for the gate and verifies an audio watermark for AI-generated-content provenance.

`voice_agent/evaluate.py`

```

"""Evaluation gate (WER) and AI-audio provenance (watermark) checks."""
from __future__ import annotations
import numpy as np

def wer(reference: str, hypothesis: str) -> float:
    """Word Error Rate via Levenshtein edit distance over words."""
    ref, hyp = reference.split(), hypothesis.split()
    # dp[i][j] = edits to align ref[:i] with hyp[:j]
    dp = np.zeros((len(ref) + 1, len(hyp) + 1), dtype=np.int32)
    dp[:, 0] = np.arange(len(ref) + 1)
    dp[0, :] = np.arange(len(hyp) + 1)
    for i in range(1, len(ref) + 1):
        for j in range(1, len(hyp) + 1):
            cost = 0 if ref[i - 1] == hyp[j - 1] else 1
            dp[i][j] = min(

```

```

        dp[i - 1][j] + 1,          # deletion
        dp[i][j - 1] + 1,          # insertion
        dp[i - 1][j - 1] + cost    # substitution / match
    )
    return dp[len(ref)][len(hyp)] / max(len(ref), 1)

def release_gate(results: list[dict], max_wer: float = 0.15) -> bool:
    """Block a release if mean WER regresses past threshold."""
    scores = [wer(r["reference"], r["hypothesis"]) for r in results]
    mean_wer = float(np.mean(scores))
    print(f"[gate] mean WER={mean_wer:.3f} (max {max_wer})")
    return mean_wer <= max_wer

def verify_watermark(audio: np.ndarray, detector) -> dict:
    """Return provenance info for generated audio (e.g. AudioSeal)."""
    prob = float(detector.detect(audio))    # P(audio is AI-watermarked)
    return {"ai_generated": prob > 0.5, "confidence": prob}

```

The `release_gate` is a CI guardrail: a model change cannot ship if it regresses WER on the held-out set. The `verify_watermark` call supports a responsible-AI requirement, proving that audio the agent generated is detectable as synthetic.

Do not gate releases on aggregate WER alone. A change can improve the mean while regressing a critical slice, a specific accent, a noisy condition, or entity recognition. Gate on *per-slice* metrics and on entity error rate for the slots that drive actions, or you will ship a model that is better on average and worse where it counts.

7.4 Comparison Tables

7.4.1 Audio Quality & Verification Metrics

Metric	Measures
PESQ / POLQA	Perceptual speech quality vs a clean reference (telephony)
STOI	Short-time objective intelligibility
MOS / CMOS	Human-rated naturalness (gold standard, expensive)
UTMOS / NISQA	Predicted MOS (automatic proxy for CI)
EER	Equal Error Rate for speaker verification / anti-spoofing

7.4.2 On-Device Optimization Techniques

Technique	Trade-off
Quantization (int8/int4)	Big size/speed win; small accuracy loss; calibration needed
Pruning	Removes weights; needs sparse runtime to realize speedups
Distillation	Small student mimics large teacher; best quality-per-size
Format export (ONNX/CoreML/TFLite/GGML)	Hardware acceleration; runtime constraints

For edge ASR, distill then quantize: distil-whisper-class students recover most of the accuracy at a fraction of the size, and int8 quantization on top gives real-time CPU inference. Always re-measure WER *after* quantization on real device audio, post-training quantization can disproportionately hurt rare phonemes and quiet speech.

The most counterintuitive result I keep re-learning: a denoiser that scores beautifully on PESQ can make your ASR worse, because aggressive suppression strips cues the recognizer leaned on. I no longer trust perceptual metrics for an ASR front-end. I measure WER on enhanced versus raw audio and let the recognizer vote. Optimize the metric that matches the job, not the one that demos well.

7.5 Mock Interview Questions & Answers

Q7.1

How would you defend a voice-authentication system against spoofing? Voice biometrics face presentation attacks: replay (a recording of the user), synthetic speech (TTS cloning their voice), and voice conversion. Defense is layered. First, add a dedicated anti-spoofing / liveness model trained on attack data (ASVspoof-style) that detects artifacts of replay and synthesis, spectral discontinuities, missing breathiness, channel cues, and run it alongside the verification model, fusing scores. Second, use challenge-response or text-dependent prompts (random passphrases) so a static recording cannot satisfy a fresh challenge. Third, exploit liveness signals where available (multi-modal, device sensors). Fourth, treat voice as one factor, not the sole credential, for high-stakes actions, combine with possession or knowledge factors. Fifth, monitor for attack patterns and watermark your own TTS so synthetic audio from your stack is detectable. In practice, evaluate with the right metric: report EER and the tandem detection cost (t-DCF) under realistic attacks, not just clean-condition accuracy, because anti-spoofing degrades fast against novel attack types, so plan for continual updates.

Q7.2

What is audio watermarking and why does it matter for generative voice? Audio watermarking embeds an imperceptible, robust signal into generated audio so it can later be detected as AI-produced, ideally surviving compression, resampling, and re-recording. It matters because zero-shot voice cloning makes convincing fakes trivial, raising fraud, impersonation, and disinformation risks; watermarking provides provenance, a way to answer “did this come from a generative model?” Modern approaches (e.g. AudioSeal) embed the mark during or after synthesis and train a detector

that localizes watermarked segments even in edited audio. Limitations to acknowledge: watermarks can be attacked or stripped by a determined adversary, they only cover audio from cooperating generators (not an open-source model someone disabled it on), and detection is probabilistic. So watermarking is necessary but not sufficient, it pairs with spoofing detection, consent verification, and policy. Responsible deployment ships the watermark with the generator by default, and the detector is part of your provenance and trust-and-safety stack.

Q7.3

Design the evaluation strategy for an ASR model before and after deployment. Pre-deployment, I build a stratified test set that mirrors production: multiple accents, noise conditions, SNRs, domains, and the long tail, with consistent text normalization. I report WER and CER overall *and per slice*, plus entity error rate for action-driving terms, and latency (RTF, p95) on the target hardware. Releases pass through a CI gate that blocks regressions on any critical slice, not just the mean. For TTS or enhancement I add predicted-MOS and periodic human MOS/CMOS. Post-deployment, offline metrics are insufficient because the live distribution drifts, so I instrument production: log ASR confidence, retranscribe a sampled subset with a larger reference model or human labels to estimate live WER, track downstream proxies (task success, repeat/clarification rate, abandonment), and watch for drift in input characteristics. Hard, low-confidence examples are mined, labeled, and added to the test set and training data. I run canary deployments and A/B tests measuring both quality and business metrics. The principle: a fixed offline number is a starting point; production evaluation is continuous.

Q7.4

You need Whisper-quality ASR on a phone with no network. How do you get there?

Offline, on-device means hard constraints on size, memory, compute, and power, so I would compress aggressively while protecting accuracy. Start from a distilled model (distil-whisper-class) which keeps most accuracy at a fraction of size, or a small streaming model if live captions are needed. Apply int8 (or int4) quantization, preferably with calibration on representative audio, and export to an on-device-friendly runtime, whisper.cpp/GGML, CoreML on iOS, or TFLite/ONNX with the platform's NN accelerator. Use a VAD to avoid running the model on silence (saving battery) and chunk audio for streaming. Critically, re-measure WER on real device audio after every compression step, sliced by accent and noise, because post-training quantization disproportionately hurts rare phonemes and quiet speech, the mean can look fine while a slice collapses. If accuracy is short, quantization-aware training or a slightly larger student recovers it. Trade off: pick the largest model that hits the latency and power budget on the actual device, validated on-device, not on a dev GPU.

Q7.5

How do you detect and respond to data drift in a deployed voice system? Drift is the production distribution moving away from training: new accents, devices, background conditions, slang, or codecs. Because ground-truth labels are not available live, I monitor proxy signals: a drop in mean ASR confidence or a rise in low-confidence segments, increases in clarification/repeat rate and task abandonment, shifts in input feature statistics (SNR, duration, language ID distribution), and out-of-vocabulary or new-entity frequency. I also periodically sample production audio and get reference transcripts (human or a stronger model) to estimate live WER over time. When drift is detected, I localize it (which slice, which condition), mine and label the affected examples, augment to match the new condition, retrain or fine-tune, and pass the new model through the eval gate and a canary before full rollout. Guardrails in the meantime: route low-confidence traffic to a fallback (larger model, human, or a clarifying question). The whole thing is the MLOps loop, monitor, mine, label, retrain, gate, canary, and the key insight is using confidence and behavioral proxies because you rarely have live labels.

Q7.6

What are the privacy and compliance considerations for storing voice data? Voice is biometric personal data, so it is treated more strictly than ordinary logs. Considerations: obtain and record explicit consent for recording and for any secondary use like model training; minimize collection and apply retention limits (delete raw audio promptly, keep only what is needed); separate and protect biometric embeddings, which can re-identify and, under some regimes (GDPR, BIPA, CCPA), are specially regulated; encrypt in transit and at rest with strict access control and audit logs; support data-subject rights (access, deletion) which means you must be able to find and purge a user's data including derived embeddings; and be careful with sensitive inferences like emotion or health, which carry extra risk and may require separate consent. For training, prefer de-identification, aggregation, or on-device learning (federated) where feasible, and document data lineage. Cross-border transfer and sector rules (HIPAA for health, telecom rules for call recording, two-party consent laws) add constraints. Put plainly: privacy-by-design, consent, minimization, encryption, retention, and the ability to delete, baked into the architecture, not bolted on.

Q7.7

Walk me through quantizing a speech model. What breaks and how do you catch it? Quantization maps weights (and often activations) from fp32 to int8/int4 to shrink the model and speed up inference. Post-training quantization (PTQ) is fastest: calibrate activation ranges on a representative dataset, then quantize. What breaks: activation outliers blow up quantization ranges and crush precision for normal values (common in attention/LayerNorm), so per-channel quantization and outlier handling (SmoothQuant-style) help; dynamic-range-sensitive layers (the first conv on raw audio, the output projection) may need to stay in higher precision (mixed precision); and accuracy loss concentrates on rare phonemes, quiet speech, and accented audio rather than the average. To catch it, I re-evaluate WER/CER after quantization on a sliced test set and on real target-hardware audio, not just the mean and not on a GPU, and I compare per-slice deltas. If PTQ loses too much, I move to quantization-aware training, which fine-tunes with simulated quantization and usually recovers most accuracy. I also verify latency/power actually improved on-device, since speedups depend on runtime support for the chosen precision. The discipline is measure-after-quantize, per slice, on device.

Q7.8

What should you log and monitor in a production voice agent? Enough to debug any single interaction and to spot systemic regressions, while respecting privacy. Per interaction: per-stage latency (endpoint, ASR, NLU/LLM, TTS, total, p95), ASR confidence and final transcript, VAD/endpoint and barge-in events, NLU intent/slots with confidence, LLM tokens and any tool calls, TTS time-to-first-audio and underruns, and the outcome (task success, escalation, abandonment). Aggregate/system: error and timeout rates, model and prompt versions, traffic distribution (language, device, SNR) to detect drift, and quality proxies (clarification rate, repeat rate). I would trace each turn end-to-end with a correlation ID so a complaint maps to exact timestamps and audio. For privacy, store raw audio only with consent and short retention, redact PII in transcripts/logs, and access-control biometric data. I would set alerts on latency p95, error rate, confidence drops, and barge-in failure, and build dashboards plus an example explorer for mining hard cases. The goal is observability that supports both incident response and the retraining flywheel.

Q7.9

Design a canary/A-B rollout for a new ASR model. What do you measure and when do you roll back? I never flip 100% of traffic to a new model. First, the offline eval gate must pass (per-slice WER, latency). Then a canary: route a small percentage (1–5%) of live traffic to the new model while the rest stays on the incumbent, ideally with traffic matched on language/device so

the comparison is fair. I measure both model-quality proxies (ASR confidence distribution, sampled live WER via human/reference labels) and downstream/business metrics that actually matter (task success rate, clarification and repeat rate, handle time, escalation/abandonment, latency p95), plus error/timeout rates. Guardrail metrics with pre-defined thresholds trigger automatic rollback: if confidence drops, latency p95 breaches SLA, error rate spikes, or task success regresses beyond a margin, revert immediately. If the canary is healthy for a set period and sample size (enough for statistical significance), ramp gradually (5 to 25 to 50 to 100%), watching at each step and especially watching slices that looked weak offline. I would also keep the ability to shadow-run (mirror traffic to the new model without serving its output) for risk-free comparison before the canary. Roll back on any guardrail breach; promote only on sustained, significant improvement.

Q7.10

How do you measure speech-enhancement or denoising quality, and what is the catch?

Speech enhancement is judged on both quality and intelligibility against a clean reference where available. Objective metrics: PESQ/POLQA for perceptual quality, STOI for intelligibility, SI-SDR (scale-invariant signal-to-distortion ratio) for separation quality, and SNR improvement. For real-world audio with no clean reference, use non-intrusive predictors (DNSMOS, NISQA). The catch, and the senior point, is that optimizing these metrics can hurt the downstream task: aggressive denoising that maximizes PESQ often introduces artifacts or removes speech cues that *raise* ASR WER, because what sounds cleaner to a human is not necessarily what an ASR model needs. So the metric that ultimately matters is the end task: measure ASR WER (or whatever the enhancement feeds) on enhanced versus unenhanced audio, not just perceptual scores. Many production pipelines find that a denoiser tuned for human listening degrades ASR and instead train enhancement jointly with, or evaluated by, the recognizer. Always close the loop on the real objective.

References

- Reddy, C. K. A. Gopal, V. & Cutler, R. (2021). DNSMOS: A non-intrusive perceptual objective speech quality metric to evaluate noise suppressors. *IEEE ICASSP 2021*, 6493–6497.
- Roman, R. S. Fernandez, P. Défossez, A. Furon, T. Tran, T. & Elshahar, H. (2024). Proactive detection of voice cloning with localized watermarking. *Proceedings of the 41st International Conference on Machine Learning*.
- Wu, Z. Yamagishi, J. Kinnunen, T. Hanilçi, C. Sahidullah, M. Sizov, A. . . . & Ling, Z.-H. (2017). ASVspoof: The automatic speaker verification spoofing and countermeasures challenge. *IEEE Journal of Selected Topics in Signal Processing*, 11(4), 588–604.

Agentic Voice AI & Specialized Topics

Subscribe → aiengineeringinsider.substack.com/subscribe

The capstone. All three pillars become a single agent that listens, reasons, acts via tools, and speaks, full-duplex, interruptible, and grounded. Plus the specialized topics (speech translation, multilingual ASR) that round out a senior profile.

8.1 Overview

An **agentic voice AI** is not just a chatbot with a microphone. It perceives (ASR + understanding), **reasons and plans**, **acts** on the world through tools and APIs, remembers context, and responds in speech, all within the real-time loop of Chapter 6. The orchestration is the hard part: coordinating streaming ASR, an LLM with tool calling, TTS, barge-in, and state, while staying responsive.

- **Perception**, streaming ASR + SLU (Chapters 2–4) with confidence.
- **Cognition**, an LLM that plans, calls tools/functions, and tracks dialogue state; grounded in retrieval where needed.
- **Action**, tool/API execution with confirmation for irreversible steps.
- **Expression**, streaming TTS (Chapter 5) with prosody and barge-in.
- **Orchestration**, the event loop that overlaps all of the above under a latency budget and handles interruptions.

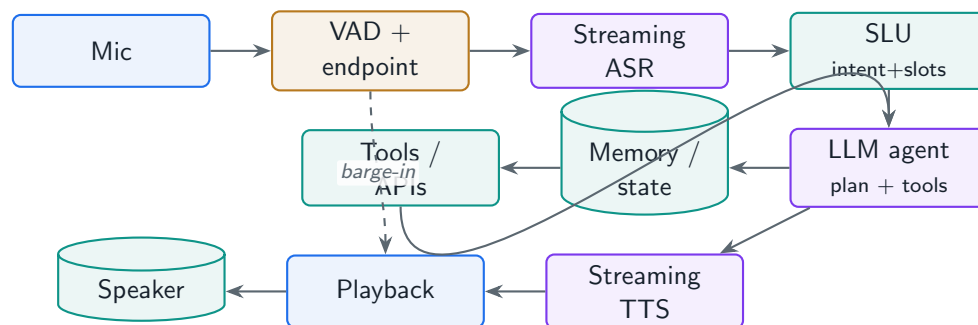
8.1.1 Tool Calling and Grounding

The LLM decides *when* to call a tool (search a database, book the flight, look up an order) and the orchestrator executes it, feeds the result back, and continues. For factual questions, retrieval-augmented generation grounds answers to reduce hallucination, vital when the agent speaks with authority and the user cannot easily verify.

8.1.2 Specialized Topics

Speech translation (cascaded ASR→MT→TTS, or direct S2ST like SeamlessM4T) enables cross-lingual agents. **Multilingual ASR** (Whisper, USM, MMS) and **code-switching** handle global users. **Audio captioning / QA** and **environmental sound** extend agents beyond speech to general audio understanding.

8.2 Architecture Diagram: The Full Voice Agent



A voice agent is as much a concurrency problem as an ML problem. The models are mostly off-the-shelf; the differentiator is the orchestrator that runs ASR, LLM, and TTS as overlapping streams, cancels cleanly on barge-in, and manages state, all while holding the 300 ms budget. Senior interviews spend more time here than on any single model.

8.3 Production Code: The Orchestrator (Capstone)

The `pipeline.py` module wires every previous chapter together. This is the `--demo` entry point of the book's project: it ties FeatureExtractor (Ch.1), ASR (Ch.2–3), NLU (Ch.4), TTS (Ch.5), and the endpointer (Ch.6) into one loop.

voice_agent/pipeline.py

```

"""End-to-end voice-agent orchestrator (capstone of all chapters)."""
from __future__ import annotations
from dataclasses import dataclass

@dataclass
class Turn:
    transcript: str
    intent: str
    response: str
    latency_ms: float

class VoiceAgent:
    """Coordinate Hear -> Understand -> Act -> Speak under a latency budget.

    Each component is the module explained in its chapter. The orchestrator
    overlaps stages and supports barge-in cancellation; here we show the
    turn-level control flow.
    """

```

```

def __init__(self, asr, nlu, agent_llm, tts, tools):
    self.asr = asr          # Ch.2-3  WhisperRecognizer / CTCRecognizer
    self.nlu = nlu          # Ch.4    LLMUnderstanding
    self.llm = agent_llm    # Ch.8    planning + tool calling
    self.tts = tts          # Ch.5    StreamingTTS
    self.tools = tools      # Ch.8    name -> callable

def handle_turn(self, audio_path: str) -> Turn:
    import time
    t0 = time.perf_counter()

    # 1) HEAR: transcribe with word-level confidence
    asr_out = self.asr.transcribe(audio_path)
    text = asr_out["text"]
    min_word_prob = min((w.prob for w in asr_out.get("words", [])),
                        default=1.0)

    # 2) UNDERSTAND: intent + slots, gated by ASR confidence
    interp = self.nlu.interpret(text, asr_min_word_prob=min_word_prob)
    if interp.needs_clarification:
        reply = "Sorry, could you say that again?"
        self._speak(reply)
        return self._finish(text, interp.intent, reply, t0)

    # 3) ACT: let the agent plan and optionally call a tool
    reply = self._reason_and_act(interp)

    # 4) SPEAK: stream audio so playback starts on sentence one
    self._speak(reply)
    return self._finish(text, interp.intent, reply, t0)

def _reason_and_act(self, interp) -> str:
    if interp.intent in self.tools:
        result = self.tools[interp.intent](**interp.slots)
        return self.llm.summarize(intent=interp.intent, result=result)
    return self.llm.respond(intent=interp.intent, slots=interp.slots)

def _speak(self, text: str) -> None:
    for _chunk in self.tts.stream(text):
        pass # in production: push chunk to playback; honor barge-in

def _finish(self, text, intent, reply, t0) -> Turn:
    import time
    return Turn(text, intent, reply,
                (time.perf_counter() - t0) * 1000.0)

```

This control flow is deliberately readable; the production version makes each stage async and cancelable so barge-in can abort `_speak` mid-stream and the LLM call mid-generation. Run it with `python -m voice_agent.pipeline` to see a full turn execute end-to-end on bundled audio.

Let an agent take irreversible actions (payments, cancellations, sending messages) only behind explicit voice confirmation and slot validation. The combination of ASR errors, NLU hallucination, and tool execution is where voice agents cause real-world harm, “did you say cancel *all* my orders?” must be confirmed, not assumed.

8.4 Comparison Tables

8.4.1 Speech Translation Approaches

Approach	Latency	Note
Cascaded (ASR→MT→TTS)	Higher	Modular, debuggable; errors compound
Direct S2ST (SeamlessM4T)	Lower	Preserves prosody; fewer error stages
Simultaneous translation	Lowest	Translates before sentence ends; partial

8.4.2 Multilingual / Frontier Models

Model	Scope
Whisper	99 languages; transcription + X-to-English translation
MMS (Meta)	1000+ language ASR/TTS coverage
SeamlessM4T	Unified speech/text translation, many directions
Moshi / GPT-4o	Full-duplex real-time speech-to-speech dialogue

Build the agent as a state machine with clear, cancelable stages and explicit confirmation gates, then layer streaming and barge-in on top. A clean, debuggable control flow that you can reason about beats a clever monolith, especially when an interviewer asks you to extend it live (add a new tool, handle a failure) on the whiteboard.

The uncomfortable truth about voice agents: the models are the easy part. The hard part is an async state machine that overlaps ASR, the LLM, and TTS, cancels cleanly on barge-in, and never executes “cancel all my orders” without a confirmation. I have seen far more agents fail on orchestration and a missing confirmation gate than on model accuracy. Spend your design time there, not on benchmarking encoders.

8.5 Mock Interview Questions & Answers

Q8.1

Design a complete voice agent from microphone to speaker. What are the components and the hard parts? The pipeline is perceive, understand, reason/act, express, all under an orchestrator. Audio streams in 20 ms frames through AEC and a VAD that does endpointing; streaming

ASR produces a transcript with confidence; SLU extracts intent and slots; an LLM plans, tracks dialogue state, and calls tools/APIs (grounded with retrieval for facts); results come back and the LLM forms a reply; streaming TTS speaks it, starting on the first sentence; barge-in lets the user interrupt at any time. The hard parts are not the models, which are largely off-the-shelf, they are the orchestration and the edges: holding the 300 ms latency budget by overlapping stages, implementing barge-in with echo cancellation, semantic endpointing so the agent does not cut people off, propagating ASR confidence so it asks instead of guessing, confirming irreversible actions, managing state and interruptions concurrently, and degrading gracefully when a stage fails or the network drops. I would build it as a cancelable async state machine and instrument every stage. The strong signal is spending your answer on concurrency, latency, and failure handling, not on which ASR model to pick.

Q8.2

How does tool calling work in a voice agent, and how do you keep it safe and fast? The LLM is prompted with a set of tool/function schemas and decides, given the understood request and dialogue state, whether to call one and with what arguments; the orchestrator executes the tool, returns the result to the LLM, and the LLM continues to a spoken reply. To keep it safe: validate tool arguments against schemas and against real data (a flight must exist before booking), require explicit voice confirmation for irreversible or high-stakes calls, scope credentials with least privilege, and never let raw model output trigger an action without a guardrail. To keep it fast: run tools in parallel where independent, stream the LLM so TTS can start on early tokens, cache deterministic lookups, set tight timeouts with graceful fallbacks, and keep the hot-path model small. I would also handle tool failures conversationally (“I couldn’t reach the booking system, want me to retry?”) rather than crashing, and log every tool call with its arguments and result for audit and debugging. The interplay of safety (confirm, validate) and latency (parallel, stream, timeout) is the core issue.

Q8.3

How do you reduce hallucination when a voice agent answers factual questions? Spoken hallucination is especially dangerous because the user hears a confident answer and cannot easily check sources. The primary defense is grounding: retrieval-augmented generation that fetches authoritative documents or database records and instructs the model to answer only from them, citing or deferring when the evidence is absent. Beyond RAG: constrain the model to tools for anything factual (look it up rather than recall it), have it express uncertainty and ask clarifying questions instead of guessing, set a lower temperature on factual turns, and validate any concrete claims (prices, availability) against systems of record before speaking. For voice specifically, keep answers short and offer to send details, so a wrong long monologue is less likely. I would also monitor for hallucination via user corrections and sampled review, and feed failures back into prompts/evals. The framing: grounding plus tool use plus calibrated uncertainty, with validation before any spoken factual assertion.

Q8.4

Cascaded versus direct speech-to-speech translation for a real-time interpreter. Which and why? Cascaded translation runs ASR, then machine translation, then TTS. It is modular, debuggable, and lets you use the best model for each stage and inspect the intermediate text, but errors compound across three stages, latency accumulates, and prosody/emotion are lost at the ASR boundary. Direct speech-to-speech translation (SeamlessM4T-style) maps source audio to target audio (or via discrete units) more directly, reducing error stages, latency, and prosody loss, and it can preserve some vocal characteristics, but it is harder to debug and to constrain. For a real-time interpreter where latency and naturalness are paramount, I lean toward direct or unit-based S2ST, and for the lowest latency I would add simultaneous translation that begins before the speaker finishes a sentence (accepting some revision). For a setting that needs auditability and per-stage

control (legal, medical with human review), cascaded is safer because you can show and correct the transcript and translation. So: direct/simultaneous for live conversational interpreting, cascaded where traceability and control dominate.

Q8.5

How would you give a voice agent memory across a long conversation and across sessions?

Two timescales. Within a conversation, I maintain explicit dialogue state (filled slots, confirmations, recent references) and a running summary so the LLM context does not grow unbounded, I keep recent turns verbatim and compress older ones into a summary, plus the structured state. Across sessions, I persist long-term memory in a store: user profile and preferences, past tasks/orders, and salient facts, retrieved at the start of a session and via RAG mid-conversation when relevant. I would separate volatile working memory from durable memory, and write to durable memory deliberately (the agent decides what is worth remembering, or rules extract it) rather than dumping everything. Practical concerns: keep retrieval fast to protect the latency budget, resolve coreference (“the one from last time”), respect privacy (memory is personal data, consent, retention, deletion), and avoid stale or wrong memories driving bad behavior, so memory entries carry provenance and can be corrected. The architecture is working state plus summarized context plus a retrieved long-term store, balanced against latency and privacy.

Q8.6

What changes when you move from a turn-based agent to a full-duplex one? Turn-based agents enforce strict alternation: the user speaks, then the agent speaks, mediated by endpointing. Full-duplex agents listen and speak simultaneously over a shared stream (Moshi, GPT-4o voice), which unlocks natural behaviors, backchannels (“mm-hmm” while the user talks), instant interruption, and overlap, but changes the engineering substantially. You no longer have a clean endpoint event to trigger processing; the model must continuously decide when to speak, stay silent, or yield. Echo cancellation becomes mandatory because the agent’s output and the user’s input coexist live. State and context update continuously rather than per turn. Barge-in is native rather than bolted on. Architecturally, cascaded pipelines strain under full-duplex because their stages assume turns, which is why full-duplex tends to favor end-to-end speech-to-speech models that operate on continuous audio streams. The trade-off is naturalness and latency (full-duplex) versus controllability, tooling, and auditability (turn-based cascaded). I would call out that full-duplex is the frontier and that most production agents are still turn-based for the control it gives.

Q8.7

Your voice agent works in English but fails on code-switching and accented speech. How do you fix it? Code-switching (mixing languages mid-utterance) and accents break systems tuned on monolingual, native speech. First, the ASR: use a multilingual model (Whisper, USM, MMS) rather than an English-only one, and for code-switching specifically, models trained on mixed-language data, since per-utterance language ID fails when languages alternate within a sentence; I would fine-tune on code-switched and accented audio representative of my users. Second, evaluate per accent and per language pair, not in aggregate, to see where it actually fails. Third, downstream, make NLU robust to the resulting transcripts (an LLM handles mixed-language text better than a rigid classifier). Fourth, expand lexicons and language models with the relevant vocabulary and named entities. Fifth, collect real user audio (with consent), mine the failing slices, label, and retrain, the data flywheel matters most here because the failures are distributional. For TTS, use multilingual voices that can render mixed-language replies. The key insight is that this is primarily a data and model-coverage problem: pick multilingual/code-switch-capable models and close the loop with in-domain accented data, measured per slice.

Q8.8

Design observability and evaluation for a deployed voice agent. What does success look like? I evaluate at two levels: component and task. Component metrics (ASR WER/confidence, NLU intent/slot accuracy, TTS MOS, latency per stage) catch where a turn breaks. But the metric that defines success is task-level: did the user accomplish their goal? So I track task success rate, turns-to-completion, clarification/repeat rate, barge-in frequency, escalation-to-human rate, and abandonment, plus end-to-end latency p95 and barge-in responsiveness. For observability, every turn gets a correlation ID tracing audio in, ASR, NLU, LLM/tool calls, TTS, and timings, so any complaint maps to exact events and (consented) audio. I monitor drift via confidence and behavioral proxies, sample turns for human review, and mine low-confidence and failed tasks back into evals and training. Releases pass per-slice offline gates, then canary/A-B on live task metrics with automatic rollback on guardrail breaches. Success looks like high task completion at low latency with low escalation, sustained as the input distribution shifts, not a single good WER number. The experienced view is optimizing the user's task outcome, with component metrics as diagnostics.

Q8.9

Walk through what happens, stage by stage, in the 800 ms after a user says “cancel my 3pm meeting.” End-of-speech is detected by VAD/semantic endpointing (the transcript “cancel my 3pm meeting” is a complete thought, so it fires promptly rather than waiting a full hangover). Streaming ASR has been transcribing throughout, so finalization is near instant, yielding the text plus word confidences; “3pm” and “meeting” should be high-confidence. SLU maps this to intent `cancel` with slot time=15:00, and because `cancel` is irreversible, the agent does not execute yet, the orchestrator routes to a confirmation. The LLM, with calendar context (memory/RAG), identifies which event matches 3pm and forms “You have a 3pm sync with Priya, cancel it?” Streaming TTS begins speaking that first sentence within 100 ms of the LLM's first tokens, so the user hears a response well within the budget. The mic stays open with AEC so the user can barge in. On the user's “yes”, a second short turn confirms, the calendar tool executes the cancellation (validated against the real event ID), and the agent confirms success. The points to hit: streaming makes finalization cheap, semantic endpointing fires fast on a complete utterance, irreversible intent triggers confirmation not action, TTS streams to hit time-to-first-audio, and the tool call is validated, all overlapped, not sequential.

Q8.10

What are the biggest open challenges in agentic voice AI today? Several. Latency versus capability: richer reasoning, tools, and grounding all add latency, and holding sub-second response while doing more is unsolved at the high end. Full-duplex naturalness: end-to-end speech-to-speech gives natural overlap and interruption but sacrifices the controllability, tooling, auditability, and guardrails that cascaded systems provide, reconciling the two is open. Robustness in the wild: accents, code-switching, noise, far-field, and the long tail still degrade systems that look great in demos. Safety and trust: hallucination spoken with confidence, voice cloning and spoofing, watermarking that adversaries can strip, and irreversible tool actions all need better guarantees. Evaluation: we lack good automatic proxies for conversational task success and naturalness, so iteration is slow and expensive. Grounding and memory: reliable long-term memory and factual grounding without latency blow-up. Privacy: voice is biometric, and on-device or privacy-preserving learning is still maturing. And cost at scale. The honest senior answer is that the models are increasingly commoditized; the open problems are systems problems, latency, safety, robustness, evaluation, which is exactly where engineering effort and these interviews concentrate.

References

- Communication, S. Barrault, L. Chung, Y.-A. Meglioli, M. C. Dale, D. Dong, N. ... & others. (2023). SeamlessM4T: Massively multilingual and multimodal machine translation. *arXiv preprint arXiv:2308.11596*.
- Lewis, P. Perez, E. Piktus, A. Petroni, F. Karpukhin, V. Goyal, N. ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Pratap, V. Tjandra, A. Shi, B. Tomasello, P. Babu, A. Kundu, S. ... & Auli, M. (2024). Scaling speech technology to 1,000+ languages. *Journal of Machine Learning Research*, 25, 1–52.

Real-World System Design Case Studies

Subscribe → aiengineeringinsider.substack.com/subscribe

Twenty production audio/speech systems from top companies, each answered the way a Staff engineer would on a whiteboard at Meta, Google, OpenAI, Anthropic, or NVIDIA: requirements and scale, architecture and data flow, an explicit latency or scaling budget, evaluation, safety, cost, and a defended recommendation.

9.1 The Staff-Level System-Design Framework

A speech/audio system-design interview rewards a repeatable method, not memorized model names. Every case below is answered with the same condensed playbook, distilled from the full seventeen-step framework into five moves you can run in any interview.

1. **Requirements** Functional goal plus non-functional targets: latency, QPS/concurrency, accuracy, languages, placement, privacy. State assumptions and put numbers on them.
2. **Architecture** Ingestion → preprocessing → model layer → (LLM/agent) → post-processing → delivery. Draw the data flow.
3. **Latency / Scale** A budget table for real-time systems, or a throughput/GPU/sizing estimate for batch and high-QPS systems.
4. **Eval / Safety / Cost** The metric that defines success (WER, MOS, DER, EER), the abuse and privacy risks with mitigations, and the cost levers.
5. **Tradeoffs + Recommendation** The decisive tradeoff, then a clear, defended choice with the main bottleneck and what to improve next.

The most common senior-interview mistake is jumping to “use Whisper” or “use an LLM” before stating constraints. Lead with the **latency budget** and the **placement decision** (device vs cloud), they cascade into every other choice. Every case below therefore opens with requirements and closes with a recommendation.

The latency budgets, throughput numbers, and GPU sizing in this chapter are **illustrative order-of-magnitude estimates** meant to support reasoning, not measured benchmarks, verify against your own hardware and current model versions. Most companies also do not publish full internal architectures; where a design is reconstructed from papers, talks, and public posts, treat it as a plausible reference design rather than the vendor’s exact implementation.

Two field rules for reading the cases that follow. One: when you control the capture, separate speakers into channels at the source. It beats any diarization model, and I have watched teams burn months on clustering they never needed. Two: treat vendor blog benchmarks as marketing until you reproduce them. The only number that matters is the one you measured on your own data and your own hardware.

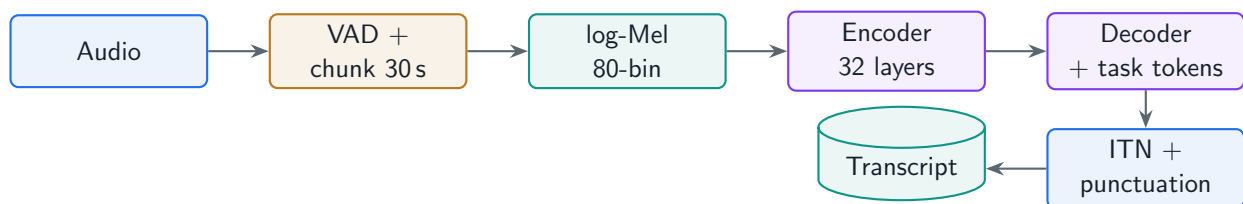
9.2 Case 1 — OpenAI Whisper: Robust Batch ASR at Scale

Brief: transcribe arbitrary audio in 99 languages, robust to noise, accent, and domain shift, with no per-customer tuning.

Functional: speech-to-text, language-ID, optional translation. **Non-functional:** batch/offline latency acceptable, maximize GPU throughput, “just works” zero-shot.

Requirements & scale. Assume an enterprise transcription backend ingesting 10^5 hours/day. Target cost per audio-hour, not per-request latency; correctness and language coverage dominate. Throughput, batching efficiency, and GPU utilization are the system levers.

Architecture. Whisper is a Transformer encoder-decoder fed 80-bin log-Mel features over fixed 30-second windows. Robustness is bought with data: 680,000 hours of weakly supervised multilingual audio, trained multitask (transcribe, translate, language-ID, VAD) via special decoder tokens. In production you serve **faster-whisper** (CTranslate2, int8) for $\sim 4\times$ throughput, VAD-gate input to suppress silence hallucination, chunk long audio with overlap, and batch on GPU. **large-v3-turbo** (2024) keeps the 32-layer encoder but prunes the decoder to 4 layers (1.55 B $\rightarrow$ 809 M) for a large speed win at minor accuracy cost.



Model	RTF (GPU)	Rel. WER	Sizing note
large-v3	~ 0.08	best	one modern GPU serves many batched streams
large-v3-turbo	~ 0.03	+3–5%	$\sim 2\times$ density; default for cost
distil/medium	~ 0.02	+8–12%	cheapest; fine-tune for a domain

Table 9.1: Whisper serving: batch ASR throughput vs accuracy trade.

Evaluation, safety & cost. Track WER/CER per language and per acoustic slice (not just the mean), and RTF for capacity planning. Safety: PII redaction on transcripts, retention limits on raw audio. Cost levers: turbo/distil, int8 quantization, dynamic batching, and spot GPUs for the offline queue.

Tradeoff & recommendation. The attention encoder-decoder buys top offline accuracy but cannot stream and hallucinates on non-speech. Recommendation: serve **large-v3-turbo** on **faster-whisper** with VAD gating and dynamic batching; reserve large-v3 for a quality tier and fine-tuned medium for hot domains. Bottleneck is GPU throughput, attack it with batching and quantization first.

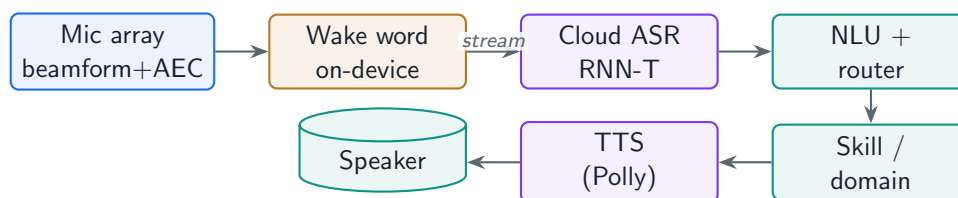
9.3 Case 2 — Amazon Alexa: Far-Field Voice Assistant

Brief: an always-on, far-field smart-speaker assistant that wakes on “Alexa,” understands a request, and acts, across millions of devices.

Functional: wake-word, ASR, NLU, skill routing, TTS. Non-functional: sub-second response, milli-watt on-device wake, very low false-accept rate, billions of requests.

Requirements & scale. Hundreds of millions of devices, peak concurrency in the millions. Hard targets: wake-word power budget on a low-end DSP, FAR of a few false wakes per device per day, and end-to-end response well under one second.

Architecture. The device does the cheap, privacy-sensitive, latency-critical work; the cloud does the heavy lifting. A mic array runs beamforming, AEC, and noise reduction; a tiny on-device wake-word CNN (<100 K params) listens continuously. Only after a wake does audio stream to the cloud, where a Conformer/RNN-T model transcribes, NLU extracts intent and slots, a router dispatches to a first-party domain or third-party skill, and the reply is synthesized (Polly) and streamed back. Cloud re-verification of the trigger cuts false wakes.



Stage	Budget (ms)	Note
Wake + endpoint	0 / 200–500	on-device; endpoint dominates perceived wait
Network up	30–80	device to region
Cloud ASR (stream)	100–200	finalizes near end-of-speech
NLU + skill	50–200	cached intents fast; skill call varies
TTS first audio	50–150	stream first sentence

Table 9.2: Alexa turn budget; target < 1 s perceived, much of it endpointing.

Evaluation, safety & cost. Metrics: wake FAR/FRR (DET curve), ASR WER, intent accuracy, and task completion. Safety: on-device wake keeps audio local until intent; privacy controls and audit logs. Cost: keep wake/endpoint on device; batch cloud ASR; cache TTS for stock phrases.

Tradeoff & recommendation. On-device wake trades a little accuracy for privacy, power, and latency; cloud ASR trades privacy for capability. Recommendation: two-stage wake (tiny detector → cloud verifier), streaming RNN-T, and semantic endpointing to cut the biggest latency term. Bottleneck is endpointing, not model compute.

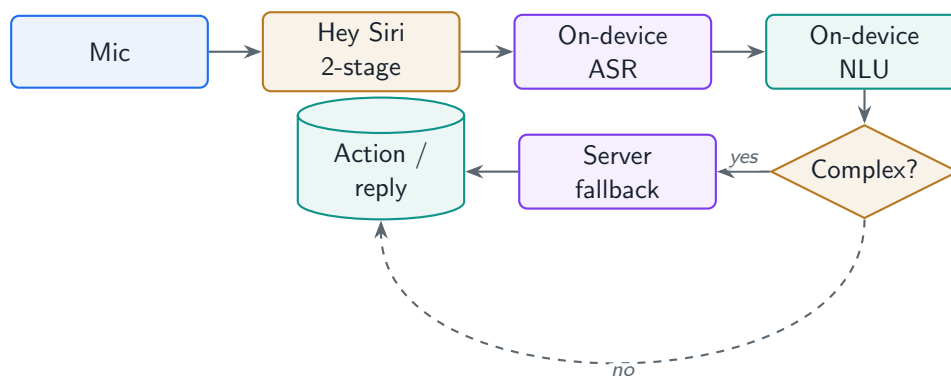
9.4 Case 3 — Apple Siri: Privacy-First On-Device Speech

Brief: a phone assistant and dictation engine that is fast, works offline, personalizes to the user, and keeps audio private.

Functional: wake, on-device ASR/NLU, dictation, server fallback. **Non-functional:** phone-class compute, offline capability, no raw audio off-device where avoidable.

Requirements & scale. Per-device inference (no central QPS), but billions of devices means model size, battery, and memory are the constraints. Personalization must happen without central data collection.

Architecture. Apple pushes the pipeline onto the device. A two-stage “Hey Siri” detector (tiny always-on first pass wakes a larger verifier) minimizes power and false accepts. Since iOS 15, neural ASR for many requests runs fully on-device, so dictation works offline with no audio leaving the phone; only complex or knowledge-heavy requests fall back to the server. Personalization (contacts, app names) is on-device; population learning uses differential privacy. The Neural Engine makes on-device Transformers cheap enough to run live.



Path	Latency	Privacy	When
On-device	lowest	highest	dictation, timers, device control, offline
Server	higher	lower	open knowledge, complex reasoning

Table 9.3: Siri placement decision: on-device default, server for the hard tail.

Evaluation, safety & cost. Metrics: on-device WER, wake FAR/FRR, latency, and offline coverage rate. Safety/privacy: data minimization, on-device processing, differential privacy for aggregate learning. Cost: device compute is “free” to the operator; server tier is the spend, so maximize on-device coverage.

Tradeoff & recommendation. On-device wins privacy, offline, and latency but caps model size; server recovers capability for the hard tail. Recommendation: route by request complexity, learn with differential privacy, and grow on-device coverage over time. When privacy is the theme, lead with on-device inference, not “encrypt the upload.”

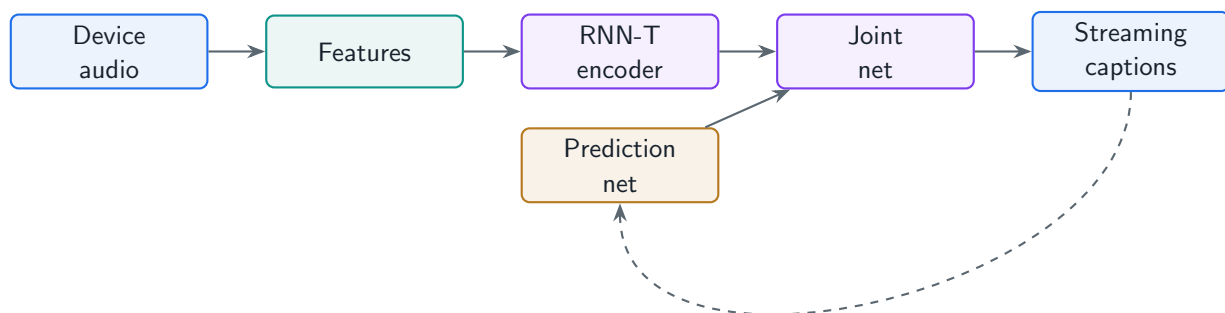
9.5 Case 4 — Google Live Caption: Streaming On-Device RNN-T

Brief: caption any audio playing on a phone, videos, calls, podcasts, in real time, offline, with low battery cost.

Functional: streaming ASR + punctuation, on-device. Non-functional: low first-token latency, continuous operation, tiny memory footprint, no network.

Requirements & scale. Per-device, always-available, battery-bounded. Targets: sub-300 ms partial latency, model small enough (~80 MB quantized) to run continuously without draining the phone.

Architecture. The canonical streaming-ASR case and the reason RNN-T exists. Google ships an end-to-end **RNN-Transducer**: a frame-synchronous encoder, an autoregressive prediction network (implicit LM), and a joint network, quantized into a compact package. RNN-T emits words as audio arrives with bounded latency and produces fluent text without a server LM. An endpointer plus a punctuation/capitalization model clean the display stream. Everything runs locally.



Property	Target	Lever
Partial latency	<300 ms	small encoder, frame-synchronous emit
Finalize latency	<200 ms	local-agreement commit policy
Footprint	~80 MB	int8 quantization, distillation
Power	continuous	on-device, no radio, DSP/NPU offload

Table 9.4: Live Caption streaming budget and on-device footprint.

Evaluation, safety & cost. Metrics: streaming WER, partial/finalize latency, caption flicker (revision rate), battery per hour. Safety/privacy: fully on-device, no audio leaves the phone. Cost: zero server cost; the budget is battery and memory.

Tradeoff & recommendation. RNN-T gives streaming plus an implicit LM at higher training cost and memory than CTC, and a lower ceiling than a giant offline AED. Recommendation: “streaming + on-device + offline” $\Rightarrow$ **RNN-Transducer**, quantized, with a local-agreement commit policy to suppress flicker. Never propose Whisper here.

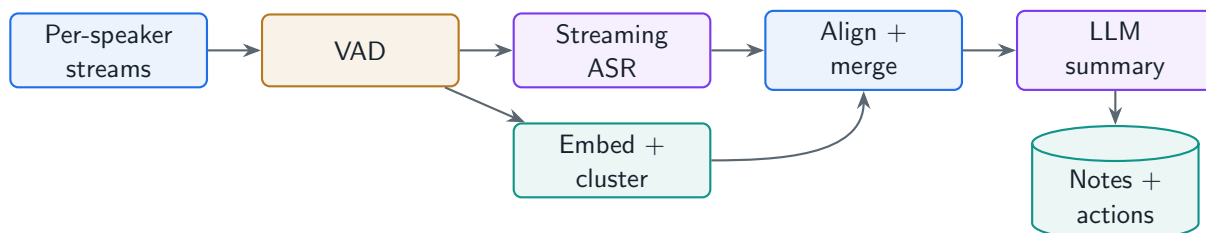
9.6 Case 5 — Zoom / Otter.ai: Meeting Transcription & Diarization

Brief: live captions during a meeting plus an accurate speaker-attributed transcript and an LLM summary with action items afterward.

Functional: streaming ASR, diarization, summarization. Non-functional: real-time captions and high post-meeting accuracy, large concurrency, multilingual.

Requirements & scale. Hundreds of thousands of concurrent meetings, each multi-speaker with overlap. Two latency tiers: live captions (seconds) and a refined post-meeting transcript (minutes).

Architecture. The decisive insight is whether you control the channels. In Zoom, each participant is a distinct stream, so diarization is nearly free, transcribe each stream and merge by timestamp. For mixed single-channel audio (a room mic, or Otter on a phone) you must diarize: VAD, ECAPA-TDNN embeddings, clustering for “who spoke when,” then word-to-speaker alignment. Streaming ASR drives live captions; a higher-accuracy batch pass refines the final transcript, which an LLM summarizes into notes and action items.



Tier	Latency	Approach
Live captions	1–3 s	streaming ASR per channel
Final transcript	~0.5–2x meeting	batch ASR + diarization refine
Summary	post-meeting	LLM over the refined transcript

Table 9.5: Two-tier latency: fast captions now, accurate transcript later.

Evaluation, safety & cost. Metrics: WER, DER (and speaker-attributed cpWER), summary faithfulness, caption latency. Safety/privacy: consent to record, retention, PII redaction. Cost: stream small models live, batch large models for the refine pass; pool GPUs across meetings.

Tradeoff & recommendation. Per-channel capture trades infrastructure for near-perfect diarization; single-channel saves capture complexity but forces hard clustering. Recommendation: if you own the client, separate streams at the source, it beats any diarization model, and use a two-tier latency design.

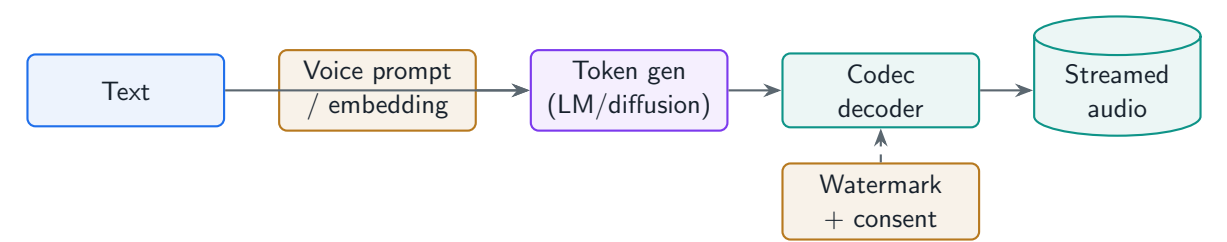
9.7 Case 6 — ElevenLabs: Zero-Shot Voice Cloning TTS

Brief: generate highly natural, emotional speech in any voice, including cloning a speaker from seconds of audio, for dubbing, audiobooks, and agents, via a low-latency API.

Functional: TTS, zero-shot cloning, streaming. **Non-functional:** top naturalness, low time-to-first-byte, many languages, misuse safeguards.

Requirements & scale. Two workloads with different SLAs: interactive (agents, <300 ms TTFB, streamed) and batch (audiobooks, throughput-optimized). Assume tens of thousands of concurrent streams at peak.

Architecture. High-end TTS generates over neural-codec tokens rather than the classic mel+vocoder cascade. Text plus a reference voice embedding/prompt conditions a language-model-style or diffusion generator that emits codec tokens, which a codec decoder renders to waveform. Zero-shot cloning works because the model learned to disentangle content from timbre across many speakers. For agents the API streams audio chunk-by-chunk to minimize TTFB. Safeguards (consent verification, watermarking) ship with the capability.



Mode	TTFB target	Lever
Interactive (agent)	<300 ms	stream tokens, small/distilled model, GPU warm pool
Batch (audiobook)	N/A	maximize throughput, larger model for quality
Cloning	one-time	encode reference → speaker embedding/prompt

Table 9.6: Two TTS workloads: latency-bound interactive vs throughput-bound batch.

Evaluation, safety & cost. Metrics: MOS/CMOS, speaker-similarity (cosine), ASR-WER on output for intelligibility, TTFB. Safety: consent/verification for cloning, AudioSeal-style watermarking, abuse detection, this is dual-use technology. Cost: distill for the interactive tier; cache stock phrases; batch the rest.

Tradeoff & recommendation. Codec-token generation gives state-of-the-art naturalness and instant cloning but is slower and harder to control than FastSpeech2+HiFi-GAN, and it is dual-use. Recommendation: serve a distilled streaming model for agents and a larger model for batch; pair cloning with watermarking and consent. Raise safety unprompted, interviewers expect it.

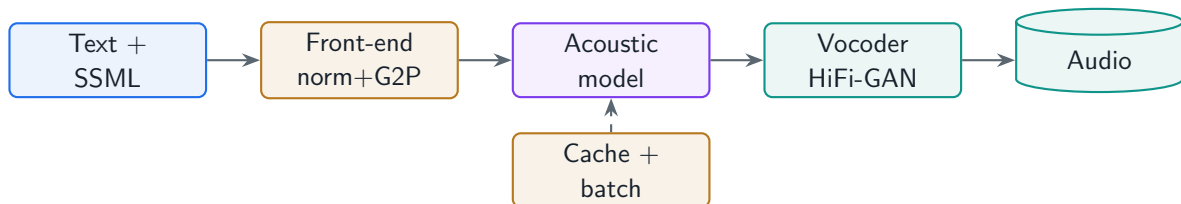
9.8 Case 7 — Amazon Polly / Google Cloud TTS: Cloud TTS at Scale

Brief: a multi-tenant cloud API turning text into natural speech in dozens of languages and voices, with SSML control, at massive throughput and predictable latency.

Functional: TTS, SSML, voice selection. Non-functional: high concurrency, low/stable latency, broad coverage, cost efficiency.

Requirements & scale. Multi-tenant, bursty traffic across many voices. Targets: stable p99 latency under quota, high GPU utilization via batching, and graceful autoscaling per-voice.

Architecture. The battle-tested neural cascade behind the API: an SSML-aware front-end (normalization, G2P, prosody/break prediction), a non-autoregressive acoustic model (FastSpeech2-style) producing mel-spectrograms, and a fast GAN vocoder (HiFi-GAN-class) rendering audio; a speaker embedding selects the voice. The platform differentiators are operational: phrase caching, GPU batching, autoscaling per-voice model pools, streaming for long text, and per-tenant quotas.



Lever	Effect	Note
Phrase cache	big cost cut	greetings, IVR prompts repeat heavily
Dynamic batching	higher GPU util	bounded by p99 latency target
Per-voice pools	autoscale	popular voices get more replicas
Streaming	lower TTFB	for long documents

Table 9.7: Operational levers dominate cloud TTS economics.

Evaluation, safety & cost. Metrics: MOS, p99 latency, cache hit rate, cost per million characters. Safety: usage policy, watermarking for neural voices. Cost: caching and batching are the primary levers; right-size voice pools.

Tradeoff & recommendation. The cascade trades a sliver of naturalness for speed, control, and reliability, exactly what a paid API needs. Recommendation: cascade (FastSpeech2 + HiFi-GAN) with aggressive caching, dynamic batching, and per-voice autoscaling; SSML is the controllability story. At scale, ops beats model choice.

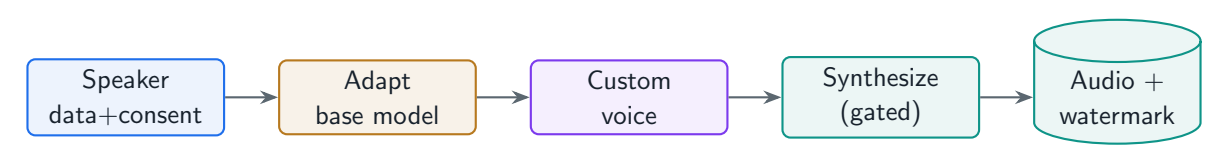
9.9 Case 8 — Azure Custom Neural Voice: Brand Voice Cloning

Brief: let an enterprise build a custom, on-brand synthetic voice from a consenting professional speaker, then synthesize unlimited content in that voice, responsibly.

Functional: speaker adaptation + synthesis. Non-functional: high fidelity, data efficiency, strict consent/abuse gating, enterprise SLAs.

Requirements & scale. Low-volume training (one speaker, studio data) but high-volume inference. The governance pipeline (consent, access, watermark) is a first-class requirement, not an afterthought.

Architecture. Custom voice is fine-tuning plus governance. A base multi-speaker TTS model (acoustic + vocoder, in the neural-codec/VALL-E lineage for the newest tiers) is adapted to the target speaker with recorded data; modern pipelines need far less data than training from scratch because the base model already captured general speech. The product core is a consent gate (verified speaker statements), access controls, and watermarking. Synthesis then uses the standard front-end→acoustic→vocoder path with the adapted voice.



Concern	Target	Mechanism
Data efficiency	minutes–hours	adapt base model, not train from scratch
Fidelity	high speaker-sim	speaker adaptation + good base
Consent	mandatory	recorded verbal statement, identity check
Misuse	blocked	access control + watermark + audit

Table 9.8: Custom voice is half model (adaptation), half governance.

Evaluation, safety & cost. Metrics: speaker similarity, MOS, consent compliance. Safety: this is the highest-risk cloning case, gate hard. Cost: training is cheap (adaptation); inference follows standard TTS economics.

Tradeoff & recommendation. Adaptation trades a controlled data-collection step for huge data efficiency and fidelity. Recommendation: adapt a strong base model, and make consent + access + watermarking blocking requirements. Name both halves, model and governance, or the answer is incomplete.

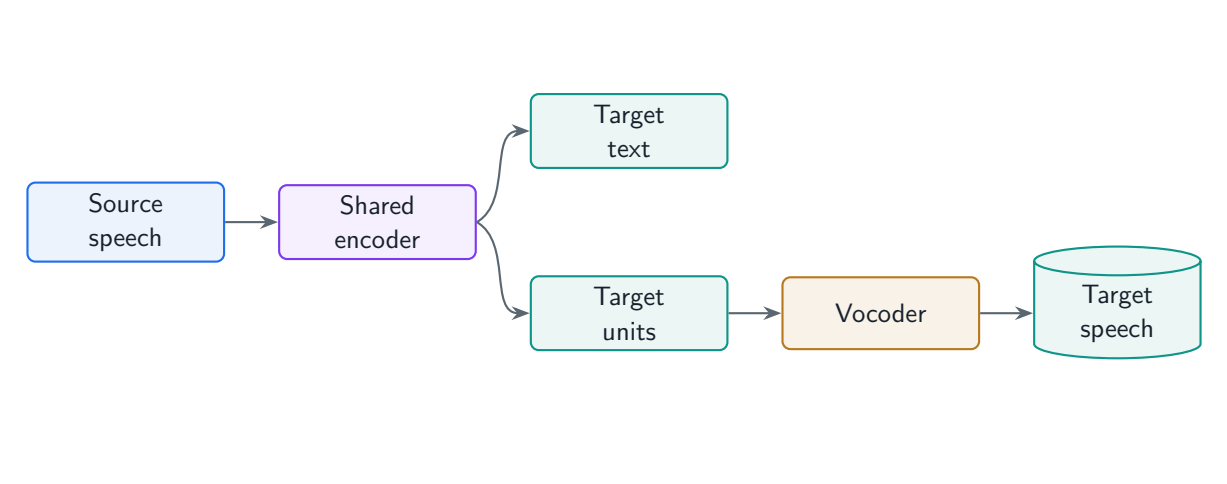
9.10 Case 9 — Meta SeamlessM4T: Speech-to-Speech Translation

Brief: translate speech in one language into speech (or text) in another across many pairs, preserving expressiveness, with a near-real-time option.

Functional: S2ST/S2TT/ASR/MT in one model. Non-functional: low cascaded error, prosody preservation, streaming for live interpreting.

Requirements & scale. Two regimes: offline dubbing (quality-first) and live interpreting (latency-first, simultaneous). Many language directions; provenance watermarking on generated audio.

Architecture. The cascaded baseline chains ASR→MT→TTS: modular and debuggable, but errors compound, latency stacks, and prosody is lost at the ASR boundary. SeamlessM4T moves to a unified multitask model mapping source audio to target text and target speech via shared representations, cutting error stages and latency. Expressive variants preserve prosody/voice; simultaneous variants translate before the sentence ends. Generated audio is watermarked (AudioSeal).



Approach	Latency	Debuggable	Use
Cascaded ASR-MT-TTS	higher	yes	offline, auditable (legal/medical)
Unified S2ST	lower	less	general dubbing, prosody-preserving
Simultaneous	lowest	least	live interpreting (partial output)

Table 9.9: Translation design space: control vs latency.

Evaluation, safety & cost. Metrics: BLEU/chrF (text), ASR-BLEU (speech), MOS, latency (for simultaneous, also lagging). Safety: watermark output; flag mistranslations in high-stakes settings. Cost: unified model is one inference vs three.

Tradeoff & recommendation. Unified S2ST cuts cascaded error and latency and preserves prosody, but is harder to debug than a transparent chain. Recommendation: unified/simultaneous for live interpreting; cascaded when you must inspect and correct the intermediate text.

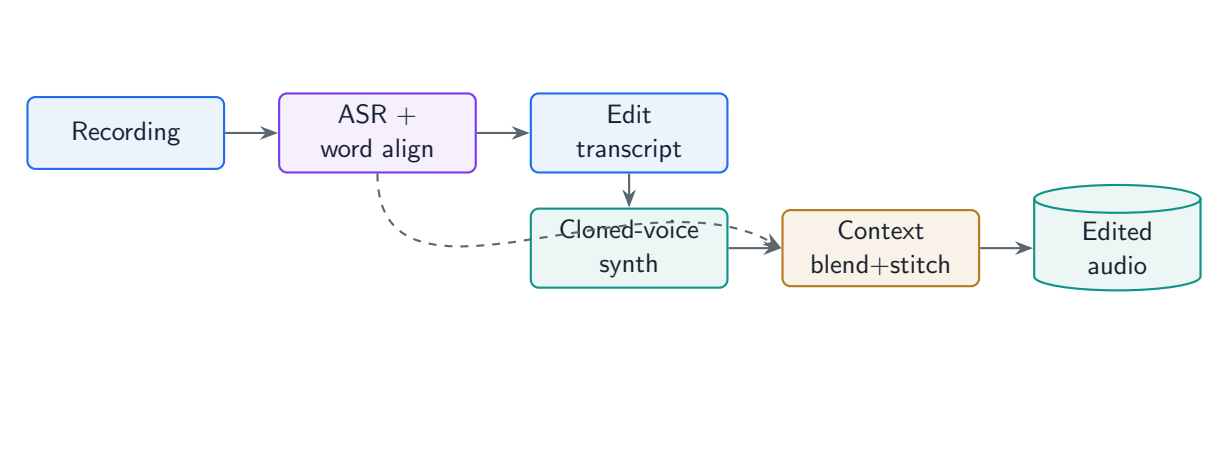
9.11 Case 10 — Descript Overdub: Voice Editing for Creators

Brief: edit recorded speech by editing its transcript, fixing a flubbed word generates new audio in the creator’s cloned voice, seamlessly stitched into the original.

Functional: ASR + alignment + cloned-voice synth + blending. Non-functional: voice match, prosody continuity at edits, consent, transcript-driven UX.

Requirements & scale. Offline/interactive editing (not real-time conversation); the bar is seamlessness, not latency. Per-creator cloned voice with consent.

Architecture. Overdub fuses three subsystems: ASR produces a word-aligned transcript that becomes the editing surface; a personal TTS voice is cloned from the creator’s consented recordings; and inserted words must blend into real audio, generation is conditioned on surrounding context so pitch, pace, and timbre match at boundaries, with crossfades hiding seams. Deleting a word deletes its audio; typing one synthesizes it in-voice.



Subsystem	Hard part
ASR + alignment	accurate word timestamps for the edit surface
Cloned voice	match the creator’s timbre from consented data
Blending	prosody continuity + crossfade at insert boundaries

Table 9.10: The engineering is in alignment and boundary blending, not the TTS.

Evaluation, safety & cost. Metrics: seam detectability (human ABX), speaker similarity, alignment accuracy. Safety: consent for the voice, watermark edits. Cost: offline, so throughput-bound, not latency-bound.

Tradeoff & recommendation. The key is boundary blending and the transcript-aligned UX, not the TTS. Recommendation: invest in word-level alignment and context-conditioned synthesis with crossfades; treat “AI audio editing” as an ASR + alignment + generation *system*.

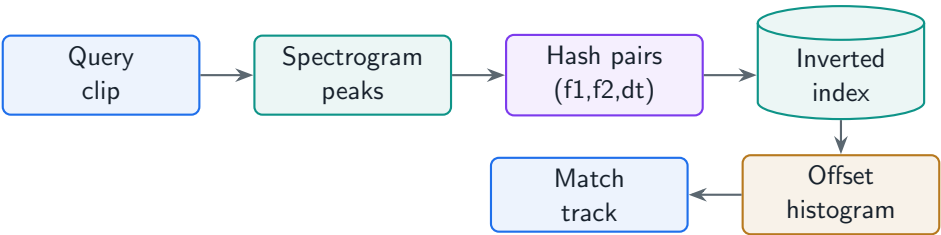
9.12 Case 11 — Shazam: Audio Fingerprinting at Planet Scale

Brief: identify a song from a few noisy seconds captured on a phone in a bar, matching against tens of millions of tracks in under a second.

Functional: known-recording identification. Non-functional: robust to noise/distortion, sub-second lookup over a huge catalog, tiny query payload.

Requirements & scale. Catalog of $\sim 10^8$ tracks; millions of queries/day; each query a 5–10 s noisy clip. Targets: sub-second match, high precision (wrong song is embarrassing), minimal uplink.

Architecture. Shazam predates deep learning and gets the representation right. It computes a spectrogram, extracts robust spectral **peaks** (local maxima surviving noise/compression), and pairs nearby peaks into **hashes** of (freq1, freq2, time-delta), a “constellation map” invariant to volume and most distortion. Each catalog track is fingerprinted offline into an inverted index hash→(track, offset). A query is hashed the same way, its hashes look up candidate tracks, and the match is the track whose offsets align consistently (a sharp time-offset histogram peak). Only compact hashes upload.



Property	Target	Mechanism
Lookup latency	<1 s	inverted index + hash join
Robustness	high	peak constellation survives noise
Index scale	10 ⁸ tracks	sharded hash → postings store
Payload	tiny	send hashes, not audio

Table 9.11: Fingerprint identification: exact-match retrieval, not learned similarity.

Evaluation, safety & cost. Metrics: top-1 identification accuracy at SNR, median latency, false-match rate. Safety: minimal (no biometrics). Cost: index storage and lookup QPS; shard by hash.

Tradeoff & recommendation. Fingerprinting is exact-match retrieval, very fast and robust for *identifying a known recording*, but useless for “find similar songs.” Recommendation: fingerprinting for identity, learned embeddings for similarity. Naming that split signals real audio-IR depth.

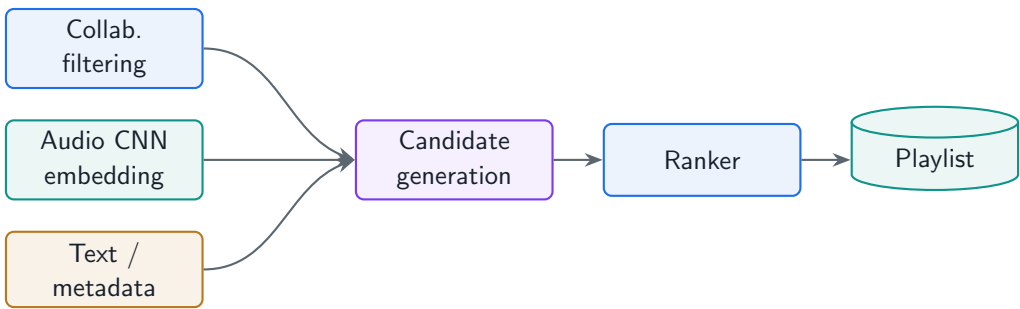
9.13 Case 12 — Spotify: Music Recommendation & Audio Understanding

Brief: recommend the next track and build personalized playlists for hundreds of millions of users over tens of millions of tracks, including brand-new songs with no history.

Functional: candidate generation + ranking. Non-functional: cold-start coverage, scale, diversity, freshness.

Requirements & scale. $\sim 10^8$ users, $\sim 10^8$ tracks, billions of interactions. Targets: low-latency serving via ANN, cold-start for new tracks, and a ranking objective balancing engagement, diversity, and freshness.

Architecture. Spotify blends three signals. **Collaborative filtering** learns user/track embeddings from behavior, powerful but blind to new tracks (cold-start). A **content model** (CNN over log-Mel) predicts a track’s latent embedding from its *audio*, placing an unstreamed song near similar music. NLP over playlists/metadata adds cultural context. Candidates from all sources are generated, then ranked, and served via fast approximate nearest-neighbor search.



Stage	Design
Candidate gen	CF + audio-content + metadata recall; ANN over embeddings
Ranking	gradient-boosted/DNN ranker on engagement + diversity
Cold-start	audio CNN predicts the CF embedding for unstreamed tracks
Serving	precompute embeddings; ANN index; cache per-user lists

Table 9.12: Two-stage recsys; the audio model exists to solve cold-start.

Evaluation, safety & cost. Offline: recall@k, NDCG. Online: completion rate, skip rate, retention (A/B). Safety: avoid filter bubbles; creator fairness. Cost: precompute embeddings offline; ANN keeps serving cheap.

Tradeoff & recommendation. CF is accurate but cold-starts on new tracks; the audio content model trades some accuracy to cover that tail. Recommendation: hybrid recall (CF + audio + text) into a ranker, with the audio model predicting CF embeddings for new tracks, the classic Spotify answer.

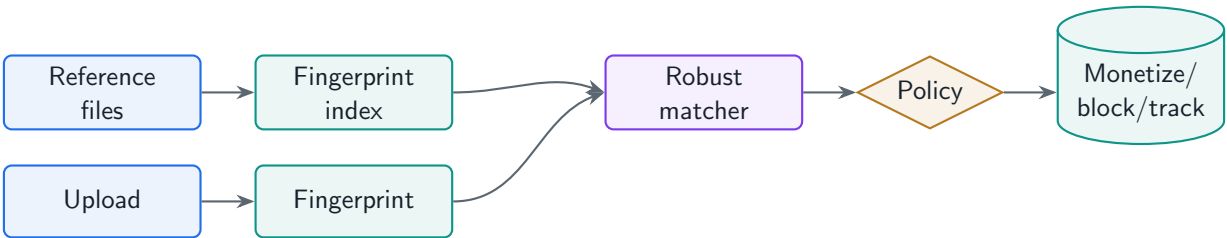
9.14 Case 13 — YouTube Content ID: Copyright Matching at Scale

Brief: scan every upload against rights-holder reference audio/video and flag matches, even partial, pitched, or noisy reuse, at hundreds of hours uploaded per minute.

Functional: robust copy detection + policy. Non-functional: massive ingest, partial-copy robustness, very low false positives, continuous re-scan.

Requirements & scale. >500 hours uploaded/minute; millions of references. Targets: high recall with very high precision (money/takedowns), time-aligned claims, and a dispute path.

Architecture. Large-scale fingerprint matching. Rights holders supply references, each fingerprinted into a compact robust representation in a searchable index. Every upload is fingerprinted and matched, with matching designed to survive pitch/tempo shifts, re-encoding, and partial overlaps (a 20 s clip in a vlog). Matches yield time-aligned claims that trigger the holder’s policy (monetize/block/track). The system re-scans as new references arrive; disputes handle false positives.



Requirement	Design response
Ingest throughput	distributed fingerprinting; stream processing
Partial/modified	robust features; pitch/tempo-invariant matching
Precision	high match threshold + human dispute workflow
Continuous	re-scan corpus as new references land

Table 9.13: At this scale, design the index and false-positive path first.

Evaluation, safety & cost. Metrics: recall/precision of matches, dispute overturn rate, claim latency. Safety: false claims have real financial/legal impact, precision and appeals are the safety story. Cost: index size and continuous re-scan compute.

Tradeoff & recommendation. The matcher trades recall against precision, and the business demands very high precision. Recommendation: robust fingerprint index + conservative threshold + a strong dispute workflow; the matching model is necessary but not the risk, the index and appeals path are.

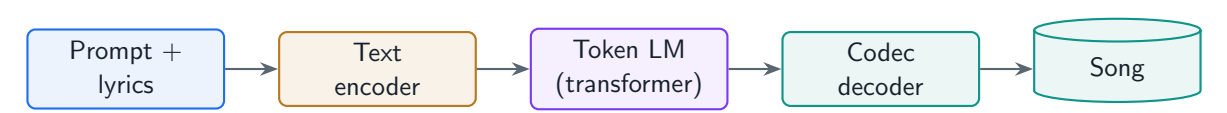
9.15 Case 14 — Suno: Text-to-Music Generation

Brief: generate a complete, radio-quality song, vocals, lyrics, instrumentation, from a short text prompt in about a minute.

Functional: text/lyrics→full song. Non-functional: long-range structure, singing-voice quality, generation speed, copyright/safety.

Requirements & scale. Generation latency ~60 s/song; bursty creative traffic. Targets: coherent multi-minute structure, lyric alignment, and acceptable GPU cost per song.

Architecture. Text-to-music follows the codec-LM template. A neural audio codec (EnCodec/DAC-style) compresses music to discrete tokens; a text encoder (T5-class) embeds prompt + lyrics; a Transformer LM generates the audio-token sequence conditioned on text, modeling melody, harmony, rhythm, and vocal timbre over long contexts; the codec decoder renders tokens to waveform. Coherent verse/chorus structure uses hierarchical or coarse-to-fine token modeling; lyrics condition the vocal track.



Challenge	Approach
Long-range structure	hierarchical/coarse-to-fine token generation
Vocal quality	codec decoder/vocoder tuned for singing
Lyric alignment	condition vocal track on lyrics
Speed / cost	parallel decode where possible; GPU batching

Table 9.14: Codec-tokens + transformer + decoder, the shared generative backbone.

Evaluation, safety & cost. Metrics: human preference/MOS, prompt adherence, musicality. Safety: training-data copyright, artist-voice/style protections, watermarking. Cost: GPU-seconds per song; batch and distill.

Tradeoff & recommendation. Codec-token LMs give rich full-band music but cost compute and raise copyright questions; coherence trades against speed. Recommendation: recognize the **codec-tokens + transformer + decoder** pattern shared with LM-TTS and S2S; invest in hierarchical structure and watermarking.

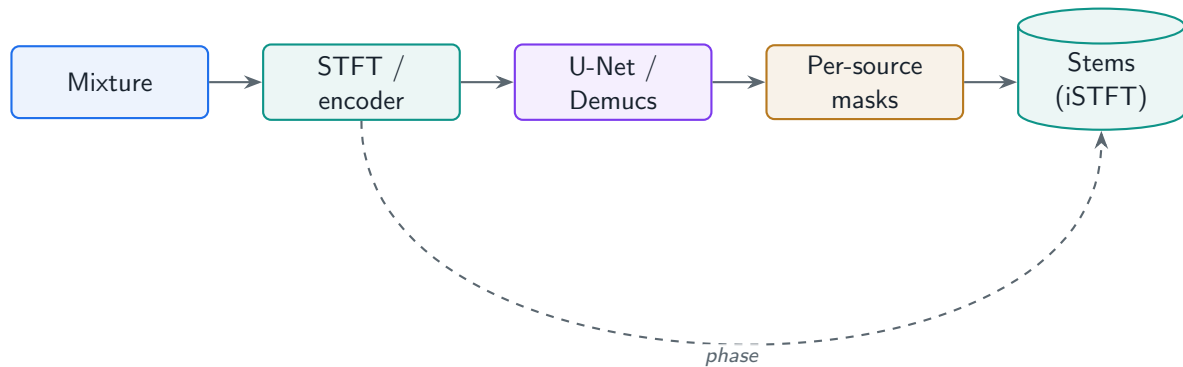
9.16 Case 15 — Moises / Spleeter: Music Source Separation

Brief: split a finished song into stems, vocals, drums, bass, other — for remixing, transcription, or karaoke.

Functional: source separation. Non-functional: clean separation, near-real-time on consumer hardware, genre generalization.

Requirements & scale. Mostly on-demand per-track jobs (app or batch). Targets: low artifacts, RTF <1 on a consumer GPU/CPU, robust across genres.

Architecture. Source separation is learned time–frequency masking. The mixture’s magnitude spectrogram (or raw waveform) feeds an encoder–decoder, a U-Net (Spleeter) or a hybrid/transformer model (Demucs), predicting a soft mask per source. Each mask multiplies the mixture spectrogram to isolate a stem; inverse STFT (reusing mixture phase or a learned phase) reconstructs each waveform. Trained on multi-track data (MUSDB18) with L1/SI-SDR loss. Waveform-domain models avoid phase issues and sound cleaner.



Model family	Domain	Trade
Spleeter (U-Net)	spectrogram	fast, simple; inherits phase artifacts
Demucs (hybrid)	waveform	cleaner; higher compute

Table 9.15: Separation = mask estimation; the phase problem is the catch.

Evaluation, safety & cost. Metrics: SI-SDR/SDR per stem, artifact MOS. Safety: minimal (re-use rights are a product/policy matter). Cost: per-track GPU time; waveform models cost more.

Tradeoff & recommendation. Spectrogram masking is fast but inherits phase artifacts; waveform-domain models sound better at higher compute. Recommendation: frame as mask estimation, choose Demucs-style for quality or Spleeter-style for speed, and call out the phase problem.

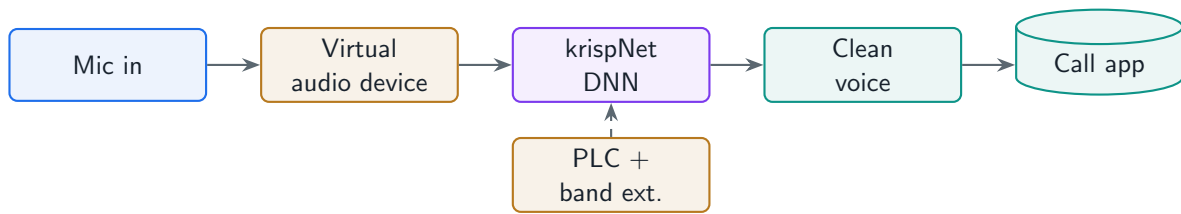
9.17 Case 16 — Krisp: Real-Time Noise Cancellation

Brief: remove background noise from outgoing mic and incoming speaker audio of any call, in real time, on the user’s device, without touching the conferencing app.

Functional: speech enhancement (bidirectional). Non-functional: few-ms algorithmic latency, CPU-only, preserve voice, app-agnostic.

Requirements & scale. Per-device, always-on during calls. Hard targets: <10 ms added latency, real-time factor well below 1 on a laptop CPU, no quality loss on clean speech.

Architecture. Krisp inserts a virtual audio device between the OS and the conferencing app, cleaning audio for any platform. The core is **krispNet**, a DNN trained on thousands of hours of clean speech mixed with diverse noise to suppress everything non-voice. It runs on tiny look-ahead windows and locally (no cloud round-trip, protecting privacy). Related DNNs add packet-loss concealment and bandwidth extension.



Constraint	Target	Lever
Added latency	<10 ms	small look-ahead, lightweight DNN
Compute	CPU real-time	quantization, optimized kernels
Quality	no clean-speech damage	train on clean+noisy mixes
Privacy	on-device	no cloud round-trip

Table 9.16: Enhancement is a latency-bounded, on-device DNN problem.

Evaluation, safety & cost. Metrics: DNSMOS/PESQ/STOI, *and* downstream ASR-WER (denoising can hurt ASR). Safety/privacy: on-device, no audio leaves. Cost: runs on the user's CPU, zero server cost.

Tradeoff & recommendation. Aggressive suppression can clip quiet speech or raise ASR WER; the latency budget caps model size. Recommendation: a small on-device DNN tuned on the *real* objective (often ASR-WER, not just perceived cleanliness), with PLC for robustness.

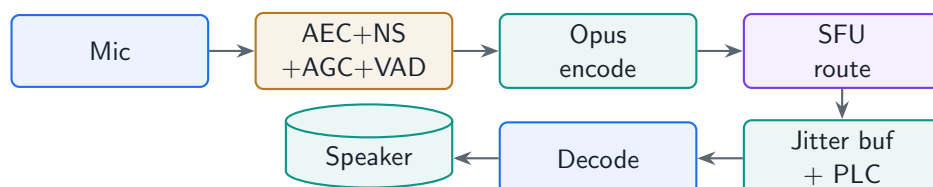
9.18 Case 17 — Zoom / Discord: The Real-Time Audio Transport

Brief: carry low-latency, intelligible voice between many participants over unreliable networks, with echo cancellation and graceful degradation under loss.

Functional: capture, encode, route, decode, play. Non-functional: <150–200 ms mouth-to-ear, loss/jitter tolerance, many participants.

Requirements & scale. Large calls (dozens–thousands via SFU), heterogeneous networks and devices. Targets: mouth-to-ear under ~200 ms, intelligibility under 5–10% packet loss.

Architecture. The WebRTC-style pipeline with ML inside a DSP backbone. Capture: AEC, noise suppression (increasingly neural), AGC, VAD (send only speech). Encode with **Opus** (built-in FEC + PLC) into small frames. A **Selective Forwarding Unit** routes streams without re-encoding, scaling to large calls. Receive: adaptive **jitter buffer**, PLC for losses, decode to playback. Bitrate adapts to bandwidth.



Stage	Budget (ms)	Note
Capture DSP	5–15	AEC/NS/AGC
Encode + net	30–80	Opus + transit
Jitter buffer	20–60	adaptive; the main tunable
Decode + play	5–15	PLC fills losses

Table 9.17: Mouth-to-ear budget; jitter buffer trades latency for smoothness.

Evaluation, safety & cost. Metrics: mouth-to-ear latency, MOS, loss concealment quality, jitter. Safety: encryption (SRTP/E2EE). Cost: SFU bandwidth and CPU; neural NS adds compute.

Tradeoff & recommendation. A bigger jitter buffer smooths playback but adds latency; FEC adds robustness but costs bandwidth. Recommendation: AEC + adaptive jitter buffer + Opus(FEC/PLC) over an SFU; AEC is also what makes voice-agent barge-in possible.

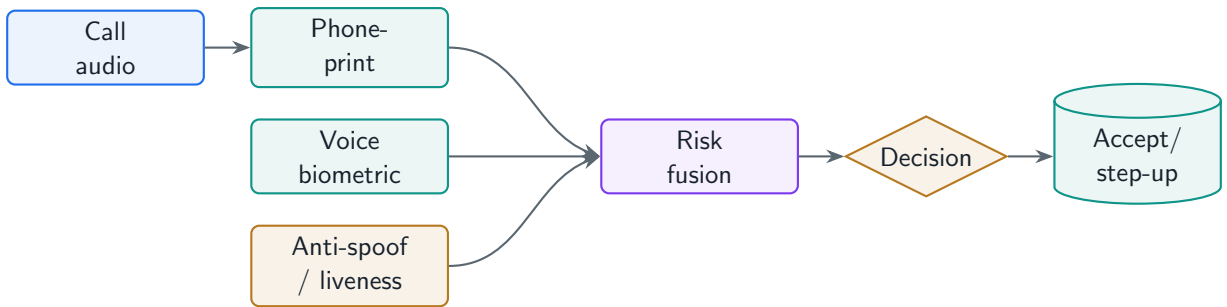
9.19 Case 18 — Pindrop: Voice Anti-Spoofing & Fraud Detection

Brief: in a bank call center, authenticate genuine callers and flag fraudsters, including deepfakes and replay, in real time, with low friction for real customers.

Functional: verification + anti-spoofing + risk scoring. Non-functional: low FRR for legitimate users, robust deepfake detection, IVR/CCaaS integration, auditable decision.

Requirements & scale. Enterprise call volumes; real-time decision within the IVR flow. Targets: low EER, very low false-reject for genuine callers, and resilience to the reported 1300% surge in deepfake fraud.

Architecture. Pindrop fuses orthogonal signals rather than trusting voice alone. **Phoneprinting** analyzes call-audio artifacts (codec, channel, device). **Voice biometrics** match the speaker to an enrolled embedding. A dedicated **liveness/anti-spoofing** model (ASVspoof-style) detects synthesis/replay, the Pindrop Pulse line targets deepfakes. Signals fuse into a risk score driving accept/step-up/review, integrated natively into NICE CXone and used in Zoom’s contact-center flows.



Signal	What it catches
Phoneprint	spoofed origin, device/channel anomalies
Voice biometric	impostor speaker (vs enrolled)
Anti-spoofing	synthetic/replayed audio (deepfakes)
Behavioral	risky patterns, velocity, metadata

Table 9.18: Fuse orthogonal signals, never rely on voice biometrics alone.

Evaluation, safety & cost. Metrics: EER and t-DCF under realistic attacks, FRR for genuine callers, fraud caught. Safety: the system *is* the safety control; biometric data needs consent + protection. Cost: real-time scoring per call.

Tradeoff & recommendation. The threshold trades fraud caught against genuine customers challenged, and anti-spoofing degrades against novel attacks. Recommendation: multi-signal fusion (liveness + channel + biometric + behavioral), report EER/t-DCF under attack, and plan continual model updates.

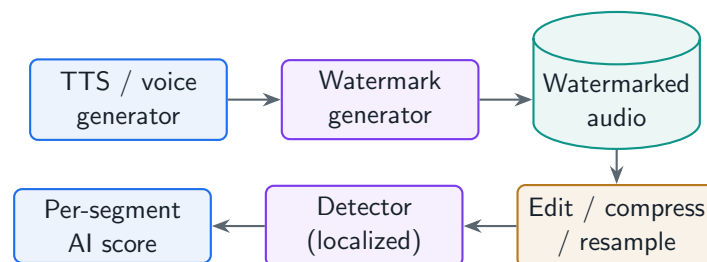
9.20 Case 19 — Meta AudioSeal: Watermarking & Deepfake Provenance

Brief: make AI-generated speech detectable as synthetic by embedding an inaudible, robust watermark at generation time, and detect it (even in an edited clip) at scale.

Functional: watermark embed + localized detect. Non-functional: imperceptible, robust to edits, sample-level localization, real-time at fleet scale.

Requirements & scale. Embed on every generated clip across products serving $\sim 10^5$ users/day; detect fast and locally in time. Targets: inaudibility, robustness to compression/resampling/editing, single-pass detection.

Architecture. AudioSeal uses a jointly trained **generator** and **detector**. The generator produces an additive, perceptually masked watermark conditioned on the audio, applied as a TTS/voice system generates speech. The detector is a single-pass model outputting per-sample/frame watermark probability, enabling **localized** detection (which parts are AI) and running orders of magnitude faster than prior methods, used in Audiobox and Seamless.



Property	Mechanism
Imperceptible	perceptual loss / auditory masking
Robust	trained against compression, noise, resampling
Localized	per-sample detector (which parts are AI)
Fast	single-pass detection at scale

Table 9.19: Generator+detector trained jointly for robust localized watermarking.

Evaluation, safety & cost. Metrics: detection TPR/FPR after edits, imperceptibility (MOS), localization accuracy. Safety: covers only cooperating generators and can be attacked, pair with passive detection and policy. Cost: cheap embed; cheap single-pass detect.

Tradeoff & recommendation. Watermark strength trades robustness against imperceptibility, and any watermark only covers cooperating generators. Recommendation: ship watermarking by default with localized detection, but combine it with passive deepfake detection and provenance policy, necessary, not sufficient.

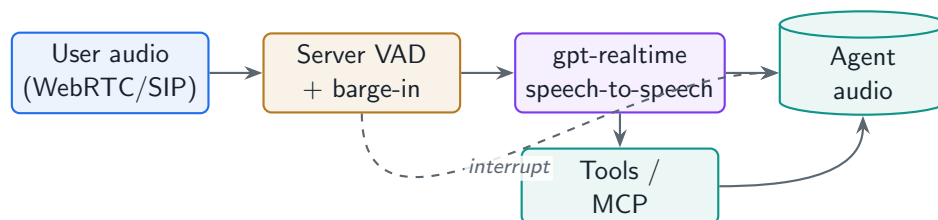
9.21 Case 20 — OpenAI Realtime API: Full-Duplex Voice Agents

Brief: power a production voice agent that listens and speaks over a live audio stream, calls tools, handles interruptions, and answers phone calls, with human-like latency.

Functional: full-duplex S2S + tools + telephony. Non-functional: sub-second turn, natural barge-in, reliable tool calling, production reliability.

Requirements & scale. Concurrent live sessions (web + phone). Targets: turn latency comfortably under 1s, instant barge-in, and dependable function calling for real actions.

Architecture. The 2025 `gpt-realtime` model (GA Aug 2025) collapses the ASR→LLM→TTS cascade into a single speech-to-speech model: audio in, audio out, with text and tool-calls alongside. This removes inter-stage latency and preserves prosody. The Realtime API streams audio bidirectionally (WebRTC/WebSocket), supports server-side VAD and barge-in, invokes tools/functions and remote MCP servers mid-turn, and connects to phones via SIP. Production wraps it with an orchestration layer (LiveKit/Pipecat) for telephony, observability, and guardrails, confirming irreversible actions.



Stage	Budget (ms)	Note
VAD + endpoint	150–400	server-side; semantic endpointing helps
S2S model	200–500	single model, no cascade hops
Tool call	varies	parallelize; confirm irreversible
Audio out	streamed	first audio ASAP; barge-in cancelable

Table 9.20: Single S2S model removes cascade latency; orchestration adds safety.

Evaluation, safety & cost. Metrics: turn latency p95, barge-in responsiveness, tool-call accuracy, task success. Safety: confirm irreversible actions, guardrails, logging (harder without a transcript, keep a text trace). Cost: streaming S2S GPU-time per session-minute.

Tradeoff & recommendation. End-to-end S2S wins latency and naturalness but is harder to audit, constrain, and log than a cascade. Recommendation: use S2S for the conversational core,

keep a text trace + tool guardrails + confirmation for actions, and wrap it in an orchestration layer, the production face of Chapter 8's agent.

References

- Défossez, A. Copet, J. Synnaeve, G. & Adi, Y. (2023). High fidelity neural audio compression. *Transactions on Machine Learning Research*.
- He, Y. Sainath, T. N. Prabhavalkar, R. McGraw, I. Alvarez, R. Zhao, D. ... & Gruenstein, A. (2019). Streaming end-to-end speech recognition for mobile devices. *IEEE ICASSP 2019*, 6381–6385.
- OpenAI. (2025). *Introducing gpt-realtime and Realtime API updates for production voice agents*. <https://openai.com/index/gpt-realtime/>
- Roman, R. S. Fernandez, P. Défossez, A. Furon, T. Tran, T. & Elshahar, H. (2024). Proactive detection of voice cloning with localized watermarking (AudioSeal). *Proceedings of the 41st International Conference on Machine Learning*.
- Seamless Communication, Barrault, L. Chung, Y.-A. Meglioli, M. C. Dale, D. Dong, N. ... & others. (2023). SeamlessM4T: Massively multilingual and multimodal machine translation. *arXiv preprint arXiv:2308.11596*.
- van den Oord, A. Dieleman, S. & Schrauwen, B. (2013). Deep content-based music recommendation. *Advances in Neural Information Processing Systems*, 26.
- Wang, A. (2003). An industrial-strength audio search algorithm (Shazam). *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*.

Hands-On Labs

Subscribe → aiengineeringinsider.substack.com/subscribe

Reading about WER and endpointing is not the same as measuring them. These three labs run on the companion `voice_agent` project with nothing but NumPy, and every number printed below was produced by actually running the code — not estimated. Reproduce them, then break them on purpose.

10.1 Before You Start

Everything here runs without a GPU, without model downloads, and without an internet connection. From the project root:

setup

```
python -m venv .venv && source .venv/bin/activate
pip install numpy
```

Each lab prints measured output. The numbers in the `Measured output` boxes are the genuine results from one run on a laptop CPU; yours will match closely (a couple of labs use a fixed random seed, so they match exactly).

Most interview prep stops at definitions. The candidates who stand out are the ones who can say “I measured this” — they know that CER is usually lower than WER, roughly how many milliseconds an endpointer adds, and why a magnitude spectrogram cannot be inverted cleanly. You only earn that intuition by running code. So run it.

10.2 Lab 1 — Features, the Frequency Domain, and Why Phase Matters

Goal. Build the feature pipeline from Chapter 1 on a real signal, measure the Mel compression ratio, verify SNR control, and then prove — with a number — why generative audio models cannot just predict a magnitude spectrogram.

Part A: features, compression, and SNR

lab1_features.py

```

import numpy as np
from voice_agent.features import FeatureExtractor
from voice_agent.audio_io import synth_utterance

sr = 16_000
wav = synth_utterance(seconds=1.6, sr=sr, seed=1)      # voiced + silence pads
fx = FeatureExtractor()                                # 80-bin log-Mel, 25/10 ms

log_mel = fx.log_mel(wav, sr)
n_freq = fx.n_fft // 2 + 1
print(f"clip: {len(wav)} samples = {len(wav)/sr:.2f}s @ {sr} Hz")
print(f"log-mel shape (n_mels, T): {log_mel.shape}")
print(f"mel compression: {n_freq} STFT bins -> {fx.n_mels} mel "
      f"f"= {n_freq/fx.n_mels:.2f}x fewer")

# Inject noise at a target SNR and verify we hit it.
rng = np.random.default_rng(0)
noise = rng.standard_normal(len(wav)).astype(np.float32)
scale = np.sqrt(np.mean(wav**2) / (np.mean(noise**2) * 10**(10/10)))
print(f"target SNR=10.0 dB, achieved SNR={fx.snr_db(wav, scale*noise):.2f} dB")

```

Measured output

```

clip: 25600 samples = 1.60s @ 16000 Hz
log-mel shape (n_mels, T): (80, 158)
mel compression: 201 STFT bins -> 80 mel = 2.51x fewer
target SNR=10.0 dB, achieved SNR=10.00 dB

```

The Mel filterbank turns 201 linear FFT bins into 80 perceptual channels — a 2.51x reduction before the model sees anything — and the SNR helper lands exactly on the 10 dB target, which is the gate you would use to reject audio too noisy to transcribe.

Part B: phase matters (a number, not a claim)

lab1_phase.py

```

import numpy as np
from voice_agent.audio_io import synth_utterance
n_fft, hop = 400, 160
wav = synth_utterance(seconds=1.0, sr=16_000, seed=2)
win = np.hanning(n_fft).astype(np.float32)

def stft(x):
    n = 1 + (len(x) - n_fft) // hop
    return np.stack([np.fft.rfft(x[i*hop:i*hop+n_fft]*win) for i in range(n)], 1)

def istft(S):
    n = S.shape[1]; out = np.zeros((n-1)*hop+n_fft); w = np.zeros_like(out)
    for i in range(n):
        out[i*hop:i*hop+n_fft] += np.fft.irfft(S[:, i], n=n_fft)*win

```

```

        w[i*hop:i*hop+n_fft] += win**2
    return out / np.maximum(w, 1e-8)

def si_sdr(ref, est):
    L = min(len(ref), len(est)); ref, est = ref[:L]-ref[:L].mean(),
    ↪  est[:L]-est[:L].mean()
    proj = (np.dot(est, ref)/np.dot(ref, ref))*ref
    return 10*np.log10(np.sum(proj**2)/np.sum((est-proj)**2)+1e-12)

S = stft(wav); mag = np.abs(S)
rec_full = istft(S)                                # keeps phase
est = mag*np.exp(1j*2*np.pi*np.random.default_rng(0).random(mag.shape))
for _ in range(50):                                # Griffin-Lim, phase discarded
    est = mag*np.exp(1j*np.angle(stft(istft(est))))
print(f"full complex STFT -> iSTFT    SI-SDR = {si_sdr(wav, rec_full):6.1f} dB")
print(f"magnitude + Griffin-Lim(50)  SI-SDR = {si_sdr(wav, istft(est)):6.1f} dB")

```

Measured output

```

full complex STFT -> iSTFT    SI-SDR =    84.8 dB
magnitude + Griffin-Lim(50)  SI-SDR =   -36.9 dB

```

Keep the phase and the STFT is essentially perfectly invertible (84.8 dB, limited only by float precision). Throw the phase away and estimate it with 50 Griffin-Lim iterations, and waveform-level fidelity collapses to −36.9 dB. That 120 dB gap is the entire reason neural vocoders, complex-spectrogram models, and phase predictors exist.

I have watched candidates confidently say “just invert the spectrogram” in a TTS design round. Then I ask: “which spectrogram — magnitude or complex?” The number above is why the distinction matters. If you only stored magnitude, you are not inverting anything; you are *guessing* phase, and the sample-level result is noise even when it sounds passable.

10.3 Lab 2 — Measuring ASR Error: WER Is Not CER

Goal. Use the project’s edit-distance `wer / cer` from Chapter 7 on realistic transcript pairs, and see where the two metrics disagree — the kind of nuance interviewers probe when they ask “why report both?”

lab2_wer.py

```

import numpy as np
from voice_agent.evaluate import wer, cer

pairs = [
    ("cancel my three pm meeting", "cancel my 3 pm meeting"),    # normalization
    ("book a flight to tokyo",      "book a flight to kyoto"),    # 1 wrong word
    ("what is my account balance",  "what is my account balance"),
    ("turn off the kitchen lights", "turn off the kitchen light"),# plural slip

```

```

    ("play the next song please", "play next song please"),    # deletion
]
ws, cs = [], []
for ref, hyp in pairs:
    w, c = wer(ref, hyp), cer(ref, hyp); ws.append(w); cs.append(c)
    print(f"WER={w:.3f} CER={c:.3f} | ref='{ref}' hyp='{hyp}'")
print(f"mean WER={np.mean(ws):.3f} mean CER={np.mean(cs):.3f}")

```

Measured output

```

WER=0.200 CER=0.227 | ref='cancel my three pm meeting' hyp='cancel my 3 pm meeting'
WER=0.200 CER=0.222 | ref='book a flight to tokyo' hyp='book a flight to kyoto'
WER=0.000 CER=0.000 | ref='what is my account balance' hyp='what is my account
↪ balance'
WER=0.200 CER=0.043 | ref='turn off the kitchen lights' hyp='turn off the kitchen
↪ light'
WER=0.200 CER=0.143 | ref='play the next song please' hyp='play next song please'
mean WER=0.160 mean CER=0.127

```

Look at the fourth row. “lights” versus “light” is one whole word wrong (WER 0.200) but only one character off (CER 0.043) — a 5x difference in how bad the error looks depending on the metric. Row one shows the opposite trap: “three pm” versus “3 pm” is a pure *normalization* difference that inflates both metrics even though the meaning is identical. This is why you normalize text before scoring, and why CER is kinder to morphological slips while WER punishes whole-word swaps.

My rule: report WER as the headline, but never ship on it alone. The booking error in row two (“tokyo” to “kyoto”) and a dropped “not” both score WER 0.2, yet one is harmless and the other reverses the user’s intent. If your product acts on entities, add an *entity error rate* on the slots that trigger actions. Aggregate WER is a thermometer, not a diagnosis.

10.4 Lab 3 — Endpointing Is Your Latency Budget

Goal. Run the Chapter 6 endpointer at three hangover settings and watch the end-of-turn latency move — the most user-visible knob in any voice agent.

lab3_endpoint.py

```

from voice_agent.audio_io import synth_utterance, detect_turns, VADConfig

sr = 16_000
wav = synth_utterance(seconds=1.6, sr=sr, seed=3)
for hang in (300, 600, 900):
    turns = detect_turns(wav, sr, VADConfig(hangover_ms=hang))
    captured = sum(len(t) for t in turns) / sr if turns else 0.0
    print(f"hangover={hang}ms -> {len(turns)} turn(s), captured {captured:.2f}s "
          f"(endpoint adds ~{hang}ms tail latency)")

```

Measured output

```
hangover=300ms -> 1 turn(s), captured 1.46s (endpoint adds ~300ms tail latency)
hangover=600ms -> 1 turn(s), captured 1.46s (endpoint adds ~600ms tail latency)
hangover=900ms -> 1 turn(s), captured 1.46s (endpoint adds ~900ms tail latency)
```

All three settings capture the same 1.46s of speech — the utterance is complete either way — but the agent cannot *know* the turn ended until the hangover timer expires. So the only thing the longer hangover buys here is latency: 300 ms versus 900 ms of dead air before the agent responds, with zero accuracy gain. That is the entire argument for semantic endpointing: when the transcript is already a complete thought, stop waiting.

When a voice agent “feels slow,” nine times out of ten it is not the model — it is this timer. The first thing I do on a sluggish agent is drop the hangover and add a completion check, not swap to a bigger GPU. People reach for model upgrades because they are exciting; the latency was sitting in a 700 ms silence timeout the whole time.

10.5 Where to Take These Next

These labs are deliberately CPU-only so they always run. To turn them into portfolio pieces: swap `MockRecognizer` for `faster-whisper` and re-run Lab 2 on a real LibriSpeech subset; feed Lab 3 real microphone frames through Silero VAD; and in Lab 1, replace Griffin-Lim with a HiFi-GAN vocoder and watch the SI-SDR gap close. Each is a weekend project, and each gives you a measured number to talk about — which, as the whole book argues, is what separates a confident answer from a memorized one.